



نسخه 1.8.0 · راهنمای کامل کاربر

Tehran Stock Exchange Data & Analytics Library

کتابخانه داده و تحلیل بازار سرمایه ایران

Stocks · Live Market · Options · ETFs · Debt · Gold & FX

سهام · بازار زنده · اختیار معامله · صندوق · اراد و گام · طلا و ارز

Python 3.8–3.14

13 September 2026

algotik.com

Mohsen Alipour · @algotik

راهنمای مطالعه

چطور از این دفترچه استفاده کنید؟

این راهنما دو لایه دارد: ابتدا مسیرهای سریع و توابعی که بیشتر کاربران هر روز نیاز دارند؛ سپس مرجع کامل ورودی‌ها، گزینه‌ها، خروجی‌ها و نکات رفتاری همهٔ API‌های عمومی پکیج.

۱

شروع سریع

نصب، import و اولین دریافت تاریخچهٔ قیمت در چند دقیقه.

۲

توابع پرکاربرد

قیمت، بازار زنده، سفارش، طلا و ارز، صندوق، اخزا، اراد، گام و اختیار.

۳

قراردادهای داده

نوع خروجی، تاریخ، freshness، ردیف امروز، CSV و مرز منابع داده.

۴

مرجع تفصیلی

signatureها، تمام ورودی‌ها، مقدارهای مجاز، پیش‌فرض‌ها و خروجی هر خانوادهٔ تابع.

نکتهٔ معاملاتی: پیش از استفاده در تصمیم‌گیری یا اجرای الگوریتم، زمان snapshot، ویژگی‌های `DataFrame.attrs`، وضعیت `partial` و freshness داده را بررسی کنید. این پکیج توصیهٔ سرمایه‌گذاری ارائه نمی‌کند.

A Python toolkit for historical, live and analytical data from Iran's capital market.

Fetch TSETMC prices, client type, trades, order books, market calendars, index impact, instrument reference data, funds, bonds, options, Iran Energy Exchange auctions, IME commodity markets and official subscriber TOP with Jalali date support, then use explainable comparison, liquidity, market-regime and market-map analytics for research and algorithmic trading.

IR معرفی فارسی

برای دریافت و تحلیل داده‌های بازار سرمایه ایران ساخته شده است. با نام فارسی نماد می‌توانید تاریخچه قیمت، `algotik-tse` حقیقی/حقوقی، معاملات ریز، سفارش‌ها و اطلاعات لحظه‌ای بازار را بگیرید؛ شاخص‌های صنعت و اعضای دقیق آن‌ها را تحلیل کنید؛ حراج و ابزارهای بورس انرژی و تابلوها و بازار فیزیکی بورس کالا را بخوانید؛ اراد، گام، اخزا، صندوق و اختیارها را بسازید؛ و دریافت کنید TGJU تاریخچه ۸۷ سری تأییدشده ارز، طلا، نقره، سکه و فلزات گران‌بها را از

بیشتر خروجی‌های جدولی به صورت **Pandas DataFrame** ارائه می‌شوند و تاریخ شمسی، تاریخ میلادی و داده‌های چندنمادی پشتیبانی می‌شوند. خروجی‌های ساختاریافته‌ای مانند snapshot بازار، زنجیره اختیار و منحنی بازده در بخش **نوع خروجی** توضیح داده شده‌اند.

ویژگی‌ها

- دریافت تاریخچه قیمت و حقیقی/حقوقی با نماد فارسی، تاریخ شمسی/میلادی، تعدیل قیمت، بازده و خروجی چندنمادی
- افزودن کنترل‌شده ردیف امروز با `include_today=True` و metadata مربوط به freshness و partial بودن داده
- نمای زنده کل بازار یا یک نماد، قدرت خریدار، جریان پول، spread و imbalance پنج سطح سفارش
- معاملات ریز، order book و صف خرید/فروش به صورت زنده و تاریخی
- تقویم معاملاتی بورس/فرا بورس، ارزش بازار تاریخی، آمار فعالیت روزانه و اثر هر نماد بر شاخص
- دریافت یکجای خلاصه روزانه همه ابزارها و فهرست مرجع کامل TSETMC با تشخیص تغییرات بر پایه `InsCode`
- قیمت تئوریک گشایش رسمی (TOP) برای مشترکان وب سرویس TSETMC و محاسبه شفاف عدم تعادل پیش‌گشایش
- SQLite جریان صنایع و ذخیره اختیاری تاریخچه روی breadth، بازار، پیام‌ها، تغییر وضعیت watcher
- ۴۵ شاخص صنعت، اعضای رسمی، snapshot تحلیلی، تاریخچه اعضا، کندل درون‌روزی، مقایسه و هم‌بستگی صنایع
- `InsCode` رخدادهای تعدیل قیمت و حل دقیق هویت ابزار با P/E و EPS
- سهامداران عمده هر نماد با تفکیک تاریخ معامله/تاریخ مؤثر، تغییر دارایی اصلاح‌شده و تاریخچه روزانه، هفتگی یا ماهانه با بودجه درخواست
- پنج‌روزه فعالیت سهامداران عمده کل بازار، تغییرات روزانه، رتبه‌بندی انباشت/توزیع، شبکه سهامدار-نماد، تمرکز snapshot هر نماد TSETMC رویدادهای timeline مالکیت و
- صندوق‌های ثبت‌شده و بورسی با طبقه‌بندی چندمحوره، پوشش ابزارهای ۳۰۵ و ۳۸۰ و فیلتر مستقیم طلا، نقره، کالایی، اهرمی، بخشی و شاخصی
- فهرست و فیلتر دقیق اوراق `اراد` و `گام` با سررسید شمسی/میلادی و جدول رسمی YTM فرا بورس، بدون حدس جریان نقدی

- تحلیل اخزا شامل YTM، duration، convexity، DV01 و منحنی بازده زنده و تاریخی
- تحلیل اختیار معامله شامل Black-Scholes، IV، Greeks، put-call parity، PCR و نقدشوندگی
- حراج‌های بورس انرژی با شرایط پرداخت و تحویل، حجم، دامنه قیمت، اندازه lot، قیمت کشف‌شده و معاملات همان حراج
- بازار برق استاندارد، سبز و آزاد؛ سلف، گواهی سپرده انرژی، گواهی ظرفیت، گواهی صرفه‌جویی و قرارداد آتی انرژی
- تابلوی رسمی بورس کالا برای گواهی سپرده، سلف استاندارد، صندوق کالایی و آتی همراه پنج سطح سفارش و حقیقی/حقوقی
- تاریخچه و خلاصه بازار فیزیکی بورس کالا، نسبت معامله به عرضه و آمار تجمیعی بازارهای فیزیکی، آتی، اختیار و مالی
- منحنی سررسید آتی، calendar spread و تحلیل cash-and-carry با فرض‌های صریح نرخ، انبارداری و convenience yield
- مقایسه هم‌زمان نمادها از نظر بازده، ریسک، افت سرمایه، نقدشوندگی، بتا و جریان پول حقیقی
- سنجه‌های نقدشوندگی تاریخی و زنده شامل Amihud، گردش، spread، عمق پنج‌سطحی و روزهای بدون معامله
- تشخیص توضیح‌پذیر وضعیت `neutral`، `risk_on` یا `risk_off` بازار با نمایش امتیاز و وزن هر مؤلفه
- داده و نمودار نقشه بازار با گروه‌بندی نماد، صنعت، بازار یا نوع ابزار و معیارهای قابل انتخاب اندازه/رنگ
- کاتالوگ ۸۷ سری ۳۶ TGJU: ارز آزاد، ۲۴ نرخ رسمی، طلای ۱۸/۲۴ عیار، مثقال و آب‌شده، نقره، سکه و حباب، و فلزات جهانی
- اینترادی، سهامداران، شاخص‌ها، ETFها، صندوق‌ها و اوراق بدهی
- و محدودسازی منبع داده `retry`، `rate limiting`، به‌صورت پیش‌فرض TLS و اعتبارسنجی HTTPS

وب‌سایت: algotik.com | تلگرام: t.me/algotik

این کتابخانه توصیه سرمایه‌گذاری نیست. برای استفاده معاملاتی، زمان `snapshot`، `freshness`، `partial` بودن داده و `DataFrame.attrs` را بررسی کنید.

API در یک نگاه

اگر اولین بار است از پکیج استفاده می‌کنید، معمولاً همین توابع نیاز شما را پوشش می‌دهند:

تابع	چه کاری انجام می‌دهد؟	خروجی
★ <code>get_history()</code>	تاریخچه OHLCV یک یا چند نماد	<code>DataFrame</code>
★ <code>get_client_type()</code>	تاریخچه خریدوفروش حقیقی/حقوقی	<code>DataFrame</code>
★ <code>get_live_symbol()</code>	نمای زنده و تحلیلی یک نماد	یک‌ردیفی <code>DataFrame</code>
★ <code>get_live_market()</code>	نمای زنده و تحلیلی کل بازار	<code>DataFrame</code>
★ <code>get_market_snapshot()</code>	خام قیمت و پنج سطح <code>snapshot</code> سفارش کل بازار	<code>dict</code>
★ <code>get_intraday()</code>	تیک یا کندل اینترادی	قدیمی API در <code>None</code> یا <code>DataFrame</code>

تابع	چه کاری انجام می‌دهد؟	خروجی
★ <code>get_symbols()</code>	فهرست نمادها بر اساس بازار و نوع ابزار	یا فهرست <code>DataFrame</code>
★ <code>get_tgju_history()</code>	تاریخچه طلا، ارز، نقره، سکه، حباب و فلزات جهانی	<code>DataFrame</code>
★ <code>list_tgju_assets()</code>	کاتالوگ نام‌ها، slugها، دسته و واحد ۸۷ سری TGJU	<code>DataFrame</code>
★ <code>list_debt_instruments()</code>	فهرست و فیلتر <code>gam</code> ، <code>erad</code> ، اخزا و سایر اوراق	<code>DataFrame</code>
★ <code>get_debt_yields()</code>	مرجع رسمی فرابورس برای اراد YTM و گام	<code>DataFrame</code>
★ <code>compare_symbols()</code>	مقایسه بازده، ریسک، نقدشوندگی و جریان پول چند نماد	<code>DataFrame</code>
★ <code>get_liquidity_metrics()</code>	سنجه‌های نقدشوندگی تاریخی و زنده	<code>DataFrame</code>
★ <code>get_market_regime()</code>	رژیم توضیح‌پذیر بازار و پنج امتیاز سازنده	یک‌ردیفی <code>DataFrame</code>
★ <code>get_market_map()</code>	داده مستقل از renderer برای نقشه بازار	<code>DataFrame</code>
★ <code>plot_market_map()</code>	نمودار تعاملی treemap با Plotly	<code>plotly.graph_objects.Figure</code>
★ <code>get_energy_auctions()</code>	حراج‌های بورس انرژی با شرایط کامل عرضه	<code>DataFrame</code>
★ <code>list_power_instruments()</code>	قراردادهای برق استاندارد، سبز و آزاد	<code>DataFrame</code>
★ <code>list_energy_securities()</code>	سلف، گواهی‌ها و آتی انرژی	<code>DataFrame</code>
★ <code>get_commodity_market()</code>	تابلوی گواهی، سلف، صندوق و آتی بورس کالا	<code>DataFrame</code>
★ <code>get_commodity_physical_history()</code>	حجم و ارزش روزانه بازار فیزیکی بورس کالا	<code>DataFrame</code>
★ <code>get_commodity_physical_summary()</code>	خلاصه تالارها و نسبت معامله به عرضه	<code>DataFrame</code>
★ <code>get_futures_curve()</code>	منحنی سررسید و basis قراردادهای آتی	<code>DataFrame</code>
★ <code>get_live_trades()</code>	معاملات ریز امروز یک نماد	<code>DataFrame</code>
★ <code>get_order_book()</code>	پنج سطح سفارش زنده	<code>DataFrame</code>
★ <code>get_trading_calendar()</code>	تقویم معاملاتی رسمی بورس و فرابورس	<code>DataFrame</code>
★ <code>get_market_activity()</code>	تعداد، حجم و ارزش معاملات روزانه کل بازار	<code>DataFrame</code>

تابع	چه کاری انجام می‌دهد؟	خروجی
★ <code>get_market_value_history()</code>	تاریخچه ارزش بازار بورس و فرابورس	DataFrame
★ <code>get_index_impact()</code>	نمادهای اثرگذار مثبت و منفی روی شاخص	DataFrame
★ <code>get_market_trades()</code>	خلاصه روزانه همه ابزارها در یک درخواست	DataFrame
★ <code>get_shareholders()</code>	سهامداران عمده فعلی یا پس از پایان یک روز معاملاتی	DataFrame
★ <code>get_shareholder_history()</code>	تاریخچه ترکیب سهامداران عمده یک نماد با درخواست‌های جفتی	DataFrame
★ <code>get_major_shareholder_changes()</code>	تغییرات سهامداران عمده در پنجره پنج‌روزه TSETMC	DataFrame
★ <code>get_active_shareholders()</code>	جمع‌بندی انباشت و توزیع سهامداران فعال	DataFrame
★ <code>rank_shareholder_accumulation()</code>	رتبه‌بندی فعالیت هر جفت سهامدار-نماد بر پایه درصد یا تعداد سهم	DataFrame
★ <code>get_shareholder_network()</code>	شبکه دویخشی سهامداران و نمادها در آخرین snapshot	DataFrame edge list
★ <code>get_ownership_concentration()</code>	سهامداران عمده یک HHI و Top-N نماد	یک‌ردیفی DataFrame
★ <code>get_symbol_events()</code>	پیام ناظر، افزایش سرمایه و timeline تعدیل قیمت یک نماد	DataFrame
★ <code>get_industry_snapshot()</code>	بازده، breadth، جریان پول و صف یک یا همه صنایع	DataFrame
★ <code>compare_industries()</code>	مقایسه سری زمانی چند شاخص صنعت	DataFrame
★ <code>get_industry_relative_strength()</code>	محاسبه برتری نسبی به مقابل شاخص مبنا	DataFrame
★ <code>get_industry_correlation()</code>	ماتریس همبستگی بازدهی روزانه صنایع	DataFrame
★ <code>get_industry_membership_overlap()</code>	اندازه‌گیری همپوشانی رسمی اعضای صنایع	DataFrame
★ <code>get_industry_membership_events()</code>	ردیف‌های <code>added</code> / <code>dropped</code> عضویت رسمی صنعت	DataFrame
★ <code>get_industry_membership_churn()</code>	شاخص تغییرپذیری اعضا بین جلسات اخیر	DataFrame
★ <code>get_industry_concentration()</code>	شاخص تمرکز و متمرکز شدن سرمایه صنعت	DataFrame

تابع	چه کاری انجام می‌دهد؟	خروجی
★ <code>get_industry_momentum_profile()</code>	پروفایل مومنتوم چنددوره‌ای برای صنایع	DataFrame
★ <code>get_industry_correlation_neighborhood()</code>	همبستگی همسایگی یک صنعت با نزدیک‌ترین‌ها	DataFrame
★ <code>get_industry_health_score()</code>	نمره سلامت ترکیبی برای رتبه‌بندی صنایع	DataFrame
★ <code>rank_industries()</code>	رتبه‌بندی صنایع با معیار انتخابی	DataFrame
★ <code>list_etfs()</code>	NAV ها همراه قیمت و ETF	DataFrame
★ <code>list_funds()</code>	صندوق‌ها همراه NAV، بازده و ترکیب دارایی	DataFrame
★ <code>list_listed_funds()</code>	صندوق‌های بورسی و فیلتر نوع دارایی، استراتژی و کالای پایه	DataFrame

نقشه کامل توابع کاربری

جدول‌های زیر API‌های canonical را نشان می‌دهند. نام‌های قدیمی مانند `stock()` و `stock_RI()` همچنان کار می‌کنند، اما برای کد جدید نام‌های `get_*` پیشنهاد می‌شوند.

قیمت، حقیقی/حقوقی و اطلاعات نماد

تابع	کاربرد
<code>get_history()</code>	تاریخچه قیمت تعدیل‌شده/خام، بازده و داده چندنمادی
<code>get_client_type()</code>	تاریخچه حقیقی/حقوقی و قدرت خریدار
<code>get_intraday()</code>	تیک و کندل‌های ۱ دقیقه تا ۱۲ ساعت
<code>get_trades()</code> , <code>get_live_trades()</code>	معاملات ریز تاریخی و امروز
<code>get_detail()</code> , <code>get_info()</code> , <code>get_stats()</code>	جزئیات، اطلاعات و آمار TSETMC نماد
<code>get_shareholders()</code>	سهامداران عمده فعلی یا تاریخی
<code>get_shareholder_history()</code>	های تاریخی یک نماد روی تاریخ مؤثر، با فرکانس و بودجه snapshot درخواست
<code>get_major_shareholder_snapshots()</code>	های منتشرشده موارد فعال در تابلوی تغییرات پنج‌روزه snapshot
<code>get_major_shareholder_changes()</code>	اختلاف دارایی بین snapshot‌های متوالی سهامداران عمده
<code>get_active_shareholders()</code>	جمع‌بندی ورود/خروج موارد فعال در پنجره اخیر
<code>rank_shareholder_accumulation()</code>	رتبه‌بندی بدون جمع‌زدن تعداد سهم نمادهای ناهمگون
<code>get_shareholder_network()</code>	اخیر snapshot شبکه سهامدار-ابزار در یک edge list
<code>get_ownership_concentration()</code>	تمرکز فهرست افشاشده سهامداران عمده یک نماد

تابع	کاربرد
<code>get_capital_increase()</code>	تاریخچه افزایش سرمایه
<code>get_price_adjustments()</code> , <code>get_latest_price_adjustment()</code>	رخدادهای تعدیل قیمت
<code>get_introduction()</code>	فقط سازگاری قدیمی؛ همیشه <code>UnsupportedDataSourceError</code>

داده زنده، سفارش و تحلیل بازار

تابع	کاربرد
<code>get_market_snapshot()</code>	خام و اتمیک کل بازار snapshot
<code>get_market_client_type()</code>	حقیقی/حقوقی bulk کل بازار
<code>get_live_market()</code> , <code>get_live_symbol()</code>	نمای زنده ادغام شده بازار یا یک نماد
<code>get_order_book()</code> , <code>get_queue()</code>	پنج سطح سفارش و صف زنده
<code>get_order_book_history()</code> , <code>get_queue_history()</code>	تاریخچه بازسازی شده سفارش و صف
<code>get_market_messages()</code>	پیام های ناظر بازار
<code>get_instrument_state_changes()</code>	تغییر وضعیت ابزارها
<code>get_symbol_events()</code>	برای پیام ناظر، تغییر سرمایه و تعدیل قیمت یک نماد TSETMC بومی timeline
<code>get_market_overview()</code>	نمای کلی رسمی بازار
<code>get_market_breadth()</code>	و شمار نمادهای مثبت/منفی A/D، breadth
<code>get_sector_flow()</code>	و جریان پول به تفکیک صنعت breadth
<code>watch_market()</code> , <code>MarketWatcher</code>	پایش افزایشی بازار و تولید <code>MarketEvent</code>
<code>get_trading_calendar()</code>	جلسات معاملاتی منتشر شده بورس و فرابورس
<code>get_market_activity()</code>	تجمیع روزانه تعداد ابزار، معامله، حجم و ارزش
<code>get_market_value_history()</code>	تاریخچه رسمی ارزش بازار با تغییر روزانه
<code>get_index_impact()</code>	اثر نمادها بر شاخص کل هر بازار attribution
<code>get_market_trades()</code>	خلاصه bulk روزانه ابزارها؛ معاملات ریز نیست
<code>get_theoretical_opening_price()</code>	TSETMC رسمی برای مشترکان وب سرویس TOP
<code>get_preopen_imbalance()</code>	عدم تعادل حجم خرید و فروش روی خروجی TOP
<code>compare_symbols()</code>	مقایسه چند نماد با benchmark از نظر بازده، ریسک، افت سرمایه، نقدشوندگی و جریان حقیقی

تابع	کاربرد
<code>get_liquidity_metrics()</code>	نقدشوندگی تاریخی و snapshot زنده با Amihud، گردش، spread، imbalance و عمق سفارش
<code>get_market_regime()</code>	طبقه‌بندی توضیح‌پذیر <code>risk_on/neutral/risk_off</code> از پنج مؤلفه قابل مشاهده
<code>get_market_map()</code>	ساخت جدول نقشه بازار برای نماد، صنعت، flow یا نوع ابزار
<code>plot_market_map()</code>	رندر treemap تعاملی از جدول نقشه با وابستگی اختیاری Plotly

شاخص‌ها و تحلیل صنایع

تابع	کاربرد
<code>list_industry_indices()</code>	فهرست ۴۵ شاخص صنعت و وضعیت فعلی آن‌ها
<code>get_industry_members()</code>	اعضای رسمی یک شاخص با اتصال دقیق <code>InsCode</code> به داده زنده
<code>get_industry_snapshot()</code>	یک ردیف تحلیلی برای هر صنعت: شاخص، breadth، معامله، حقیقی و صف
<code>get_industry_history()</code>	تاریخچه روزانه خود شاخص صنعت بدون حجم ساختگی
<code>get_industry_members_history()</code>	تاریخچه کوتاه همه اعضای فعلی صنعت در قالب long-form
<code>get_industry_intraday()</code>	مشاهدات خام یا کندل‌های درون‌روزی شاخص صنعت
<code>compare_industries()</code>	مقایسه سری زمانی شاخص‌ها و متریک‌های قیمت/بازده
<code>get_industry_relative_strength()</code>	محاسبه بازده تجمعی نسبی هر شاخص در برابر benchmark
<code>get_industry_correlation()</code>	ماتریس همبستگی بازده روزانه بین چند صنعت
<code>get_industry_membership_overlap()</code>	همپوشانی رسمی اعضای چند شاخص صنعت را pairwise محاسبه می‌کند
<code>get_industry_membership_events()</code>	رویدادهای <code>added</code> / <code>dropped</code> تغییرات رسمی اعضا
<code>get_industry_membership_churn()</code>	اعضای رسمی صنعت طی چند جلسه اخیر churn
<code>get_industry_concentration()</code>	تمرکز روی چند عضو بزرگ و وزن سهمی، HHI
<code>get_industry_momentum_profile()</code>	پروفایل مومنتوم چند پنجره‌ای برای هر صنعت
<code>get_industry_correlation_neighborhood()</code>	نزدیک‌ترین همبستگی‌ها برای یک صنعت هدف
<code>get_industry_health_score()</code>	نمره سلامت ترکیبی breadth، momentum و concentration
<code>rank_industries()</code>	رتبه‌بندی صنایع بر اساس بازده، breadth، ارزش یا جریان پول

تاریخچه محلی و فاندمنتال

تابع	کاربرد
<code>save_market_snapshot()</code> , <code>load_market_snapshots()</code>	ذخیره و خواندن snapshot‌های SQLite

تابع	کاربرد
<code>get_live_market_history()</code>	تاریخچه نمای زنده ذخیره شده
<code>get_market_overview_history()</code>	تاریخچه overview رسمی
<code>get_market_snapshot_summary_history()</code>	خلاصه snapshot های ذخیره شده
<code>get_market_breadth_history()</code> , <code>get_sector_flow_history()</code>	تاریخچه تحلیل بازار و صنایع
<code>record_market_event()</code> , <code>get_market_event_history()</code>	ثبت و replay رخداد های watcher
<code>archive_market_records()</code>	آرشیو batch رکوردهای مستقل
<code>get_market_messages_history()</code>	پیام های آرشیو شده بازار
<code>get_instrument_state_changes_history()</code>	تغییر وضعیت های آرشیو شده
<code>check_market_history()</code>	بررسی سلامت و سازگاری فایل SQLite
<code>get_market_fundamentals()</code>	بازار EPS و P/E snapshot
<code>get_market_fundamentals_history()</code>	تاریخچه EPS/P/E بدون look-ahead

فهرست ابزارها و بازارها

تابع	کاربرد
<code>get_symbols()</code>	فهرست سهام، حق تقدم، صندوق، اوراق و اختیار
<code>get_instrument_master()</code>	جدول مرجع کامل ابزارها با ISIN و <code>InsCode</code>
<code>get_instrument_changes()</code>	مقایسه دو snapshot مرجع و گزارش افزوده/حذف/تغییر
<code>list_options()</code>	فهرست قراردادهای اختیار فعال
<code>get_options_chain()</code>	زنجیره call/put یک دارایی پایه
<code>list_etfs()</code>	مسیر قدیمی صندوق های بورسی عمومی نوع ۳۰۵ همراه NAV و discount/premium
<code>list_funds()</code>	صندوق های ثبتی با NAV، بازده، ترکیب دارایی و دسته/استراتژی قابل فیلتر
<code>list_listed_funds()</code>	ابزارهای بورسی دقیق، شامل صندوق های عمومی و کالایی با فیلتر طلا/نقره
<code>list_bonds()</code>	اوراق بدهی همراه سررسید
<code>list_debt_instruments()</code>	نمای جدید اوراق با <code>DebtType</code> دقیق و فیلتر <code>erad / gam</code>
<code>list_indices()</code> , <code>get_index_companies()</code>	های بالا پیشنهاد می شوند API قدیمی شاخص ها و اعضای شاخص؛ برای صنعت API
<code>list_tgju_assets()</code>	کاتالوگ تأیید شده دارایی های TGJU، دسته و واحد هر سری
<code>get_tgju_history()</code> , <code>get_currency()</code>	تاریخچه ارز، طلا، نقره، سکه/حباب و فلزات جهانی؛ نام قدیمی حفظ شده است

بورس انرژی و بورس کالا

تابع	کاربرد
<code>get_energy_auctions()</code>	فهرست عرضه‌های آماده، فعال، مازاد یا پایان‌یافته در تابلوهای فیزیکی و برق
<code>get_energy_auction()</code>	جزئیات یک حراج، معاملات خود حراج و در صورت درخواست سابقه همان ابزار
<code>get_energy_market_overview()</code>	آمار روزانه جریان‌های رسمی بورس انرژی و بازار برق
<code>list_power_instruments()</code>	برق استاندارد، برق سبز و برق آزاد
<code>list_energy_securities()</code>	سلف استاندارد، گواهی ظرفیت/صرفه‌جویی/سپرده و آتی انرژی
<code>get_energy_future_contract()</code>	مشخصات قرارداد، سررسید، وجه تضمین، کارمزد و قواعد تحویل آتی انرژی
<code>get_commodity_market()</code>	برای گواهی، سلف، صندوق کالایی و آتی IME رسمی snapshot
<code>get_commodity_physical_history()</code>	سری روزانه حجم و ارزش بازار فیزیکی در بازه صریح
<code>get_commodity_physical_summary()</code>	تجمیع تالارهای فیزیکی و نسبت معامله به عرضه
<code>get_commodity_market_activity()</code>	فعالیت تجمیعی بازار فیزیکی، آتی، اختیار، مالی و گواهی
<code>get_futures_curve()</code>	ساخت term structure از ردیف‌های صریح قرارداد
<code>get_calendar_spreads()</code>	contango/backwardation سررسیدهای مجاور و تشخیص spread
<code>analyze_cash_and_carry()</code>	مقایسه قیمت آتی و ارزش منصفانه با فرض‌های صریح هزینه حمل

اخزا و درآمد ثابت

تابع	کاربرد
<code>parse_treasury_maturity()</code>	استخراج سررسید از نماد اخزا
<code>day_count_fraction()</code>	محاسبه فاصله زمانی با day-count convention
<code>treasury_yield()</code>	بازده اخزا از قیمت و سررسید
<code>bond_price()</code> , <code>yield_to_maturity()</code>	قیمت اوراق و حل YTM
<code>bond_analytics()</code>	duration، convexity و DV01
<code>build_yield_curve()</code>	ساخت شیء <code>YieldCurve</code> از nodeها
<code>get_ifb_yield_table()</code>	جدول مرجع YTM فرابورس
<code>get_debt_yields()</code>	جدول رسمی همه اوراق با فیلتر دقیق <code>erad</code> ، <code>treasury</code> و <code>gam</code>
<code>get_treasury_yields()</code>	اخزا و تحلیل بازده snapshot
<code>get_treasury_yield_history()</code>	تاریخچه YTM اخزا
<code>get_yield_curve()</code> , <code>get_yield_curve_history()</code>	منحنی بازده زنده و تاریخی

اختیار معامله

تابع	کاربرد
<code>black_scholes_price()</code>	قیمت بلک-شولز اروپایی
<code>black_scholes_greeks()</code>	Delta, Gamma, Vega, Theta و Rho
<code>option_price_bounds()</code>	کران‌های بدون آربیتراژ
<code>implied_volatility()</code>	حل نوسان ضمنی
<code>get_option_market()</code>	اتمیک بازار اختیار snapshot
<code>analyze_option_chain()</code>	و نقدشوندگی IV, Greeks, parity
<code>option_put_call_ratios()</code>	حجم، ارزش و موقعیت باز PCR
<code>get_option_history()</code>	تاریخچه قرارداد اختیار
<code>save_option_snapshot()</code> , <code>load_option_snapshots()</code>	ذخیره و خواندن snapshot های JSON

هویت ابزار و ابزارهای کمکی

تابع	کاربرد
<code>resolve_instrument()</code>	تبدیل نماد یا <code>InsCode</code> به <code>InstrumentRef</code> دقیق
<code>validate_ins_code()</code>	اعتبارسنجی شناسه ابزار
<code>normalize_instrument_text()</code>	یکسان‌سازی ی/ک و فاصله‌های فارسی

برای signature و همه ورودی‌های هر تابع به مرجع [تفصیلی همه توابع](#) مراجعه کنید.

فهرست مطالب

- نصب و به‌روزرسانی
- شروع سریع
- راهنمای توابع پرکاربرد
- قراردادهای مهم داده
- حل دقیق هویت نماد
- قیمت و حقیقی/حقوقی
- معاملات ریز
- سفارش و صف
- و تحلیل کل بازار `Watcher`

- تقویم، نمای کلان، مستر ابزار و پیش‌گشایش
- تحلیل مدیریتی بازار
- شاخص‌ها و تحلیل صنایع
- تاریخچهٔ محلی SQLite
- فاندمنتال، صندوق و تعدیل قیمت
- اخزا و درآمد ثابت
- اختیار معامله
- بورس انرژی و بورس کالا
- سایر API‌های بازار
- تنظیمات و خطاها
- مرجع تفصیلی همهٔ توابع
- مثال‌های کاربردی
- تست و مشارکت
- تاریخچهٔ تغییرات

نصب و به‌روزرسانی

```
pip install algotik-tse
```

برای ارتقا به آخرین نسخه:

```
python -m pip install --upgrade algotik-tse
```

نیازمندی نسخهٔ فعلی: **3.8** Python تا **3.14**.

برای نصب نسخهٔ توسعه:

```
git clone https://github.com/mohsenalipour/algotik_tse.git
cd algotik_tse
python -m pip install -e "[dev]"
```

شروع سریع

```
import algotik_tse as att
```

نیاز شما	تابع پیشنهادی
تاریخچه قیمت تعدیل شده	<code>att.get_history("شتران")</code>
حقیقی/حقوقی یک نماد	<code>att.get_client_type("شتران")</code>
اطلاعات زنده یک نماد	<code>att.get_live_symbol("شتران")</code>
لحظه ای کل بازار snapshot	<code>att.get_market_snapshot()</code>
کندل های اینترادی	<code>att.get_intraday("شتران", interval="5min")</code>
معاملات ریز امروز	<code>att.get_live_trades("شتران")</code>
پنج سطح سفارش	<code>att.get_order_book("شتران")</code>
تقویم روزهای معاملاتی	<code>att.get_trading_calendar(market="tse")</code>
ارزش و فعالیت تاریخی بازار	<code>att.get_market_value_history()</code> و <code>att.get_market_activity()</code>
اثرگذارترین نمادهای شاخص	<code>att.get_index_impact(top=10)</code>
خلاصه همه ابزارهای یک روز	<code>att.get_market_trades("1405-06-18")</code>
تصویر تحلیلی صنایع	<code>att.get_industry_snapshot()</code>
رتبه بندی صنایع	<code>att.rank_industries(metric="breadth")</code>
فهرست نمادها و ابزارها	<code>att.get_symbols()</code>
طلای ۱۸/۲۴ عیار، ارز و سکه	<code>att.get_tgju_history("gold-18k")</code>
دیدن همه سری های TGJU	<code>att.list_tgju_assets()</code>
صندوق های ETF با NAV	<code>att.list_etfs()</code>
اخزا و YTM	<code>att.get_treasury_yields()</code>
اراد و گام	<code>att.list_debt_instruments(["erad", "gam"])</code>
مرجع اراد و گام YTM	<code>att.get_debt_yields(["erad", "gam"])</code>
زنجیره و تحلیل اختیار	<code>att.get_option_market()</code> و <code>att.analyze_option_chain()</code>
عرضه های بورس انرژی	<code>att.get_energy_auctions(top=10)</code>
برق سبز، استاندارد و آزاد	<code>att.list_power_instruments()</code>
سلف و گواهی های انرژی	<code>att.list_energy_securities()</code>
تابلوی بورس کالا	<code>att.get_commodity_market(kind="certificate")</code>

نیاز شما	تابع پیشنهادی
سابقه بازار فیزیکی کالا	<code>att.get_commodity_physical_history("1405-01-01")</code>
calendar spread و منحنی آتی	<code>att.get_futures_curve(...)</code> و <code>att.get_calendar_spreads(...)</code>

اولین دریافت: تاریخچه قیمت

```
prices = att.get_history(
    "شتران",
    start="1403-01-01",
    end="1403-03-31",
    progress=False,
)
print(prices.tail())
```

خروجی نماینده:

J-Date	Open	High	Low	Close	Volume
1403-03-26	2630	2680	2605	2668	82471422
1403-03-27	2670	2715	2641	2692	70938510
1403-03-28	2695	2734	2670	2718	93612045

قیمت‌ها در حالت پیش‌فرض تعدیل می‌شوند. برای داده خام از `auto_adjust=False` و برای ستون‌های کامل‌تر از `output_type="full"` استفاده کنید.

چند کاربرد رایج در یک نگاه

```
# حقیقی/حقوقی ۳۰ روز اخیر
client_type = att.get_client_type("فملی", limit=30, progress=False)

# است opt-in امروز؛ این رفتار observation تاریخچه به همراه
prices_today = att.get_history(
    "فملی", limit=20, include_today=True, progress=False
)

# نمای زنده نماد و قدرت خریدار
live = att.get_live_symbol("فملی", fallback="none")
print(live[["Symbol", "Last", "Close", "IndividualPower"]])

# پنج سطح سفارش، صف و معاملات امروز
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
today_trades = att.get_live_trades("فملی")

# ابزارهای بازار
etfs = att.list_etfs()
funds = att.list_funds(fund_type="fixed_income")

# اخزا و اختیار معامله
treasuries = att.get_treasury_yields(min_volume=1)
options = att.get_option_market(underlying="خودرو")
analytics = att.analyze_option_chain(options, risk_free_rate=0.30)

# بورس انرژی: عرضه‌های آماده و قراردادهای برق سبز
auctions = att.get_energy_auctions(status="upcoming", board="physical", top=10)
green_power = att.list_power_instruments(market="green")

# بورس کالا: گواهی‌های سپرده با پنج سطح سفارش و حقیقی/حقوقی
certificates = att.get_commodity_market(kind="certificate")

# سابقه بازار فیزیکی و نسبت معامله به عرضه هر تالار
physical = att.get_commodity_physical_history("1405-01-01", "1405-06-01")
halls = att.get_commodity_physical_summary("1405-01-01", "1405-06-01")
```

خروجی نماینده `get_live_symbol()` در زمان بازار:

	Symbol	Last	Close	IndividualPower
0	1.31		73950	74200 فملی

مقادیر مثال‌های متصل به شبکه «خروجی نماینده» هستند و با زمان بازار تغییر می‌کنند. مثال‌های ریاضی deterministic هستند و در تست‌های آفلاین کنترل می‌شوند. ستون‌های `Last` و `Close` در feed زنده به ترتیب «آخرین معامله» و «قیمت پایانی» هستند؛ تفاوت نام‌گذاری live و history در بخش بعد آمده است.

سند مرجع واحد پروژه است. اگر تازه شروع کرده‌اید، بخش راهنمای توابع پرکاربرد را بخوانید؛ [مرجع تفصیلی همه README.md](#) ورودی، خروجی و قرارداد هر تابع است و لازم نیست از ابتدا تا انتها خوانده شود، signature دقیق lookup توابع برای

راهنمای توابع پرکاربرد

این بخش برای استفاده روزمره است. ورودی‌های هر تابع کنار همان تابع توضیح داده شده‌اند؛ پس از آن، مرجع تخصصی تمام API‌ها قرار دارد.

get_history() – تاریخچه قیمت

```
get_history(
    symbol="", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
symbol	نماد فارسی یا فهرست نمادها؛ مانند "فملی" یا ["فملی", "شتران"] .
start , end	ابتدا و انتهای شامل بازه؛ تاریخ شمسی 1403-01-01 یا میلادی 2024-03-20 .
limit	تعداد آخرین ردیف‌های معاملاتی؛ 0 یعنی همه تاریخچه موجود.
raw	با True نام ستون‌ها به فرمت نزدیک TSETMC برمی‌گردد.
auto_adjust	پیش‌فرض True ؛ OHLC را با سری تعدیل‌شده بازسازی می‌کند.
output_type	و اطلاعات بیشتر Value , No. , Final برای "full" یا OHLCV برای "standard"
date_format	"both" یا "gregorian" ، "jalali"
progress	نمایش پیام پیشرفت؛ روی داده خروجی اثری ندارد.
save_to_file	ذخیره CSV را فعال می‌کند.
save_path	پوشه مقصد CSV؛ نام فایل از نماد ساخته می‌شود.
dropna	در خروجی چندنمادی ستون‌های کاملاً تهی را حذف می‌کند.
adjust_volume	حجم را نیز متناسب با ضریب افزایش سرمایه تعدیل می‌کند.
return_type	["simple", "Close", 5] یا فرم سفارشی مانند "both" ، "log" ، "simple"
ascending	جدید به قدیمی False قدیمی به جدید؛ True
include_today	با observation True معتبر امروز را به صورت opt-in اضافه/جایگزین می‌کند.
ins_code	شناسه دقیق ابزار؛ برای نمادهای تکراری و production پیشنهاد می‌شود.
asset_type	"equity" ، "index" ، "bond" نوع ابزار مانند hint یا "auto"

ورودی	گزینه‌ها و معنی
**kwargs	فقط aliasهای قدیمی مستند مانند stock , values , tse_format ; کلید ناشناخته API عمومی نیست.

خروجی: **DataFrame** تک‌نمادی یا **DataFrame** با ستون‌های MultiIndex برای چند نماد.

get_client_type() – حقیقی/حقوقی

```
get_client_type(
    symbol="", start=None, end=None, limit=0, raw=False,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, ascending=True, save_path=None,
    include_today=False, *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
symbol , ins_code , asset_type	انتخاب نماد؛ قواعد آن‌ها مانند get_history() است.
start , end , limit	بازه یا تعداد آخرین روزهای معاملاتی.
raw	خروجی نزدیک به schema خام provider.
output_type	سرانه و قدرت را نیز نگه می‌دارد، value "full" ؛ حالت کامل یا "standard"
date_format	"both" یا "gregorian" ، "jalali"
include_today	ردیف حقیقی/حقوقی امروز را با provenance و برچسب برآوردی بودن اضافه می‌کند.
progress , dropna , ascending	نمایش پیشرفت، حذف ستون تهی چندنمادی و ترتیب زمانی.
save_to_file , save_path	ذخیره CSV و پوشه مقصد.
**kwargs	limit=30 به جای values=30 های قدیمی مانند alias

خروجی: **DataFrame** روزانه تعداد/حجم/ارزش خریدوفروش حقیقی و حقوقی و شاخص‌های مشتق‌شده.

get_live_symbol() و get_live_market() – داده زنده

```
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_live_market(symbol=None, *, strict=False)
```

تابع/ورودی	گزینه‌ها و معنی
get_live_symbol.symbol	یک نماد فارسی یا InsCode .
get_live_symbol.ins_code	شناسه دقیق keyword-only؛ بر selector مبهم ترجیح دارد.
fallback	نقطه‌ای استفاده می‌کند endpoint در نبود نماد از "point" ؛ MarketWatch فقط "none"

تابع/ورودی	گزینه‌ها و معنی
<code>get_live_market.symbol</code>	برای کل بازار، یک نماد یا فهرست نمادها <code>None</code>
<code>strict</code>	با <code>False</code> نماد گم‌شده در <code>attrs['missing_selectors']</code> ثبت می‌شود؛ با <code>True</code> خطا می‌دهد.

خروجی: `get_live_symbol()` یک `DataFrame` یک‌ردیفی و `get_live_market()` یک `DataFrame` فیلترشده یا کل بازار می‌دهد. ستون‌های مهم شامل `SpreadBps`, `EstimatedNetIndividualFlow`, `IndividualPower`, `Close`, `Last` و `L5Imbalance` هستند.

`get_market_snapshot()` و `feed` – `get_market_client_type()` خام `bulk`

```
get_market_snapshot()
get_market_client_type()
```

این دو تابع ورودی کاربری ندارند. `get_market_snapshot()` یک `dict` شامل `order_book`, `stocks`, زمان بازار و `metadata` تازگی می‌دهد. `get_market_client_type()` یک `bulk DataFrame` حقیقی/حقوقی کل بازار برمی‌گرداند. برای مصرف معمول، `get_live_market()` نسخهٔ ادغام‌شده و راحت‌تر است.

`get_intraday()` – تیک و کندل

```
get_intraday(
    symbol="شتران", interval="1min",
    start=None, end=None, progress=True, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code>	نماد فارسی.
<code>interval</code>	"1m", هایی مانند "12h", "1h", "4h", "30min", "15min", "5min", "1min", "tick", نیز پذیرفته می‌شوند "720", "240m", "60min".
<code>start</code> , <code>end</code>	بدون مقدار، دادهٔ امروز؛ با تاریخ، snapshot‌های تاریخی بازه.
<code>progress</code>	نمایش پیشرفت.
<code>**kwargs</code>	فقط سازگاری نام‌های قدیمی.

خروجی: در حالت `tick` دادهٔ معامله و در حالت `candle` ستون‌های `Open`, `High`, `Low`, `Close`, `Volume`, `TradeCount`. این API قدیمی برای ورودی نامعتبر ممکن است پیام چاپ کند و `None` بدهد.

فهرست ابزارها – `get_symbols()`

```
get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False,
    options=False, mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=True, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>bourse</code> , <code>farabourse</code> , <code>payeh</code>	بازارهای سهام که به صورت پیش فرض فعال‌اند.
<code>haghe_taqadom</code>	افزودن حق تقدم‌ها.
<code>sandogh</code>	افزودن صندوق‌ها.
<code>bonds</code>	افزودن اوراق بدهی.
<code>options</code>	افزودن اختیار معامله‌ها.
<code>mortgage</code>	افزودن اوراق تسهیلات مسکن.
<code>commodity</code>	افزودن گواهی‌های کالایی.
<code>energy</code>	افزودن ابزارهای انرژی.
<code>payeh_color</code>	"قرمز" , "نارنجی" , "زرد" یا فهرستی از آن‌ها.
<code>output</code>	یا خروجی فهرستی قدیمی <code>"dataframe"</code>
<code>progress</code>	نمایش پیشرفت.
<code>**kwargs</code>	نام‌های انگلیسی قدیمی فیلترها.

خروجی: فهرست ابزارها با هویت، نماد، نام، بازار و نوع دارایی.

`get_currency()` و `get_tgju_history()` – طلا، ارز، سکه و فلزات

```
get_currency(
    name="", start=None, end=None, limit=0,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, return_type=None,
    ascending=True, save_path=None, **kwargs,
)
get_tgju_history(...) # همان قرارداد؛ نام پیشنهادی برای همه دسته‌های
list_tgju_assets(category=None, include_aliases=False)
```

ورودی	گزینه‌ها و معنی
<code>name</code>	یک نام canonical، نام فارسی، slug رسمی یا فهرست؛ مانند، "طلای ۲۴ عیار"، "gold-18k"، "geram18"، "dollar"، "seke".
<code>start</code> ، <code>end</code> ، <code>limit</code>	بازه یا آخرین تعداد ردیف.
<code>output_type</code>	قدیمی API یا حالت کامل پشتیبانی‌شده "standard".
<code>date_format</code>	"jalali"، "gregorian"، "both".
<code>return_type</code>	محاسبه بازده ساده/لگاریتمی.
<code>progress</code> ، <code>dropna</code> ، <code>ascending</code>	پیشرفت، خروجی چندارزی و ترتیب زمانی.
<code>save_to_file</code> ، <code>save_path</code>	ذخیره CSV و پوشه مقصد.
<code>**kwargs</code>	های سازگاری قدیمی alias.
<code>category</code>	در <code>list_tgju_assets()</code> یکی یا فهرستی از، <code>currency</code> ، <code>official_rate</code> ، <code>gold</code> ، <code>silver</code> ، <code>coin</code> ، <code>coin_bubble</code> ، <code>global_metal</code> .
<code>include_aliases</code>	افزودن tuple نام‌های جایگزین به کاتالوگ.

خروجی: تاریخچه `DataFrame[Open, High, Low, Close]` است؛ چند دارایی ستون‌های MultiIndex می‌سازد. کاتالوگ ستون‌های `Name`، `PersianName`، `Slug`، `Category`، `Unit` دارد. واحد سری در `df.attrs['unit']` هم ثبت می‌شود.

گرفتن معاملات — `get_live_trades()` و `get_trades()`

```
get_trades(
    symbol=None, *, ins_code=None, start=None, end=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
get_live_trades(
    symbol=None, *, ins_code=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code> ، <code>ins_code</code>	انتخاب نماد با نام یا شناسه دقیق.
<code>start</code> ، <code>end</code>	بازه تاریخی شامل دو سر؛ فقط در <code>get_trades()</code> .
<code>include_canceled</code>	نگه‌داشتن معاملات ابطالی.
<code>max_requests</code>	سقف سخت تعداد درخواست‌ها؛ برای بازه‌های بزرگ حتماً تعیین کنید.
<code>raw</code>	افزودن ستون‌های audit نزدیک provider.
<code>progress</code>	نمایش پیشرفت.

خروجی: `DataFrame` معاملات با `TradeNo`, `Timestamp`, `Price`, `Volume`, `Value`, `Canceled`, `Source`.

get_order_book() و get_queue() – سفارش و صف زنده

```
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code>	برای کل بازار، یک نماد یا فهرست نمادها <code>None</code>
<code>selector_strict</code>	گم‌شده یا مبهم را به خطا تبدیل می‌کند <code>selector</code>
<code>side</code>	در <code>get_queue()</code> : <code>"buy"</code> , <code>"sell"</code> , <code>"both"</code>
<code>strict</code>	فقط صف قطعی مبتنی بر قیمت مجاز و <code>level</code> یک را نگه می‌دارد.

خروجی: `order book` به شکل `long` با پنج `level`؛ `queue` با سمت، قیمت، حجم، تعداد سفارش و ارزش صف.

list_etfs(), list_funds(), list_bonds() ، اوراق و اختیارها

```
list_etfs(progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False,
           strategy=None, commodity_underlying=None,
           classification_status=None)
list_listed_funds(progress=True, *, fund_category=None, strategy=None,
                  commodity_underlying=None, classification_status=None,
                  include_unknown=True)
list_bonds(progress=True)
list_debt_instruments(debt_type=None, active_only=False, progress=True)
list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
```

تابع/ورودی	گزینه‌ها و معنی
<code>progress</code>	در همه این توابع فقط نمایش پیشرفت را کنترل می‌کند.
<code>fund_type</code>	<code>fixed_income</code> , <code>commodity</code> , <code>equity</code> , <code>mixed</code> , <code>market_maker</code> , <code>venture_capital</code> , <code>project</code> , <code>private</code> , <code>fund_of_funds</code> , <code>real_estate</code> , <code>sector</code> , <code>leveraged</code> , <code>index</code> , <code>capital_guaranteed</code> , <code>supplementary_retirement</code> . نیز پذیرفته می‌شود <code>venture</code> نام قدیمی.
<code>listed_only</code>	در <code>list_funds()</code> فقط ابزارهای قابل معامله را برمی‌گرداند؛ هم‌زمان با <code>fund_type</code> مجاز نیست.
<code>fund_category</code>	در <code>list_listed_funds()</code> دسته اثبات‌شده، مانند <code>"commodity"</code> .
<code>strategy</code>	تگ دقیق مانند <code>"sector"</code> , <code>"leveraged"</code> , <code>"index_tracking"</code> یا <code>"market_making"</code> .
<code>commodity_underlying</code>	کالای پایه اثبات‌شده: <code>"gold"</code> , <code>"silver"</code> یا <code>"saffron"</code> ؛ نام عمومی حدس زده نمی‌شود.

تابع/ورودی	گزینه‌ها و معنی
<code>classification_status</code>	فیلتر بر اساس <code>authoritative_field</code> , <code>verified_exact_mapping</code> , <code>explicit_official_name</code> یا <code>unknown</code> .
<code>include_unknown</code>	در فهرست بورسی، با <code>False</code> ردیف‌های فاقد طبقه‌بندی کافی را حذف می‌کند؛ پیش‌فرض <code>True</code> همه را نگه می‌دارد.
<code>debt_type</code>	در <code>list_debt_instruments()</code> یک نوع یا فهرست از <code>treasury</code> , <code>erad</code> , <code>gam</code> , <code>murabaha</code> , <code>ijara</code> , <code>salaf</code> , <code>other</code> .
<code>active_only</code>	فقط اوراقی را نگه می‌دارد که سررسیدشان امروز یا بعد از امروز است.
<code>underlying</code>	در <code>list_options()</code> فیلتر اختیاری و در <code>get_options_chain()</code> دارایی پایه اجباری.
<code>fetch_oi</code>	در زنجیرهٔ اختیار، دریافت Open Interest و metadata تکمیلی را فعال می‌کند و ممکن است درخواست‌های بیشتری بسازد.

خروجی: همه `DataFrame` هستند، به‌جز `get_options_chain()` که `dict` شامل `calls` , `puts` , `underlying_price` , `expiry_dates` و `market_time` می‌دهد.

قراردادهای مهم داده

نوع خروجی

همهٔ خروجی‌ها `DataFrame` نیستند:

خانواده	نوع خروجی
قیمت، حقیقی/حقوقی، <code>trades</code> ، <code>order book</code> ، <code>fundamentals</code> و فهرست ابزارها	<code>None</code> گاهی legacy های API یا در <code>pandas.DataFrame</code> هنگام خطا
<code>market_watch()</code> / <code>get_market_snapshot()</code>	و زمان <code>metadata</code> , <code>order_book</code> , <code>stocks</code> شامل <code>dict</code> مشاهده
<code>get_options_chain()</code>	<code>calls</code> و <code>puts</code> شامل <code>dict</code>
<code>resolve_instrument()</code>	شیء <code>immutable</code> از نوع <code>InstrumentRef</code>
<code>watch_market()</code> / <code>MarketWatcher</code>	<code>MarketEvent</code> از <code>iterator</code>
<code>get_yield_curve()</code> / <code>build_yield_curve()</code>	شیء <code>YieldCurve</code>
توابع Black–Scholes	عدد، <code>tuple</code> یا <code>dict</code> مطابق تابع

برای `DataFrame` های جدید، خروجی خالی همان `columns` و `dtypes` خروجی غیرخالی را نگه می‌دارد. `metadata` های مهم مانند منبع، `freshness`، پوشش، `partial` بودن و بودجهٔ درخواست در `df.attrs` قرار می‌گیرند:

```
df = att.get_live_market("فملی")
print(df.attrs)
```

```
{
  'trade_date': datetime.date(...),
  'exchange_time': '12:28:41',
  'fetch_at': Timestamp(..., tz='Asia/Tehran'),
  'is_realtime_fresh': True,
  'is_partial': False,
  'missing_selectors': []
}
```

قیمت پایانی و آخرین معامله

- در feed زنده: **Last** آخرین قیمت معامله و **Close** قیمت پایانی TSETMC است.
- در تاریخچه سهام: **Close** آخرین قیمت معامله روز است؛ **Final** قیمت پایانی است و فقط در **output_type="full"** دیده می‌شود.
- با **auto_adjust=True** (پیش‌فرض)، **OHLC** و **Final** در صورت حضور، تعدیل شده‌اند.
- با **Close**، **auto_adjust=False** و **Final** خام‌اند و **Adj Close** نیز ارائه می‌شود.
- هنگام **Last**، **include_today=True** زنده به **Close** تاریخچه و **Close** زنده به **Final** تاریخچه نگاشت می‌شود. بنابراین صرفاً بر اساس نام ستون بین live و history join و نزدیک.

تاریخ، ردیف امروز و freshness

- می‌پذیرند (2024-07-22) یا میلادی (1403-05-01) شمسی ISO شامل دو سر بازه‌اند و تاریخ **end** و **start**.
- رفتار قبلی حفظ شده است: **include_today=False** هیچ درخواست زنده اضافه‌ای انجام نمی‌دهد.
- قدیمی snapshot. می‌کند append/replace معتبر همان روز معاملاتی را observation فقط **include_today=True**.
- خواهد داشت **live_is_realtime_fresh=False** همان روز ممکن است برای تکمیل تاریخچه پذیرفته شود ولی
- توقف نماد، نبود معامله، نبود هویت قطعی یا شکست live باعث جعل ردیف امروز نمی‌شود؛ تاریخچه موفق برمی‌گردد و هشدار/attrs دلیل را نشان می‌دهند.
- مانند server-side ادعا نمی‌کند. تاریخچه backfill محلی فقط از زمان ضبط شما پوشش دارد و history قرارداد جداگانه دارد price/trades/order-book.

تقارن API زنده/تاریخی:

داده	زنده	تاریخی
قیمت و نمای نماد	<code>get_live_symbol</code> , <code>get_live_market</code>	<code>get_history(include_today=...)</code>
حقیقی/حقوقی	<code>get_live_market</code> , <code>get_market_client_type</code>	<code>get_client_type(include_today=...)</code>

داده	زنده	تاریخی
معاملات ریز	<code>get_live_trades</code>	<code>get_trades</code>
پنج سطح سفارش	<code>get_order_book</code>	<code>get_order_book_history</code>
صف	<code>get_queue</code>	<code>get_queue_history</code>
فاندامنتال snapshot	<code>get_market_fundamentals</code>	<code>get_market_fundamentals_history</code>
اخزا و YTM	<code>get_treasury_yields</code>	<code>get_treasury_yield_history</code>
منحنی بازده	<code>get_yield_curve</code>	<code>get_yield_curve_history</code>
اختیار	<code>get_option_market</code>	های snapshot و <code>get_option_history</code> <code>save_option_snapshot</code> / <code>load_option_snapshots</code>
overview/breadth/sector/message/state	متناظر live های helper	یا <code>archive_to</code> پس از <code>*_history</code> های helper snapshot

تقارن به معنی یکسان بودن منبع نیست: بعضی history ها server-side هستند و بعضی فقط observation های ذخیره شده کاربر را می خوانند. `NoBackfill` , `Source` و coverage را بررسی کنید.

ذخیره در CSV

در API های تاریخی `save_path`، legacy مسیر پوشه است، نه نام فایل. نام فایل از نماد/دارایی ساخته می شود:

```
att.get_history(
    "فملی", save_to_file=True, save_path="exports", progress=False
)
# exports/فملی.csv
```

source boundary

درخواست های runtime فقط به provider های پشتیبانی شده TSETMC، صفحه مرجع YTM فرابورس ایران (`ifb.ir`) و API تاریخچه بازار TGJU برای دارایی های کاتالوگ محدودند. redirect به origin دیگر پیش از درخواست دوم رد می شود. هیچ API کدال در این پکیج وجود ندارد.

قطعی و قابل استناد منتشر نمی کند. اختلاف قیمت تعدیل شده و تعدیل نشده معادل DPS، برای رخداد تعدیل قیمت TSETMC استنتاج نمی کند DPS سود نقدی نیست و کتابخانه از آن

حل دقیق هویت نماد

نمادهای تکراری، ابزار منقضی، حق تقدم، اختیار و اوراق می توانند نام مشابه داشته باشند. API های جدید از `InstrumentRef` و `InsCode` استفاده می کنند:

```
ref = att.resolve_instrument("فملی", asset_type="equity")
print(ref)
```

خروجی نماینده:

```
InstrumentRef(
  ins_code='35425587644337450', symbol='فملی',
  name='ملی صنایع مس ایران', asset_type='equity',
  is_active=True, provenance='tsetmc_search_exact', selector='فملی'
)
```

برای اجرای قطعی در `ins_code`، production، بدهید:

```
prices = att.get_history(
  ins_code="35425587644337450", asset_type="equity", progress=False
)
live = att.get_live_symbol(ins_code="35425587644337450")
```

را می‌پذیرد و نتیجه را به صورت رشته برمی‌گرداند؛ ASCII عدد صحیح مثبت دقیق یا رشته ۱ تا ۲۰ رقم `validate_ins_code()` بیش از یک نتیجه معتبر داشته باشد، selector رقم فارسی/عربی، صفر و مقدار مبهم پذیرفته نمی‌شوند. اگر، `bool`، `float` کتابخانه حدس نمی‌زند:

```
try:
  att.resolve_instrument("نماد تکراری")
except att.AmbiguousSymbolError as exc:
  print(exc)
```

```
AmbiguousSymbolError: selector matched more than one instrument; pass ins_code
```

گزینه‌های اصلی:

```
att.resolve_instrument(
  selector=None,
  ins_code=None,
  asset_type="auto",
  snapshot=None,
  require_active=True,
)
```

- دوباره را می‌دهد fetch بدون resolve امکان `snapshot`
- برای تاریخچه ابزار منقضی مناسب است `require_active=False`

- هویتی fuzzy join و فاصله‌های متداول را یکسان می‌کند، اما `ک/ی`، `ی/ی` اختلاف `normalize_instrument_text()` انجام نمی‌دهد.

قیمت و حقیقی/حقوقی؛ تاریخچه و زنده

`get_history()`

```
att.get_history(
    symbol="فملی", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    ins_code=None, asset_type="auto",
)
```

```
history = att.get_history(
    "فملی", limit=10, include_today=True,
    output_type="full", progress=False,
)
print(history.tail(2))
print(history.attrs["include_today_appended"])
```

خروجی نماینده:

Date	Open	High	Low	Close	Final	Volume	No.	Value
1405-06-02	73100	74600	72800	74200	73950	1820043	3912	1.35e+11
1405-06-03	74000	74900	73600	74700	74420	814220	1830	6.06e+10

['فملی']

در `standard` ستون‌ها `Open, High, Low, Close, Volume` هستند. `full` ستون‌های `Final, No., Value` و اطلاعات تقویم/Ticker را نیز اضافه می‌کند. `raw=True` فرمت TSE را برمی‌گرداند. `return_type` یکی از `both`، `log`، `simple` یا فرم سفارشی مانند `['simple', 'Close', 5]` است. برای چند نماد، DataFrame با MultiIndex ستونی برمی‌گردد.

`get_client_type()`

```
client = att.get_client_type(
    "فملی", limit=30, include_today=True,
    output_type="full", progress=False,
)
print(client.tail(1))
```

خروجی نماینده ردیف امروز:

Date	No_buy_retail	Vol_buy_retail	No_sell_retail	Vol_sell_retail	Power_retail	Is_partial
1403-05-07	1284	920000	991	701000	1.24	True

ردیف امروز volume/count را از `ClientTypeAll` می‌گیرد. valueهای امروز در صورت نیاز با VWAP بازار برآورد می‌شوند و ستون‌های `Value_source` / `Is_estimated` در خروجی opt-in مشخص می‌کنند که مقدار رسمی یا برآوردی است. رفتار پیش‌فرض و schema قدیمی بدون `include_today` تغییر نکرده است.

live کل بازار و یک نماد

```
snapshot = att.get_market_snapshot()      # dict سازگار با market_watch()
live_all = att.get_live_market()          # DataFrame ادغام شده
live_one = att.get_live_market("فملی")
point = att.get_live_symbol("فملی", fallback="none")
```

می‌کند. ستون‌های کلیدی join و بهترین سفارش‌ها را client type، قیمت `get_live_market()`

```
InsCode, Symbol, Name, Last, Close, PreviousClose, Volume, Value,
IndividualBuyVolume, IndividualSellVolume, LegalBuyVolume, LegalSellVolume,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
BidPrice1, AskPrice1, Spread, SpreadBps, L1Imbalance, L5Imbalance,
trade_date, exchange_time, fetched_at, is_realtime_fresh, is_partial
```

freshness در `get_live_market()` row-wise ستون attr عمومی است، نه attr واقعی برای audit schema/selection هستند:

```
print(live_all.attrs.keys())
print(live_all.attrs["missing_selectors"])
```

```
dict_keys(['field_validity', 'source_schema_presence', 'migration', 'missing_selectors'])
[]
```

می‌دهد `StockNotFoundError`، `MarketWatch` در نبود نماد در `get_live_symbol(..., fallback="none")` دقیقاً یکی از `SnapshotSource` را امتحان می‌کند؛ ستون TSETMC نقطه‌ای endpoint فقط در آن حالت `fallback="point"` است. `PresentInMarketWatch`، `FallbackReason`، `IdentityVerified`، `has_trade_today` و `PriceActionable` قابلیت استفاده قیمت را شفاف می‌کنند.

پنج‌سطحی دارد long خروجی `get_order_book()`

InsCode	Symbol	Level	BidOrderCount	BidVolume	BidPrice	AskPrice	AskVolume	AskOrderCount
...	31		124000	74200	74100	180000	42	1

فملى

ستون‌های metadata آن شامل `trade_date` , `exchange_time` , `fetch_at` , `snapshot_age_seconds` , `is_partial` و `is_realtime_fresh` , `is_stale` است.

معاملات ریز

همان قرارداد را `get_live_trades()` استفاده می‌کند و TSETMC تاریخچه معاملات endpoint برای هر روز از `get_trades()` برای روز جاری ارائه می‌دهد:

```
trades = att.get_trades(
    "فملى",
    start="1403-05-01",
    end="1403-05-03",
    include_canceled=False,
    max_requests=10,
    raw=False,
    progress=False,
)

live_trades = att.get_live_trades(
    ins_code="35425587644337450",
    include_canceled=False,
    max_requests=1,
    progress=False,
)
```

استاندارد schema:

InsCode, Symbol, GregorianDate, JalaliDate, TradeNo, Time, Timestamp,
Price, Volume, Value, Canceled, Source

خروجی نماینده:

Timestamp	TradeNo	Time	Price	Volume	Value	Canceled	Source
...	1	09:01:01	50000	100	5000000	False	tsetmc_trade_history_lossless
...	2	09:01:01	50010	50	2500500	False	tsetmc_trade_history_lossless

را نیز نگه `RawCanceled` و `RawDate` , `RawInsCode` , `iSensVarP` , `qTitNgJ` مانند provider فیلهای خام `raw=True` های مهم `attrs` می‌دارد.

```
request_count, trade_request_count, max_requests, request_budget_scope,
partial, failures, source_coverage, reconciliation,
resolved_ins_code, resolved_symbol
```

- ممکن است هزینه جداگانه داشته باشد و resolver است؛ trade endpoint سقف سخت درخواست‌های `max_requests` مشخص می‌کند شمارش کل شناخته‌شده است یا نه `attrs`.
- شکست بخشی از روزها با `partial=True` و جزئیات `failures` برمی‌گردد؛ داده موجود دور ریخته نمی‌شود.
- رد می‌شود `DataParsingError` پاسخ، تاریخ یا فیلدهای عددی با `InsCode` در `mismatch`.
- رکورد ابطال فقط با `include_canceled=True` نگه داشته می‌شود.

سفارش و صف

سفارش زنده

```
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
```

قیمت صف را با `strict=True` قطعی `False` است، نه `NA` صف را سه‌حالت می‌کند؛ نبود شواهد کافی `get_queue()` بازار تطبیق می‌دهد state دامنه مجاز و

```
InsCode Symbol Side QueuePrice QueueVolume QueueOrders QueueValue
...      فملی   buy      73500      820000      163  6.027e+10
```

```
is_queue=True, is_strict=True, is_partial=False,
threshold_source='market_watch_price_limits', book_source='market_watch_live_snapshot'
```

تاریخچه پنج سطح سفارش

```
books = att.get_order_book_history(
    "فملی",
    limit=20,
    output_type="standard", # long ردیف در هر level
    include_today=True,
    complete_only=True,
    max_requests=25,
    progress=False,
)

wide = att.get_order_book_history(
    "فملی", limit=10, output_type="wide", max_requests=25, progress=False
)
```

داده تاریخی **BestLimits** یک delta stream روزانه است. کتابخانه state هر روز را مستقل و به ترتیب **hEven/refID** بازسازی می‌کند؛ مقدار صفر پاک‌شدن level است. schema long:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, refID, _sequence, Level,
BidOrderCount, BidVolume, BidPrice, AskPrice, AskVolume, AskOrderCount,
is_partial, is_complete, market_partial_status, is_reconstructed,
is_stale, record_type, source
```

خروجی نماینده:

Timestamp	Level	BidPrice	BidVolume	AskPrice	AskVolume	is_complete	source
2024-07-22 09:05:01+03:30	1	74100	180000	74200	124000	True	tsetmc_best_limits_history_reconstructed
2024-07-22 09:05:01+03:30	2	74050	95000	74250	88000	True	tsetmc_best_limits_history_reconstructed

برابر source زنده معتبر را با **include_today=True** snapshot ناقص را حذف می‌کند **complete_only=True**. سقف سخت است و **max_requests** اضافه یا جایگزین می‌کند **market_watch_live_snapshot**. پوشش ناقص را توضیح می‌دهند **attrs['request_count']** و **attrs['failed_requests']**. نام **get_orderbook_history** alias عینی **get_order_book_history** است.

تاریخچه صف

```
queues = att.get_queue_history(
    "فملی",
    limit=20,
    include_today=True,
    complete_only=True,
    side="both",
    strict=True,
    max_requests=25,
    progress=False,
)
```

schema:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, Side, QueuePrice, QueueVolume, QueueOrders, QueueValue,
Pricelimit, is_strict, is_queue, is_partial, is_complete,
market_partial_status, book_state, is_crossed, is_preopen_or_stopped,
threshold_hEven, threshold_source, book_source, source
```

اگر static threshold تاریخی در دسترس نباشد، **threshold_source='unavailable'** و **is_queue=NA** می‌ماند. این رفتار برای backtest مهم است: نبود داده به «صف نبود» تبدیل نمی‌شود.

Watcher و تحلیل کل بازار

بازار iterator

```
for event in att.watch_market(
    symbol="فملى",
    interval=2,
    max_updates=5,
    notifications=("messages", "state"),
):
    print(event.kind, event.sequence, event.fetched_at, event.changed_inscodes)
```

شامل event. وجود ندارد `kind="update"`. است `initial`, `delta`, `heartbeat`, `resync` یکی از `MarketEvent.kind` است. `snapshot`، `persistence` و وضعیت `retry`، `cursor`، `level` تغییرکرده، `InsCode` تغییرکرده فهرست،

برای کنترل کامل:

```
watcher = att.MarketWatcher(
    symbol=None,
    interval=2,
    max_updates=None,
    include_initial=True,
    emit_heartbeats=True,
    notifications=("messages", "state"),
    notification_top=50,
    error_policy="retry",
    max_consecutive_retries=5,
    max_backoff=30,
    callback_error_policy="raise",
)
```

`resync` محدود `backoff` را نگه می‌دارد و در قطع ارتباط با `cursor`، استفاده می‌کند `MarketWatchInit/Plus` از `Watcher` هستند `state` و `messages` ها فقط `notification` می‌شود.

پیام، وضعیت و نمای بازار

```
messages = att.get_market_messages(flow=0, top=20, since_id=None)
states = att.get_instrument_state_changes(top=20, since_id=None)
overview = att.get_market_overview(flow=0)
```

پیام schema:

```
message_id, date, time, timestamp, title, description, flow
```

وضعیت schema:

```
event_id, date, time, timestamp, InsCode, Symbol, Name,
state_code, state, real_time, under_supervision, state_title
```

رسمی payload برمی‌گرداند؛ مجموعه ستون‌ها به `flow` را با ستون `overview` رسمی `raw` فیلدهای `get_market_overview()` وابسته است و به تعداد ثابتی از ستون‌ها متعهد نیست `provider`.

breadth و جریان صنایع

```
breadth = att.get_market_breadth(traded_only=True)
sectors = att.get_sector_flow(traded_only=True)
```

خروجی نماینده breadth:

instrument_count	advances	declines	unchanged	ad_difference	ad_ratio	total_volume	upper_limit_count
lower_limit_count	612	318	241	53	77	1.32	8.91e+09
18							42

ستون‌های breadth علاوه بر موارد بالا شامل `no_trade`, `missing_previous`, `missing_current_price` درصد صعود/نزول، `total_value`, `trade_date`, `exchange_time`, `fetch_at`, `is_realtime_fresh` است.

همراه با پوشش حقیقی/حقوقی ارائه می‌کند `SectorCode` را برای هر breadth همین `get_sector_flow()`:

SectorCode	instrument_count	advances	declines	client_coverage	net_individual_volume
estimated_net_individual_value	27	43	25	13	0.91
5.8e+10	market_vwap	False			820000

freshness و `client_coverage`, `value_method`, `value_available` برآورد است؛ `estimated_net_individual_value` در استراتژی لحاظ کنید.

فیلترهای مشترک عبارت‌اند از `instrument_types` و `symbol`, `flow`, `sector`, `traded_only`, `include_base_market` برای محاسبه چند خروجی روی یک مشاهده، snapshot را یک‌بار دریافت و به مسیر خصوصی `_snapshot` ندهید؛ API عمومی `save_market_snapshot()` یک snapshot اتمیک می‌سازد و history derivation را هم‌زمان نگه می‌دارد.

تقویم، نمای کلان، مستر ابزار و پیش‌گشایش

برای تقویم، TSETMC های این بخش دو گروه داده را یکجا در نسخه 1.4.0 فراهم می‌کنند: داده‌های عمومی و بدون حساب API قابل دریافت است TSETMC رسمی که فقط با اشتراک وب‌سرویس TOP ارزش بازار، اثر شاخص، خلاصه روزانه و مستر ابزار؛ و

تقویم معاملاتی بورس و فرابورس

```
calendar = att.get_trading_calendar(
    start="1405-06-01",
    end="1405-06-18",
    market="all",
    include_closed=True,
)
print(calendar.tail())
```

تاریخ شمسی یا `end` و `start` است. `farabourse` / `ifb` یا `bourse` / `tse`، یکی از `all`، `market` در حالت `include_closed` آخرین جلسه هر بازار را برمی‌گرداند. پارامتر `limit`، میلادی و شامل دو مرز هستند. اگر بازه ندهید روزهای تقویمی بین دو مرز را نیز می‌سازد و `True` فقط جلسات منتشرشده شاخص مرجع را می‌آورد؛ مقدار `False` پیش‌فرض می‌گذارد `IsTradingDay=False`.

```
Market, Flow, GregorianCalendar, JalaliDate, IsTradingDay,
ReferenceIndex, ReferenceInsCode, Source
```

تعریف روز معاملاتی در این تابع «وجود مشاهده در تاریخچه شاخص کل رسمی همان بازار» است. بنابراین تعطیلی عادی یا نبود جلسه مشخص می‌شود، اما برنامه ساعات جلسه و علت تعطیلی از این منبع استخراج نمی‌شود. این معنا در `attrs['calendar_semantics']` ثبت شده است.

ارزش و فعالیت روزانه بازار

```
market_value = att.get_market_value_history(
    start="1405-01-01",
    market="all",
    limit=90,
)

activity = att.get_market_activity(
    start="1405-06-15",
    end="1405-06-18",
    market="tse",
    max_requests=10,
    progress=False,
)
```

`get_market_value_history()` برای هر بازار `MarketValue` دو ستون مشتق‌شده `Change` و `ChangePct` را به ریال و دو ستون مشتق‌شده `MarketValue` برای هر بازار `get_market_value_history()` مشخص باشد تا ۵۰۰۰ مشاهده از منبع درخواست می‌شود و سپس بازه فیلتر می‌شود؛ در حالت بدون بازه `start` می‌دهد. اگر سقف دریافت است `limit`.

```
Market, Flow, GregorianCalendar, JalaliDate, MarketValue,
Change, ChangePct, Source
```

بودجه سخت ۱ تا ۳۶۶ است و `max_requests` انجام می‌دهد bulk برای هر جلسه یک درخواست `get_market_activity()` `InstrumentCount` , `TradedInstrumentCount` , `TradeCount` , `Volume` , `Value` , `AverageTradeValue` است. این آمار از خلاصه رسمی روزانه ابزارها ساخته می‌شود؛ `AverageTradeValue = Value` و `Value` است `NaN` و در نبود معامله `/ TradeCount`

اثر نمادها بر شاخص

```
impact = att.get_index_impact(
    date="1405-06-18",
    market="tse",
    top=10,
    direction="both",
)
print(impact[["Rank", "Symbol", "Impact", "AbsSharePct"]])
```

`both` فقط `direction` . عدد صحیح مثبت تا ۱۰۰۰ است `top` . آخرین تاریخ دارای ارزش بازار را انتخاب می‌کند `date=None` , سهم قدرمطلق هر ردیف از مجموع قدرمطلق `AbsSharePct` عدد رسمی اثر و `Impact` . را می‌پذیرد `negative` یا `positive` . نیست مگر اینکه منبع تمام ردیف‌ها را برگرداند universe **ردیف‌های بازگردانده شده همان بازار است**؛ این درصد سهم از کل

خلاصه bulk همه ابزارها در یک روز

```
daily = att.get_market_trades(
    date="1405-06-18",
    market="all",
    progress=False,
)
leaders = daily.nlargest(10, "Value")
```

هر ردیف یک خلاصه روزانه یک هویت ابزار است و ستون‌های `TradeCount` , `Volume` , `Value` , `Open` , `High` , `Low` , `Close` , `ChangePct` و `Last` , `PreviousClose` , `Change` را دارد. نام قدیمی سرویس رسمی `TradeOneDayAll` است، اما این خروجی **معاملات ریز و شماره‌دار نیست**. برای معامله به معامله از `get_trades()` یا `get_live_trades()` استفاده کنید.

ردیف‌های `market='all'` فیلتر می‌شوند. در ISIN با نگاشت قطعی خانواده پیشوند `market='tse'` و `market='ifb'` و `MarketClassification` جریان‌های دیگر، بورس کالا، انرژی یا طبقه‌بندی نشده نیز حفظ می‌شوند و کیفیت اتصال در `attrs['unknown_market_rows']` اما پیشوندهایی که بازارشان به‌تنهایی قابل اثبات fuzzy دیده می‌شود. این روش `progress` . می‌مانند `unknown` یا `other` نیست عمداً فقط پیام پایان را کنترل می‌کند

مستر ابزارها و کشف تغییرات

```
master = att.get_instrument_master(
    active=None,
    market=None,
    asset_type=["stock", "fund", "bond"],
    progress=False,
)
master.to_parquet("instrument-master-1405-06-18.parquet", index=False)

changes = att.get_instrument_changes(
    previous="instrument-master-1405-06-18.parquet",
    current=None,
    progress=False,
)
```

در صورت استفاده یکی از `market` می‌خواند `InsCode` را با کلید پایدار TSETMC جدول عمومی `get_instrument_master()` `stock`, `right`, `fund`, `bond`, `option`, `future`, `asset_type` از iterable یک رشته یا `asset_type`. بورس یا فرابورس است وضعیت فعالیت را نامعلوم نگه می‌دارد و `None` با مقدار `active` است. پارامتر `mortgage`, `commodity`, `energy`, `unknown` `True` یا `False` درخواست زنده اضافه ندارد؛ مقدار تطبیق می‌دهد MarketWatch لحظه‌ای snapshot مستر را با `True` یا `False` درخواست زنده اضافه ندارد؛ مقدار

InsCode, ISIN, Symbol, Name, EnglishName, CompanyCode, CompanyISIN,
Market, MarketGroup, Industry, AssetType, InstrumentStatus, Active, Source

یا مسیر `DataFrame` مقایسه می‌کند. هر ورودی می‌تواند `current` قبلی را با snapshot یک `get_instrument_changes()` `current=None` داشته باشد `InsCode` باشد و باید ستون CSV, Parquet, JSON/JSONL است. `Field`, `OldValue`, `NewValue` و ستون‌های `added`, `removed`, `changed` `ChangeType` های با long-form خروجی های خودتان را نگه دارید snapshot نمی‌کند؛ برای مقایسه روزهای گذشته باید `backfill` این تابع تاریخچه را از سرور

TOP رسمی و عدم تعادل پیش‌گشایش

```
top = att.get_theoretical_opening_price(
    username="TSETMC_WEB_SERVICE_USERNAME",
    password="TSETMC_WEB_SERVICE_PASSWORD",
    market="tse",
    timeout=20,
    progress=False,
)

imbalance = att.get_preopen_imbalance(top_data=top)
print(imbalance.nlargest(10, "ImbalanceRatio"))
```

کاربرد ندارد. PyPI هستند؛ حساب عادی سایت یا حساب TSETMC مشخصات اشتراک وب‌سرویس `username` و `password` به سرویس رسمی می‌فرستد و ذخیره نمی‌کند. برای امنیت، رمز را در کد HTTPS پکیج آن‌ها را فقط در همان فراخوانی از راه مهلت درخواست بر حسب ثانیه است `all` و `timeout` بازار یا `market`. قرار ندهید repository یا log notebook.

```
Market, Flow, InsCode, Symbol, Name, GregorianCalendar, JalaliDate, Time,
TheoreticalOpeningPrice, TheoreticalVolume, RemainderSide, RemainderVolume,
TheoreticalBuyVolume, TheoreticalBuyPrice, TheoreticalSellPrice,
TheoreticalSellVolume, ChangePct, Source
```

اگر قبلاً TOP را گرفته‌اید، آن را با `top_data` به `get_preopen_imbalance()` بدهید تا درخواست دیگری انجام نشود. اگر `top_data=None` باشد، همان تابع با `username`, `password`, `market`, `timeout` و `progress` داده رسمی را می‌گیرد. خروجی سه ستون اضافه دارد:

```
ImbalanceVolume = TheoreticalBuyVolume - TheoreticalSellVolume
ImbalanceRatio = ImbalanceVolume / (TheoreticalBuyVolume + TheoreticalSellVolume)
ImbalanceSide = buy | sell | balanced | unavailable
```

این نسبت صرفاً عدم تعادل حجم‌های TOP رسمی است و سیگنال خرید یا فروش محسوب نمی‌شود. در مجموع حجم صفر یا داده ناقص، نسبت `NaN` و سمت `unavailable` است.

تحلیل مدیریتی بازار

این لایه داده تازه‌ای را حدس نمی‌زند؛ محاسبات آن روی خروجی‌های تاریخچه، حقیقی/حقوقی، نمای زنده و فعالیت روزانه همین پکیج انجام می‌شود. هر تابع تحلیلی علاوه بر حالت دریافت خودکار، ورودی `DataFrame` صریح می‌پذیرد تا یک `snapshot` مشخص را بازپخش کنید، تست تکرارپذیر بسازید و از `look-ahead` جلوگیری شود.

مقایسه نمادها

```
comparison = att.compare_symbols(
    ["فولاد", "فملی", "شپنا"],
    start="1404-01-01",
    benchmark="شاخص کل",
    align="inner",
    min_observations=20,
)

print(comparison[[
    "Symbol", "TotalReturnPct", "AnnualizedVolatilityPct",
    "MaxDrawdownPct", "Beta", "NetIndividualFlow"
]])
```

`benchmark`. تاریخ شمسی/میلادی را می‌پذیرند `end` و `start`. نمادهای غیرخالی است `iterable` یک نماد یا `symbols` فقط تاریخ‌های مشترک همه سری‌ها را نگه `align='inner'`. اختیاری است و با همان قرارداد هویت به تاریخچه اضافه می‌شود. اتحاد تاریخ‌ها را نگه می‌دارد و هر متریک از مشاهدات موجود خودش محاسبه می‌شود `align='outer'` می‌دارد؛ حداقل بازده روزانه لازم برای `min_observations`. تعداد جلسات سال، با پیش‌فرض ۲۴۰ و بازه ۱ تا ۳۶۶ است `annualization` تبدیل می‌شود `QualityFlags` و `NaN` آمار ریسک است؛ کمبود داده به

با `include_client_type=False` دریافت حقیقی/حقوقی حذف می‌شود. `history_data` و `client_data` نگاشت `{symbol: DataFrame}` برای محاسبه آفلاین هستند و ورودی کاربر `mutate` نمی‌شود. `strict=True` برای نماد فاقد تاریخچه `DataParsingError` می‌دهد؛ در حالت پیش‌فرض ردیف همان نماد با `flag` برمی‌گردد. `progress` فقط نمایش پیشرفت دریافت چندنمادی را کنترل می‌کند.

خروجی شامل بازده کل/سالانه، نوسان سالانه، بیشینه افت، بهترین/بدترین روز، درصد روز مثبت، `Beta`، همبستگی، آلفا، `tracking error`، `information ratio`، ارزش و حجم متوسط، ارزش متوسط هر معامله، خالص پول حقیقی و قدرت حقیقی است. ستون‌های مبلغی بر حسب ریال هستند. `Observations` تعداد قیمت‌ها، `ReturnObservations` تعداد بازده‌ها و `CoveragePct` پوشش سری روی تقویم هم‌تراز را ثبت می‌کنند.

سنجه‌های نقدشوندگی

```
liquidity = att.get_liquidity_metrics(
    ["فولاد", "فملی"],
    window=20,
    include_live=True,
)

print(liquidity[[
    "Symbol", "AverageDailyValue", "Amihud1B", "CurrentTurnover",
    "SpreadBps", "L5DepthValue", "ZeroTradeRatio"
]])
```

است؛ اگر نماد بدهید، تاریخچه هر نماد هم خوانده `snapshot` یک حالت سریع فقط-زنده برای تمام نمادهای `symbols=None` فقط `annualization` را به کار می‌گیرد `start/end` کل بازه `None` تعداد آخرین جلسات تاریخی است و `window` می‌شود `live_data` را حذف می‌کند و در این حالت دادن `snapshot` سنجه‌های `include_live=False` نوسان را سالانه می‌کند `strict`، است. `get_live_market()` خروجی صریح `live_data` نگاشت تاریخچه‌ها و `history_data` خطاست. همان معنای بخش قبل را دارند `end` و `start`، `progress`.

می‌آید `AmihudUnit` برابر میانگین قدرمطلق بازده تقسیم بر ارزش معامله به میلیارد ریال است و واحد آن در `Amihud1B` تقسیم می‌کند؛ این فرض در `snapshot` حجم متوسط تاریخی را بر `تعداد سهام فعلی` `HistoricalTurnover` `CurrentTurnover`، `SpreadBps`، `L1Imbalance`، `QualityFlags` و `SharesOutstandingSource` ثبت می‌شود. زنده می‌آیند. عمق پنج‌سطحی مجموع `snapshot` از `VolumeToBaseVolume` و `MarketCap`، `L5DepthValue`، `L5Imbalance`، `LiveCoveragePct` و `HistoryCoveragePct`. قیمت ضربدر حجم سفارش خرید و فروش است و واحد مبلغی آن ریال است کامل بودن داده را نشان می‌دهند.

رژیم توضیح‌پذیر بازار

```
regime = att.get_market_regime()
print(regime[[
    "Regime", "Score", "Confidence", "TrendScore", "BreadthScore",
    "FlowScore", "LiquidityScore", "QueueScore"
]])
```

پنجره‌های محاسبه را تعیین `annualization=240` و `liquidity_window=20`، `lookback=60`، `benchmark='شاخص کل'` است؛ وزن‌های پیش‌فرض `queue`، `liquidity`، `flow`، `breadth`، `trend` نگاهی اختیاری با کلیدهای `weights` می‌کنند سقف دریافت تاریخچه `max_requests`. به ترتیب ۰/۳۰، ۰/۲۵، ۰/۲۰، ۰/۱۵ و ۰/۱۰ هستند و پس از اعتبارسنجی نرمال می‌شوند سه ورودی صریح برای بازپخش کامل `activity_history` و `live_data`، `benchmark_history`. فعالیت روزانه است دریافت خودکار را کنترل می‌کنند `progress` و `end`، `start`. محاسبه‌اند

پنج امتیاز در بازه تقریبی منفی یک تا مثبت یک‌اند:

- نسبت به نوسان همان افق؛ `benchmark` بازده لگاریتمی: `TrendScore`
- نسبت خالص نمادهای مثبت به منفی در ابزارهای سهامی نوع ۳۰۰، ۳۰۳ و ۳۰۹: `BreadthScore`
- سازگار؛ `snapshot` خالص برآوردی پول حقیقی نسبت به ارزش معاملات همان ابزارهای سهامی و فقط برای: `FlowScore`
- `get_market_activity(market='all')`؛ بازار به میانه ارزش روزانه `snapshot` نسبت ارزش کل: `LiquidityScore`
- عدم تعادل ارزش صف خرید و فروش تازه در ابزارهای سهامی: `QueueScore`

دوباره نرمال `Score` میانگین وزنی **مؤلفه‌های موجود** است. مؤلفه گمشده صفر فرض نمی‌شود؛ وزن‌های باقی‌مانده برای `Score` `risk_on`، برای `Score > 0.25` می‌آید. آستانه‌های ثابت `Confidence` می‌شوند و جمع وزن اولیه موجود در می‌شود. `Regime='unavailable'`، است. اگر هیچ مؤلفه‌ای موجود نباشد `neutral` و فاصله میان آن‌ها برای `risk_off` برای ثبت می‌شوند؛ بنابراین این برچسب `DataFrame.attrs` در `is_partial` وزن‌ها، آستانه‌ها، فهرست مؤلفه‌های مفقود و سیگنال معامله یا مدل پیش‌بینی نیست

نقشه بازار؛ داده و نمودار

```
market_map = att.get_market_map(
    group_by="sector",
    size="market_cap",
    color="net_individual_flow",
    top=30,
)

fig = att.plot_market_map(
    market_map,
    output_path="market-map.html",
    show=False,
    title="نقشه جریان پول صنایع",
)
```

`get_market_map()` مستقل از کتابخانه رسم است. `group_by` یکی از `symbol`، `sector`، `flow`، `instrument_type`، `return`، `net_individual_flow`، `color` یکی از `queue_value`، `market_cap`، `volume`، `value` یکی از `size`، `flow` همه گروه‌ها و عدد مثبت حداکثر ردیف‌ها را نگه می‌دارد `top=None` است. `individual_power`، `order_imbalance` همه ابزارها و `'all'` فقط سهام ۳۰۰/۳۰۳/۳۰۹، مقدار `instrument_types=None` یک یا چند کد جریان بازار را فیلتر می‌کند است؛ نبود آن یک `get_live_market()` صریح از `snapshot` یک `data`. مجموعه دلخواه را انتخاب می‌کند `iterable` یک مقدار تازه می‌گیرد `snapshot`

در گروه‌بندی تجمیعی، بازده/قدرت/عدم تعادل با معیار `size` وزن‌دهی و جریان پول جمع می‌شود. `UniverseWeightPct` سهم از کل universe پیش از `top` و `DisplayedWeightPct` سهم از ردیف‌های نمایش‌داده‌شده است؛ پس بعد از برش، فقط دومی جمع ۱۰۰ می‌شود. `QualityFlags` , `IsRealtimeFresh` , `ColorCoveragePct` و `attrs` کیفیت نقشه را قابل بررسی می‌کنند.

رسم اختیاری است و dependency اصلی پکیج را سنگین نمی‌کند:

```
pip install "algotik-tse[visualization]"
```

برمی‌گرداند Plotly از `Figure` یک `plot_market_map(map_data=None, output_path=None, show=False, title=None)` تعاملی می‌نویسد؛ فایل HTML `output_path` را اجرا می‌کند؛ ابتدا `get_market_map()` `map_data=None` با دستور `ImportError` نصب نباشد عنوان نمودار است. اگر `title` را باز می‌کند و Plotly تنظیم‌شده `renderer/merger` قرارداد نقشه باشد و داده‌ی تهی یا ستون‌های ضروری ناقص `DataFrame` باید `map_data` نصب بالا صادر می‌شود `DataParsingError` می‌دهد.

شاخص‌ها و تحلیل صنایع

صنعت در نسخه 1.2.2 دو مفهوم را از هم جدا می‌کند API:

- **عضویت رسمی شاخص:** نمادهایی که endpoint رسمی `GetIndexCompany` برای همان شاخص برمی‌گرداند. توابع این فصل به‌طور پیش‌فرض از این universe استفاده می‌کنند.
- **گروه دیده‌بان:** دسته‌بندی سریع `SectorCode` در MarketWatch که مبنای `get_sector_flow()` است و الزاماً با اعضای رسمی شاخص برابر نیست.

پارامترهای اختصاصی این خانواده عبارت‌اند از `industry` , `industries` , `include_member_count` , `include_live` , `max_workers` و `include_client_type` , `include_orderbook` , `include_empty` , `days` , `metric` , `top` , `refresh` . ورودی‌های عمومی `ascending` , `limit` , `end` , `start` , `progress` و `interval` همان معنای توضیح‌داده‌شده در هر تابع را دارند.

هویت عضوها فقط با `InsCode` متصل می‌شود. نام مشابه یا جست‌وجوی fuzzy برای join استفاده نمی‌شود. همچنین بعضی شاخص‌ها تجمیعی‌اند و عضویت صنایع می‌تواند هم‌پوشانی داشته باشد؛ بنابراین جمع‌زدن ردیف‌های تمام صنایع، کل بازار بدون تکرار تولید نمی‌کند.

مسیر سریع برای کاربر معمولی

```
import algotik_tse as att

# شاخص صنعت؛ سریع و تنها با یک درخواست ۴۵
indices = att.list_industry_indices(progress=False)

# اعضای رسمی یک صنعت همراه داده جاری بازار
members = att.get_industry_members("فلزات اساسی", progress=False)

# تحلیل همه صنایع همراه حقیقی/حقوقی snapshot
snapshot = att.get_industry_snapshot(progress=False)

# همپوشانی رسمی اعضا برای شناسایی همپوشانی بین صنایع
overlap = att.get_industry_membership_overlap(
    industries=["بانک", "خودرو", "فلزات"],
    progress=False,
)

# تغییرات عضویت رسمی هر روز در بازه کوتاه
membership_events = att.get_industry_membership_events(
    industries=["بانک", "خودرو"],
    days=7,
    progress=False,
)

# breadth پنج صنعت برتر از نظر
leaders = att.rank_industries(
    metric="breadth",
    top=5,
    progress=False,
)
```

فهرست شاخص‌های صنعت

```
indices = att.list_industry_indices(
    progress=False,
    include_member_count=True,
    refresh=False,
    max_workers=6,
)
```

تهی می‌مانند. با `HasMembers` و `MemberCount` پیش‌فرض و سریع است؛ در این حالت `include_member_count=False` عضویت را دور می‌زند `cache refresh=True` می‌شود عضویت رسمی همه صنایع دریافت و در حافظه، `True` درخواست‌های عضویت است concurrency و فقط سقف 1.16 عدد صحیح `max_workers`

schema:

```
IndustryName, IndustryNameEn, IndustryGroupCode, IndexInsCode,
IndexValue, IndexPreviousValue, DayHigh, DayLow,
IndexChange, IndexChangePct, MemberCount, HasMembers, ExchangeTime
```

نسخه‌های `list_indices()` درصد تغییر است. این نگاشت در `IndexChangePct` تغییر واحد شاخص و `IndexChange` برعکس بود و در 1.2.0 اصلاح شده است `<=1.1.3`.

اعضای دقیق یک شاخص

```
members = att.get_industry_members(
    industry="بانک",
    include_live=True,
    include_client_type=True,
    include_orderbook=True,
    progress=False,
    refresh=False,
)
```

ورودی `industry` می‌تواند نام فارسی، alias شناخته‌شده مانند `بانک`، صورت کامل مانند `شاخص صنعت بانکها` یا `IndexInsCode` باشد.

- ادغام می‌کند MarketWatch جاری snapshot اعضای رسمی را با `include_live=True`
- شده قابل reconcile قدرت خریدار و جریان حقیقی/حقوقی را اضافه می‌کند؛ فقط ردیف‌های `include_client_type=True` استفاده‌اند.
- `include_live=True` و صف تخمینی پنج سطح را اضافه می‌کند و به `spread`، `imbalance` و `include_orderbook=True` نیاز دارد.
- می‌گیرد TSETMC عضویت را دوباره از `refresh=True`

ثابت خروجی schema:

```
IndustryName, IndustryIndexCode, InsCode, Symbol, Name,
SectorCode, Flow, MarketCode, InstrumentType,
PreviousClose, Open, High, Low, Close, Last, Change, ChangePct,
TradeCount, Volume, Value, SharesOutstanding, EstimatedMarketCap,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
ClientDataAvailable, BidPrice1, AskPrice1, SpreadBps,
L1Imbalance, L5Imbalance,
EstimatedBuyQueueVolume, EstimatedBuyQueueValue,
EstimatedSellQueueVolume, EstimatedSellQueueValue,
TradeDate, ExchangeTime, FetchedAt, IsRealtimeFresh, IsStale
```

ستون‌های مربوط به client type یا order book وقتی درخواست نشده‌اند nullable می‌مانند. `EstimatedMarketCap` حاصل `SharesOutstanding × Close` و `EstimatedNetIndividualFlow` برآورد مبتنی بر VWAP است؛ هیچ‌کدام وزن رسمی شاخص نیستند.

در `DataFrame.attrs` مواردی مانند `universe='exact_index_members'` , `membership_is_current` , `historical_membership_available` , `cache_hit` , `client_type_requested` , `orderbook_requested` و `membership_count` ثبت می‌شوند.

snapshot تحلیلی صنعت

برای یک صنعت:

```
metals = att.get_industry_snapshot(
    industries="فلزات اساسی",
    include_client_type=True,
    include_orderbook=True,
    include_empty=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

برای چند صنعت:

```
selected = att.get_industry_snapshot(
    industries=["بانک", "خودرو", "شیمیایی"],
    include_client_type=True,
    progress=False,
)
```

با `industries=None` تمام صنایع بررسی می‌شوند. `include_empty=False` شاخص‌هایی را که provider در آن لحظه عضو ندارند حذف می‌کند؛ نام آن‌ها در `attrs['empty_industries']` باقی می‌ماند. این تابع فقط یک MarketWatch bulk، حداکثر یک client-type bulk و عضویت‌های cache شده را مصرف می‌کند.

گروه‌های اصلی خروجی:

گروه	ستون‌ها
وضعیت شاخص	<code>IndexValue</code> , <code>IndexPreviousValue</code> , <code>IndexChange</code> , <code>IndexChangePct</code> , <code>IndexDayHigh</code> , <code>IndexDayLow</code>
breadth	<code>MemberCount</code> , <code>TradedCount</code> , <code>Advances</code> , <code>Declines</code> , <code>Unchanged</code> , <code>NoTrade</code> , <code>AdvanceDeclineRatio</code> , <code>AdvancePct</code> , <code>DeclinePct</code>
عملکرد اعضا	<code>EqualWeightReturn</code> , <code>MedianReturn</code> , <code>ReturnDispersion</code>
معاملات	<code>TotalTradeCount</code> , <code>TotalVolume</code> , <code>TotalValue</code> , <code>MarketValueSharePct</code> , <code>UpperLimitCount</code> , <code>LowerLimitCount</code>
حقیقی/حقوقی	<code>ClientCoveredCount</code> , <code>ClientCoverage</code> , <code>NetIndividualVolume</code> , <code>EstimatedNetIndividualValue</code> , <code>IndividualPower</code> , <code>FlowValueMethod</code>

گروه	ستون‌ها
سفارش و صف	<code>OrderBookCoveredCount</code> , <code>OrderBookCoverage</code> , <code>BuyQueueCount</code> , <code>BuyQueueValue</code> , <code>SellQueueCount</code> , <code>SellQueueValue</code>
زمان و تازگی	<code>TradeDate</code> , <code>ExchangeTime</code> , <code>FetchAt</code> , <code>IsRealtimeFresh</code> , <code>IsStale</code>

`IndividualPower` میانگین ساده بازده اعضای دارای قیمت معتبر است و بازده رسمی شاخص نیست `EqualWeightReturn`.
 های سازگار با client row از سرانۀ تجمیعی خرید حقیقی به سرانۀ تجمیعی فروش حقیقی ساخته می‌شود. جریان پول فقط از تازه و متعلق به روز order book را کنار آن کنترل کنید. صف‌ها فقط برای `ClientCoverage` حجم بازار جمع می‌شود؛ همیشه تهران محاسبه می‌شوند.

در `attrs`: `memberships_may_overlap=True` , `membership_is_current=True` , `historical_membership_available=False` , تعداد cache hit/request و روش تخمین جریان پول ثبت می‌شود.

همپوشانی اعضای رسمی صنعت‌ها

```
industry_overlap = att.get_industry_membership_overlap(
    industries=["بانک", "خودرو", "فلزات"],
    progress=False,
)
```

این تابع همپوشانی رسمی اعضای هر جفت صنعت را در `GetIndexCompany` محاسبه می‌کند. خروجی long-form است و برای هر جفت unordered یک ردیف دارد:

```
IndustryAName, IndustryAIndexCode, IndustryBName, IndustryBIndexCode,
CommonMembers, UnionMembers, Jaccard, OverlapA_Pct, OverlapB_Pct
```

است و union تعداد منحصر به فرد اعضای `UnionMembers`، تعداد نمادهای مشترک در دو صنعت را می‌دهد `CommonMembers`.
 همپوشانی نسبت به اندازه هر `OverlapA_Pct` و `OverlapB_Pct` است. `CommonMembers/UnionMembers` نسبت `Jaccard` صنعت را نشان می‌دهند و برای شناسایی همپوشانی‌های نامتقارن مفید هستند.

رویدادهای عضویت point-in-time برای صنایع

```
membership_events = att.get_industry_membership_events(
    industries=["بانک", "خودرو"],
    days=30,
    progress=False,
)
```

این تابع اختلاف اعضای رسمی هر صنعت را بین تاریخ‌های متوالی در `GetIndexCompany` payload نشان می‌دهد. خروجی long-form است و برای هر تغییر، ردیف با `EventType` برابر `added` یا `dropped` می‌آورد:

IndustryName, IndustryIndexCode, TradeDate, JalaliDate, PreviousTradeDate,
PreviousJalaliDate, EventType, InsCode, Symbol, Name

یعنی در تاریخ قبلی `EventType="dropped"` یعنی نماد در تاریخ فعلی ظاهر شده و در تاریخ قبلی نبود `EventType="added"` معتبر است `days=1..30` وجود داشته اما در تاریخ فعلی حذف شده است. این نگاه در حال حاضر فقط برای بازه کوتاه

تاریخچه شاخص و اعضا

تاریخچه خود شاخص:

```
history = att.get_industry_history(
    industry="شیمیایی",
    start="1404-01-01",
    end=None,
    limit=120,
    ascending=True,
    progress=False,
)
```

یعنی `limit=0` هستند. فیلتر تاریخ ابتدا اعمال می شود و `YYYY-MM-DD / YYYYMMDD` تاریخ شمسی یا میلادی `start` و `end` جلسه را نگه می دارد. خروجی N بدون محدودیت؛ مقدار مثبت آخرین

است. منبع رسمی برای `IndustryName, IndustryIndexCode, TradeDate, JalaliDate, High, Low, Close, Change, ChangePct` `attrs['volume_available']=False` روزانه نمی دهد؛ ستون حجم مصنوعی ساخته نمی شود و `volume` شاخص صنعت است.

تاریخچه کوتاه همه اعضای فعلی:

```
member_history = att.get_industry_members_history(
    industry="خودرو",
    days=30,
    ascending=True,
    progress=False,
    refresh=False,
)
```

long-form رسمی است. خروجی `payload` و تعداد آخرین تاریخ معاملاتی موجود در `1..30` عدد صحیح `days`

IndustryName, IndustryIndexCode, TradeDate, JalaliDate,
InsCode, Symbol, Name, Close, Last, Change, ChangePct,
TradeCount, Volume, Value

این `history` برای **اعضای فعلی** شاخص است، نه عضویت `point-in-time`. اگر ترکیب شاخص تغییر کرده باشد، `survivorship` `bias` محتمل است؛ به همین دلیل `attrs` صریح `point_in_time_membership=False` و `survivorship_bias_possible=True` دارد.

داده درون‌روزی شاخص صنعت

```
intraday = att.get_industry_intraday(
    industry="فلزات",
    interval="5min",
    progress=False,
)
```

مقادیر مجاز `interval` عبارت‌اند از `raw`، `1min`، `5min`، `15min`، `30min`، `60min`، `1h`. حالت `raw` هر مشاهده provider را به صورت یک کندل تک‌نقطه‌ای نگه می‌دارد؛ سایر حالت‌ها سطح شاخص را به OHLC تبدیل می‌کنند.

```
IndustryName, IndustryIndexCode, Timestamp, JalaliDate, Interval,
Open, High, Low, Close, Change, ChangePct
```

مصنوعی تولید نمی‌کند و فقط `turnover` یا `volume` حجم ندارد؛ تابع `endpoint` تهران است. این `timezone` دارای `Timestamp` را برمی‌گرداند provider آخرین روز موجود در

رتبه‌بندی صنایع

```
ranking = att.rank_industries(
    metric="money_flow",
    top=10,
    ascending=False,
    include_client_type=True,
    include_orderbook=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

های پشتیبانی‌شده `alias` و `metric`:

alias های رایج	معیار canonical
<code>change_pct</code> ، <code>return</code>	<code>IndexChangePct</code>
<code>equal_weight_return</code>	<code>EqualWeightReturn</code>
<code>median_return</code>	<code>MedianReturn</code>
<code>advance_pct</code> ، <code>breadth</code>	<code>AdvancePct</code>
<code>total_value</code> ، <code>turnover</code>	<code>TotalValue</code>
<code>estimated_net_individual_value</code> ، <code>money_flow</code>	<code>EstimatedNetIndividualValue</code>
<code>individual_power</code> ، <code>buyer_power</code>	<code>IndividualPower</code>

ردیف اول را نگه می‌دارد N همهٔ ردیف‌های دارای معیار معتبر را برمی‌گرداند؛ مقدار مثبت فقط `top=None` `ascending=False` دارد و معیار نهایی در `Rank` را همراه snapshot رتبهٔ بزرگ‌تر به کوچک‌تر است. خروجی تمام ستون‌های ثبت می‌شود `attrs['ranking_metric']`

مقایسه و همبستگی شاخص‌های صنعت

```
compare = att.compare_industries(
    industries=["خودرو", "بانک", "فلزات"],
    metric="close",
    limit=20,
    ascending=True,
    progress=False,
)
```

در خروجی، برای هر تاریخ یک ردیف است و هر ستون یک شاخص مقایسه‌شده است. ستون‌های خروجی به این شکل هستند:

TradeDate, JalaliDate, صنعت خودرو [ID], صنعت بانک [ID], صنعت فلزات [ID]

```
relative = att.get_industry_relative_strength(
    industries=["بانک", "شیمیایی", "فلزات"],
    benchmark="خودرو",
    metric="close",
    limit=30,
    ascending=False,
    progress=False,
)
```

این خروجی long-form است و برای هر industry (به جز benchmark) ستون‌های `IndustryReturn`, `BenchmarkReturn`, `RelativeStrength` را تولید می‌کند.

TradeDate, JalaliDate, BenchmarkIndexCode, BenchmarkName,
IndustryIndexCode, IndustryName, IndustryReturn, BenchmarkReturn, RelativeStrength

```
corr = att.get_industry_correlation(
    industries=["بانک", "خودرو", "صنعت فولاد", "ساختمان"],
    limit=120,
    ascending=False,
    progress=False,
)
```

خروجی تابع ماتریس همبستگی $n \times n$ است با `index/columns` برچسب‌دار به فرمت `IndustryName [IndexCode]`.

هر یک از توابع بالا همانند زیر امضای رسمی‌شان را دارند:

```
compare_industries(industries, start=None, end=None, limit=0, metric='close', ascending=True,
progress=True, max_workers=6)
get_industry_relative_strength(industries, benchmark, start=None, end=None, limit=0, metric='close',
ascending=True, progress=True, max_workers=6)
get_industry_correlation(industries, start=None, end=None, limit=0, ascending=True, progress=True,
max_workers=6)
get_industry_membership_overlap(industries, progress=True, refresh=False, max_workers=6)
get_industry_membership_events(industries, days=30, progress=True, refresh=False, max_workers=6)
```

cache، پوشش و محدودیت داده

عضویت کامل شاخص و تاریخچه کوتاه اعضا در یک payload مشترک می‌آیند. TTL پیش‌فرض cache حافظه یک ساعت است:

```
att.settings.industry_membership_cache_ttl = 3600.0
```

مقدار cache 0 را غیرفعال می‌کند. برای دریافت اجباری عضویت تازه از `refresh=True` استفاده کنید. cache فقط در حافظه همان process است و فایلی روی دیسک نمی‌نویسد.

محدودیت‌های رسمی این نسخه:

- وزن رسمی هر عضو، ضریب سهام شناور و divisor شاخص در منبع فعلی ارائه نمی‌شود؛ contribution دقیق نماد به واحد شاخص محاسبه نمی‌شود.
- فعلی ساخته می‌شود و ممکن است در صورت تغییر ترکیب universe بر پایه `get_industry_members_history()` داشته باشد؛ برای تغییرات دقیق اعضا از `survivorship` شاخص، بایاس استفاده کنید.
- صنایع می‌توانند هم‌پوشانی داشته باشند و بعضی کدهای صنعت ممکن است در یک روز بدون عضو باشند.
- برآوردند و نام آن‌ها عمداً `EstimatedNetIndividualValue` و `EstimatedNetIndividualFlow` ، `EstimatedMarketCap` این موضوع را نشان می‌دهد.
- برای تصمیم معاملات، `TradeDate` ، `OrderBookCoverage` ، `ClientCoverage` ، `IsStale` ، `IsRealtimeFresh` و attrs را بررسی کنید.

تاریخچه محلی SQLite

ذخیره‌سازی کاملاً opt-in است؛ بدون path صریح هیچ فایلی نوشته نمی‌شود.

snapshot های بازار

```
db = "market-history.sqlite"

snapshot_id = att.save_market_snapshot(db)
history = att.load_market_snapshots(db, symbol="فملی", limit=500)

# alias ذخیره شده live معنایی برای همان نمای
live_history = att.get_live_market_history(db, symbol="فملی", limit=500)
breadth_history = att.get_market_breadth_history(db, limit=100)
sector_history = att.get_sector_flow_history(db, limit=100)
overview_history = att.get_market_snapshot_summary_history(db, limit=100)
```

خروجی نماینده:

AsOf	InsCode	Symbol	Last Close	Volume	NoBackfill	SnapshotAtomic
2026-08-25 10:11:12+03:30	...	812200	73950	74200	فملی	True

و `CoverageStart`، `CoverageEnd`، `SnapshotFrameAttrs`، `NoBackfill`، `SnapshotAtomic` و `SourceAtomic` هایمانند `attrs` را توضیح می‌دهند `archive` محدود و کیفیت `SourceAtomic`، `application-id`، `schema version`، `SQLite` دارای `transaction` است خراب است.

record_to برای replay event ها

```
watcher = att.MarketWatcher(
    interval=2,
    max_updates=3,
    record_to=db,
    checkpoint_interval=100,
    record_max_records=10_000,
    record_retention_seconds=7 * 24 * 3600,
    storage_error_policy="raise",
)
for _ in watcher:
    pass

events = att.get_market_event_history(db, kind="delta", limit=1000)
```

اولین `event` هر `session` یک `checkpoint` کامل است؛ `delta` ها پس از آن `replay` می‌شوند و `pruning` فقط از مرز `checkpoint` معتبر انجام می‌شود. `storage_error_policy="raise"` انتخاب امن پیش فرض است. `record_market_event()` برای ثبت مستقیم `MarketEvent` موجود است.

archive_to برای رکوردهای مستقل

دو مسیر مستقل اند `record_to` و `archive_to`

```
att.get_market_overview(flow=0, archive_to=db)
att.get_market_messages(flow=0, top=20, archive_to=db)
att.get_instrument_state_changes(top=20, archive_to=db)

overview = att.get_market_overview_history(db, flow=0, limit=100)
messages = att.get_market_messages_history(db, flow=0, limit=100)
states = att.get_instrument_state_changes_history(db, symbol="فملی", limit=100)
```

های پشتیبانی شده است kind سطح پایین برای API `archive_market_records(path, kind, frame, source=...)` اعمال SQL در pagination قبل از `start/end`, `limit/offset` هستند؛ archive بخشی از هویت `source` و `selector` می شوند.

توابع مهم نگهداری:

```
info = att.check_market_history(db)
att.record_market_event(db, event, session_id="strategy-a")
```

عمومی اند migration/inspection برای `MARKET_HISTORY_SCHEMA_VERSION` و `MARKET_HISTORY_APPLICATION_ID`

فاندامنتال بازار، صندوق و تعدیل قیمت

EPS و P/E زنده

```
fundamentals = att.get_market_fundamentals(
    symbols=["فملی", "شتران"],
    pe_min=None,
    pe_max=None,
    positive_pe=False, # نگه دار audit را هم برای unavailable ردیف
    strict=False,
    allow_stale=False,
    archive_to="market-history.sqlite",
    max_requests=1,
    progress=False,
)
```

این تابع یک bulk snapshot می گیرد و EPS/P/E را از همان مشاهده می سازد؛ برای هر نماد سراغ منبع دیگری نمی رود.
schema ثابت:

```
InsCode, Symbol, Name, SectorCode, InstrumentType, Close, Last,
EPS, PE, PECalculated, EPSSource, PriceSource, PESTatus,
TradeDate, ExchangeTime, AsOf, SnapshotAgeSeconds,
IsRealtimeFresh, IsStale, Source, NoLookahead
```

خروجی نماینده:

Symbol	Close	EPS	PE	PECalculated	EPSSource	PriceSource	PEStatus	NoLookahead
10.0	7395	73950	فملى	True	market_watch	Close	ok	True
42100	شتران	<NA>	<NA>	True	missing_in_market_watch	Close	eps_missing	True

EPS صفر/خالی به مقدار جعلی تبدیل نمی‌شود. `PEStatus` یکی از `ok` , `price_missing` , `price_nonfinite` , `price_nonpositive` , `eps_missing` , `eps_nonfinite` , `eps_nonpositive` است؛ بودن در `stale` جداسازی `IsStale` snapshot. مفقود را به خطا تبدیل می‌کند `selector` مثبت را نگه می‌دارد و `P/E` فقط `positive_pe=True` زنده `attrs` می‌شود `attrs['stale_rejected']=True` نیست: خروجی تهی و `exception` `allow_stale=False` با `stale` دقیقاً شامل

`request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_loo`
`oahead,no_backfill,archive_path` است.

تاریخچه fundamentals

```
history = att.get_market_fundamentals_history(
    "market-history.sqlite",
    start="1403-05-01",
    end="1403-05-07",
    symbols="فملى",
    limit=1000,
)
print(history.attrs["no_lookahead"], history.attrs["current_eps_used"])
```

AsOf	Symbol	Close	EPS	PE	EPSSource	Source	NoLookahead
2026-08-24 10:00:00+03:30	10.0	7350	73500	فملى	market_watch	local_market_snapshot	True
2026-08-25 10:00:00+03:30	10.0	7420	74200	فملى	market_watch	local_market_snapshot	True

این `history` فقط از `snapshot` های ذخیره شده شما ساخته می‌شود: `attrs['no_backfill']=True` و `attrs['current_eps_used']=False` دیگر `missing_selectors,strict,source,price_source,eps_source,no_loo`
`ahead,coverage_start,coverage_end` هستند. `EPS` فعلی برای گذشته `forward-fill` نمی‌شود.

صندوق‌های قابل معامله

```
listed = att.list_listed_funds(progress=False)

gold = att.list_listed_funds(
    fund_category="commodity",
    commodity_underlying="gold",
    progress=False,
)
silver = att.list_listed_funds(commodity_underlying="silver", progress=False)

sector_registry = att.list_funds(fund_type="sector", progress=False)
leveraged_registry = att.list_funds(fund_type="leveraged", progress=False)
index_registry = att.list_funds(fund_type="index", progress=False)
```

و (صندوق بورسی عمومی) `InstrumentType=305` هر دو گروه bulk call با یک `list_listed_funds()` NAV، صندوق‌ها می‌آید و اطلاعاتی مثل registry از `list_funds()` را می‌گیرد (صندوق کالایی بورسی) `InstrumentType=380` بازده، ترکیب دارایی و مدیر را برمی‌گرداند. این دو منبع شناسه مشترک قطعی ندارند و با شباهت نام به هم متصل نمی‌شوند «کالایی» مترادف «طلا» نیست و ETF نیز سازوکار معامله است، نه طبقه دارایی. طلا، نقره و زعفران فقط از نگاشت دقیق `InsCode+ISIN` یا عبارت صریح در نام رسمی تشخیص داده می‌شوند. عباراتی مثل «زر»، «گوهر» یا «سیمین» به‌تنهایی مدرک نیستند. اگر مدرک کافی وجود نداشته باشد، مقدار `unknown` برمی‌گردد و `needs_review=True` می‌شود؛ پوشش زیرنوع‌ها ممکن است کامل نباشد.

فهرست بورسی schema

```
InsCode, ISIN, Symbol, Name, InstrumentType, Flow, MarketCode,
Last, Close, Yesterday, Volume, Value, TradeCount, Low, High, NAV,
NAV_Discount, Change, ChangePct, fund_category, asset_exposure,
strategy_tags, trading_mechanism, unit_class, commodity_profile,
commodity_underlyings, primary_commodity, classification_status,
classification_source, classification_evidence, taxonomy_version, needs_review
```

Symbol	InsCode	ISIN	Last	Close	NAV	NAV_Discount	Volume
افران	...	IR03AFRZ0001	21650	21620	...	...	1250040

در خروجی registry نیز `registry_type_id`, `registry_category`, `asset_exposure`, `strategy_tags`, `classification_status`, `classification_source`, `classification_evidence`, `taxonomy_version`, `record_date` و `needs_review` در کنار ستون‌های قبلی حفظ می‌شوند. ثابت `FUND_TAXONOMY_VERSION` نسخه مرجع طبقه‌بندی را نشان می‌دهد. `attrs`های `no_fuzzy_join=True` و `registry_joined=False` قرارداد هویتی را روشن می‌کنند.

رخدادهای تعدیل قیمت

```
adjustments = att.get_price_adjustments(
    "فملی", start="1402-01-01", end="1403-12-29", progress=False
)
latest = att.get_latest_price_adjustment("فملی", progress=False)
```

typed ثابت و schema:

```
InsCode, Symbol, GregorianDate, JalaliDate,
AdjustedClosingPrice, UnadjustedClosingPrice, AdjustmentAmount,
CorporateTypeCode, CorporateActionType, IsConfirmedDPS,
IdentityVerified, Source, FetchedAt
```

خروجی نماینده:

GregorianDate	JalaliDate	AdjustedClosingPrice	UnadjustedClosingPrice	AdjustmentAmount	CorporateTypeCode
2024-01-01	1402-10-11	8000	10000	2000	7
<NA>	False	True			

قرارداد مهم:

- فقط اختلاف ریاضی قیمت‌هاست $\text{AdjustmentAmount} = \text{UnadjustedClosingPrice} - \text{AdjustedClosingPrice}$
- provider خام `CorporateTypeCode` است؛ nullable `CorporateActionType` و `False` همیشه `IsConfirmedDPS` بدون تفسیر نگه داشته می‌شود.
- ثبت می‌شود `attrs['dps_reason']` و دلیل در `attrs['dps_available'] == False`
- رکورد با `InsCode` متفاوت رد می‌شود؛ نبود `InsCode` در خود رکورد با `IdentityVerified=False` و `InsCode` حل‌شده گزارش می‌شود.
- می‌دهد `schema/dtypes/attrs` صفر یا یک‌ردیفی با همان `DataFrame` `get_latest_price_adjustment()`

```
assert adjustments["IsConfirmedDPS"].eq(False).all()
assert adjustments.attrs["dps_available"] is False
```

از این API برای ساخت «سری DPS» استفاده نکنید. TSETMC در این endpoint طبقه‌بندی قطعی سود نقدی ارائه نمی‌دهد.

اخزا و درآمد ثابت

قاعده تاریخ در نماد اخزا

شش رقم انتهای نماد به صورت تاریخ شمسی `YYMMDD` تفسیر می‌شود؛ دو رقم سال با `13` یا `14` تکمیل می‌شود:

```
att.parse_treasury_maturity("020322 اخزا")
```

```
{
  'maturity_jalali': '1402/03/22',
  'maturity_gregorian': datetime.date(2023, 6, 12),
  'maturity_source': 'user_confirmed_symbol_jalali_yymdd'
}
```

```
att.parse_treasury_maturity("991117 اخزا")
```

```
{
  'maturity_jalali': '1399/11/17',
  'maturity_gregorian': datetime.date(2021, 2, 5),
  'maturity_source': 'user_confirmed_symbol_jalali_yymdd'
}
```

برای نماد non-match یا تاریخ شمسی نامعتبر، تابع exception نمی‌دهد و **None** برمی‌گرداند. در صورت تعارض یا نماد غیرقابل تفسیر، **maturity_date** یا **maturity_map** صریح بدهید؛ provenance ستون **MaturitySource** را بررسی کنید.

محاسبات deterministic

```
result = att.treasury_yield(
    price=800_000,
    maturity_date="2028-01-01",
    settlement_date="2026-01-01",
    face_value=1_000_000,
)
```

خروجی واقعی این مثال:

```
EffectiveAnnualYield  0.1180339887
SimpleAnnualYield     0.125
ContinuousYield       0.1115717757
BankDiscountYield     0.0986301370
MacaulayDuration      2.0
ModifiedDuration       1.7888543820
Convexity              4.8
DV01                  143.10835056
DiscountFactor         0.8
Status                 ok
```

برای اوراق کوپنی:

```
cashflows = [
    ("2027-01-01", 80_000),
    ("2028-01-01", 80_000),
    ("2029-01-01", 1_080_000),
]
price = att.bond_price(0.08, cashflows, settlement_date="2026-01-01")
ytm = att.yield_to_maturity(price, cashflows, settlement_date="2026-01-01")
risk = att.bond_analytics(price, cashflows, settlement_date="2026-01-01")
```

`bond_price`، پشتیبانی می‌کند `30/360`، `ACT/360`، `ACT/365F` از قراردادهایی مانند `day_count_fraction()`، `yield_to_maturity` و `bond_analytics` پارامترهای compounding/frequency، clean/dirty price و accrued interest را می‌شوند sign-changing های مبهم یا cashflow دارند؛

اخزای زنده

```
ytm = att.get_treasury_yields(
    symbol=None,
    settlement_date=None,
    face_value=1_000_000,
    include_stale=False,
    min_volume=1,
    price_source="auto", # auto/bid/ask/last/close
    strict=False,
    source="tsetmc", # مرجع join برای hybrid یا
)
```

ستون‌های اصلی:

```
InsCode, ISIN, Symbol, MaturityJalali, Maturity, MaturitySource,
MaturityConflict, SettlementDate, DaysToMaturity, Tenor,
Price, PriceSource, NoTrade, InstrumentPriceAsOf, IsInstrumentStale,
BidPrice, AskPrice, DiscountFactor,
EffectiveAnnualYield, ContinuousYield, SimpleAnnualYield, BankDiscountYield,
MacaulayDuration, ModifiedDuration, Convexity, DV01,
Volume, Value, TradeCount, FaceValue, FaceValueSource,
DayCount, IsStale, SnapshotAgeSeconds, FetchedAt, Status
```

مقایسه `https://ifb.ir/ytm.aspx` را با جدول مرجع مجاز فرابورس ایران در TSETMC داده معاملاتی `source="hybrid"` گزارش می‌شود با provenance با bps جای قیمت ابزار را نمی‌گیرد و اختلاف در IFB می‌کند؛

```
ifb = att.get_ifb_yield_table(category="treasury")
print(ifb.attrs["reference_url"])
```

schema IFB:

```
Symbol, Price, LastTradeJalali, LastTradeDate, PublishJalali, PublishDate,
MaturityJalali, Maturity, Volume, ReferenceYTM,
ReferenceSimpleYield, ReferenceSource
```

اراد و گام

نمی‌کرد. اکنون امضای قدیمی universe می‌شناخت، اما گام را وارد `murabaha` از نسخه‌های قبلی اراد را در گروه `list_bonds()` تاریخ سررسید شش یا هشت‌رقمی و قیمت/حجم روز دیده می‌شود. برای، `BondType='gam'` آن بدون تغییر مانده و گام نیز با جدید استفاده کنید API فیلتر روشن نوع بدهی از

```
debt = att.list_debt_instruments(
    debt_type=["erad", "gam"],
    active_only=True,
    progress=False,
)

gam = att.list_debt_instruments("gam", progress=False)
print(gam[["Symbol", "MaturityJalali", "DaysToMaturity", "Last"]])
```

برابر `گام` و پیشوند `erad` برابر `اراد`، پیشوند `treasury` برابر `اجزا` با قواعد صریح تعیین می‌شود: پیشوند `DebtType` است و TSETMC فهرست و قیمت امروز snapshot قدیمی نیز حفظ شده تا کد قبلی نشکند. این `BondType` است `gam` تاریخچه بازده محسوب نمی‌شود. برای YTM منتشرشده فرابورس:

```
reference = att.get_debt_yields(["erad", "gam"])
print(reference[["Symbol", "DebtType", "Price", "MaturityJalali", "ReferenceYTM"]])
```

این تابع نمای `all` جدول رسمی `ifb.ir/ytm.aspx` را می‌خواند. بازده درصدی منبع به عدد اعشاری تبدیل می‌شود؛ مثلاً ۳۵٪ برابر `0.35` است. پکیج برای اراد و گام ارزش اسمی، کوپن یا جریان نقدی حدس نمی‌زند (`cashflows_inferred=False`). برای ارزش‌گذاری مستقل اوراق کوپنی باید `cashflow`های قطعی قرارداد را به `yield_to_maturity()`، `bond_price()` یا `bond_analytics()` بدهید. اگر صفحه فرابورس در دسترس نباشد `ConnectionError` می‌گیرید؛ این خطا با بازده محاسبه‌شده ساختگی جایگزین نمی‌شود.

تاریخچه YTM

```
history = att.get_treasury_yield_history(
    "اخز ۱۴۰۱۰۹۰۱",
    limit=20,
    include_today=True,
    price_source="close",
    max_requests=25,
    progress=False,
)
```

عینی همین تابع است. هر ردیف از قیمت همان روز ساخته می‌شود و `get_treasury_yields_history` alias. `Close` , `Final` , `OHLC` مناسب نیست. ستون‌های تاریخچه شامل YTM است؛ قیمت تعدیل‌شده برای `auto_adjust=False` , `price` از همان `include_today` . سررسید/قیمت است `provenance` و `Status` , `IsPartial` , `FetchAt` , `analytics` ، `live` متناظر `source` استفاده می‌کند.

منحنی بازده

```
curve = att.get_yield_curve(
    min_nodes=3,
    interpolation="log_discount",
    duplicate_policy="volume_weighted",
    extrapolate=False,
)

df_18m = curve.discount_factor(1.5)
zero_18m = curve.zero_rate(1.5)
fwd = curve.forward_rate(1.0, 2.0)
```

`YieldCurve` شامل `settlement_date` , `maturities` , `times` , `discount_factors` , `continuous_zero_rates` , `node_metadata` و `diagnostics` است. `extrapolate=False` را می‌گیرد.

ساخت مستقیم:

```
nodes = [
    {"Maturity": "2027-01-01", "DiscountFactor": 0.90},
    {"Maturity": "2028-01-01", "DiscountFactor": 0.80},
    {"Maturity": "2029-01-01", "DiscountFactor": 0.70},
]
curve = att.build_yield_curve(nodes, settlement_date="2026-01-01")
```

و `curve.diagnostics` دقیقاً کلیدهای `input_node_count` , `node_count` , `duplicate_count` , `duplicate_policy` و `monotonic_discount_enforced` `discount factor` نزولی بودن `guard` و `dedupe` ، ورودی `audit` برای `metadata` را دارد؛ این است.

تاریخچه منحنی:

```
curves = att.get_yield_curve_history(
    limit=10,
    min_nodes=3,
    include_today=True,
    max_requests=50,
    progress=False,
)
```

هر node دارای `CurveID` , `CurveStatus` , `CurveNodeCount` , `CurveInterpolation` و `flag`های `CurvePricesNoLookahead` , `CurveUniverseNoLookahead` , `CurveNoLookahead` است. اگر `universe` تاریخی از `catalog` امروز بازیابی شود، survivor-bias در diagnostics صریح است و `CurveUniverseNoLookahead=False` می‌شود.

اختیار معامله

تحلیل‌های قیمت‌گذاری این بخش برای اختیار اروپایی هستند. مشخصات واقعی اعمال قرارداد، تعدیلات شرکتی و `dividend` `yield` از TSETMC حدس زده نمی‌شود.

ریاضی Black-Scholes

```
call = att.black_scholes_price(
    spot=100, strike=100, time_to_expiry=1,
    rate=0.05, volatility=0.20,
    option_type="call", dividend_yield=0.0,
)
greeks = att.black_scholes_greeks(100, 100, 1, 0.05, 0.20, "call")
bounds = att.option_price_bounds(100, 100, 1, 0.05, "call")
iv = att.implied_volatility(10.4505835722, 100, 100, 1, 0.05, "call")
```

خروجی واقعی و deterministic:

```
call = 10.4505835722

greeks = {
    'Delta': 0.6368306512,
    'Gamma': 0.0187620173,
    'Vega': 37.5240346917,
    'Vega1Pct': 0.3752403469,
    'ThetaPerYear': -6.4140275464,
    'ThetaPerDay': -0.0175726782,
    'Rho': 53.2324815454,
    'Rho100bp': 0.5323248155,
    'Status': 'ok'
}

bounds = (4.8770575499, 100.0)
iv = {'ImpliedVolatility': 0.1999999955, 'Status': 'ok', 'Iterations': 27}
```

واحدها مهم‌اند: `Vega` تغییر قیمت برای یک واحد `volatility` و `Vega1Pct` برای یک واحد درصد است؛ `ThetaPerYear/Day` و `Rho/Rho100bp` جدا گزارش می‌شوند. خروجی `Greeks` همیشه کلید `Status` دارد. `implied_volatility()` همواره `dict` می‌دهد و کلیدهای پایه آن `ImpliedVolatility`, `Status`, `Iterations` هستند؛ یکی از `expiry`, `missing`, `ok`, `out_of_bounds`, `no_bracket`, `non_converged` است. `out-of-bounds/no-bracket` ممکن است `LowerBound/UpperBound` و `non-converged` مقدار تشخیصی `CandidateVolatility` داشته باشد؛ عدم همگرایی `exception` نیست. فقط `constraint`های ورودی مانند `bracket/tolerance/style` نامعتبر `ValueError` می‌دهند.

snapshot اتمیک بازار اختیار

```
options = att.get_option_market(
    exchange=0,
    underlying="خودرو",
    max_requests=1,
    progress=False,
)
```

هر جفت call/put فقط وقتی metadata کافی و هویت سازگار داشته باشد وارد snapshot می‌شود. ستون‌های اصلی:

```
InsCode, PairID, PairSequence, ISIN, Symbol, Name, OptionType,
UnderlyingInsCode, UnderlyingSymbol, ContractSize, Strike,
BeginDate, EndDate, DaysToExpiry,
Last, Close, Yesterday, Volume, Value, TradeCount, NotionalValue,
OpenInterest, YesterdayOpenInterest, BidPrice, AskPrice, BidVolume, AskVolume,
UnderlyingLast, UnderlyingClose, Price, PriceSource,
AsOf, AsOfSource, SnapshotFreshnessKnown, PriceFreshnessKnown,
Stale, NoTrade, AnalyticsEligible, AnalyticsEligibilityReason,
MetadataConflict, Source
```

attrs های `atomic_snapshot` , `duplicate_pairs_dropped` , `incomplete_pairs_quarantined` ,
`exchange_event_freshness_known` و `malformed_pair_status` را توضیح می‌دهند snapshot کیفیت

`get_options_chain(underlying, fetch_oi=False)` فهرست قراردادها را می‌دهد و `list_options(underlying=None)` قدیمی API یک `dict` با کلیدهای دقیق `calls` , `puts` , `underlying_name` , `underlying_price` , `expiry_dates` پیشنهاد می‌شود `get_option_market()` حرفه‌ای analytics وجود ندارد. برای `price` برمی‌گرداند؛ کلید `market_time` و

IV، Greeks، parity و نقدشوندگی

```
analysis = att.analyze_option_chain(
    options=options,
    spot=None, # اگر معتبر باشد snapshot از
    risk_free_rate=0.30, # یا yield_curve=curve
    dividend_yield=0.0,
    valuation_date=None,
    exercise_style="european",
    parity_tolerance=None,
    allow_unverified_freshness=True,
)
```

ستون‌های تحلیلی افزوده‌شده:

```
TimeToExpiry, Spot, SpotSource, RiskFreeRate, RiskFreeRateSource,
DividendYield, DividendYieldSource,
ImpliedVolatility, ImpliedVolatilityBid, ImpliedVolatilityMid, ImpliedVolatilityAsk,
IVStatus, IVStatusBid, IVStatusMid, IVStatusAsk,
Delta, Gamma, Vega, Vega1Pct, ThetaPerYear, ThetaPerDay, Rho, Rho100bp,
PremiumContract, DeltaContract, GammaContract, VegaContract,
Vega1PctContract, ThetaPerYearContract, ThetaPerDayContract,
RhoContract, Rho100bpContract,
SpreadAbs, SpreadPct, QuotedDepth, LiquidityScore,
ParityResidual, ParityToleranceBand, ParityStatus, ImpliedForward,
GreeksStatus, AnalyticsReliability, AnalyticsWarning, AnalyticsComputed
```

خروجی نماینده:

```
Symbol OptionType Strike Price ImpliedVolatility Delta Vega1PctContract SpreadPct LiquidityScore
ParityStatus AnalyticsReliability
... call 3000 420.0 0.41 0.62 18320.0 0.03 0.81
ok verified_inputs
... put 3000 265.0 0.39 -0.38 17790.0 0.04 0.76
ok verified_inputs
```

است و توصیهٔ آربیتراژ نیست؛ محدودیت وجه تضمین، سبک اعمال، کارمزد و امکان معامله را diagnostic فقط **ParityStatus** قابل اتکا ساخته نمی‌شود analytics معتبر نداشته باشید curve ندهید و **risk_free_rate** مدل نمی‌کند. اگر فقط زمانی ثبت می‌شود که کاربر مقدار را تعیین کرده باشد؛ مقدار صفر پیش‌فرض **DividendYieldSource='user_supplied'** نیست DPS به معنی کشف

PCR

```
pcr = att.option_put_call_ratios(analysis, group_by="underlying")
```

```
UnderlyingInsCode CallVolume PutVolume PCRVolum PCRVolumStatus CallValue PutValue PCRValue
PCRVolumStatus CallOpenInterest PutOpenInterest PCROpenInterest PCROpenInterestStatus
65883838195688438 20000 13000 0.65 ok 9.2e9 5.1e9 0.55 ok
40000 20000 0.50 ok
```

است. خروجی برای هر معیار علاوه بر **underlying_expiry** یا **expiry**، **underlying**، **market** دقیقاً یکی از **group_by** را می‌دهد؛ نسبت با **PCROpenInterestStatus** و **PCRValueStatus** متناظر **status**، نسبت **zero_denominator** برابر **status** و **denominator** صفر

تاریخچه اختیار و snapshot محلی

```
# opt-in قیمت قرارداد؛ ردیف امروز server-side تاریخچه
history = att.get_option_history(
    "...ضخود", limit=10,
    include_today=True, max_requests=3, progress=False,
)

# snapshot بازار اختیار فقط با درخواست صریح کاربر روی فایل نوشته می‌شود.
path = "option-snapshots.json"
att.save_option_snapshot(path, options=options)
saved = att.load_option_snapshots(path)
```

schema history:

```
Timestamp, InsCode, Symbol, Open, High, Low, Close, Last,
Volume, Value, TradeCount, OpenInterest,
BidPrice, AskPrice, BidVolume, AskVolume,
UnderlyingLast, UnderlyingClose, ContractSize, Strike, EndDate,
Price, PriceSource, Source, AsOf, Stale, NoTrade, AnalyticsEligible
```

attrهایی مانند `prices_no_lookahead` , `underlying_prices_no_lookahead` , `rates_no_lookahead` ,
`valuation_date_source` , `curve_applied` و `source_coverage` را برای `backtest` فایل کنید.
`OPTION_SNAPSHOT_SCHEMA_VERSION` ، `writer` ، `write` و `dedupe` است.

بورس انرژی و بورس کالا

نسخه ۱.۶.۰ سه لایه را از هم جدا نگه می‌دارد: داده حراج و ابزارهای بورس انرژی از TSETMC، تابلوی مالی و گزارش‌های بازار فیزیکی از وبسایت رسمی بورس کالای ایران، و محاسبات مشتقه که فقط از ورودی صریح کاربر ساخته می‌شوند. بنابراین یک ستون با واحد «هزار ریال» هیچ‌گاه بدون اعلام به ریال تبدیل نمی‌شود و سررسید قرارداد از روی نام فارسی حدس زده نمی‌شود.

حراج‌های بورس انرژی

```

auctions = att.get_energy_auctions(
    status="upcoming",
    board="physical",
    top=20,
)

one = att.get_energy_auction(
    auctions.iloc[0]["AuctionID"],
    include_trades=True,
    include_instrument_history=False,
)
details = one["auction"]
trades = one["auction_trades"]

```

`get_energy_auctions(status='upcoming', board='physical', top=100)` :

- `all` : آماده ؛ معادل‌های فارسی یا `ended` ، `surplus / mazad` ، `active` ، `upcoming / ready` یکی از : `status` .
نیز پذیرفته می‌شوند `پایان` و `مازاد` ، `فعال`
- این انتخاب مستقیماً به دسته رسمی منبع `all` یا `free` ، `green` ، `special` ، `power` ، `physical` یکی از : `board` متصل است.
- سقف محلی ردیف برای هر ترکیب وضعیت/تابلو، عدد صحیح ۱ تا ۱۰۰۰. چون منبع گاهی پارامتر تعداد را نادیده : `top` می‌گیرد، پکیج سقف را بعد از دریافت نیز اعمال می‌کند.

خروجی شامل `AuctionID` ، `Description` ، `Volume` ، `BasePrice` ، `AuthorizedPriceMin` ، `AuthorizedPriceMax` ، `AuctionDateTime` ، `Producer` ، `PaymentTerms` ، `DeliveryTerms` ، `DeliveryPlace` ، `VolumeUnit` ، `PriceUnit` ، `MinimumPurchase` ، `MinimumPriceDiscoveryVolume` ، `MaximumPurchase` ، `TickSize` ، `LotSize` ، `Status` ، `DiscoveredPrice` ، `TradedVolume` ، `SurplusTradedVolume` و `VWAP` است.

`get_energy_auction(auction_id, include_trades=True, include_instrument_history=False)` یک `dict[str, DataFrame]` برمی‌گرداند :

کلید	معنا
<code>auction</code>	مشخصات کامل همان حراج
<code>auction_trades</code>	فقط معاملات همان <code>AuctionID</code> ؛ در صورت <code>include_trades=True</code>
<code>instrument_history</code>	معاملات حراج‌های قبلی همان ابزار؛ فقط با <code>include_instrument_history=True</code>

`bool` فقط `include_instrument_history` و `include_trades` . باید شناسه عددی مثبت حراج باشد `auction_id` .
تاریخچه ابزار به معنای معامله همان عرضه نیست `endpoint` می‌پذیرند. جداسازی دو جدول معامله ضروری است؛

بازار برق و اوراق انرژی

```
# برق استاندارد، سبز و آزاد
power = att.list_power_instruments(market="all", top=100)

# سلف، گواهی ظرفیت، گواهی صرفه‌جویی، سپرده و آتی
securities = att.list_energy_securities(
    kind="all",
    active=True,
    top=100,
    enrich_futures=True,
)

# آمار روزانه جریان‌های رسمی
overview = att.get_energy_market_overview(market="all")
```

را می‌پذیرد `all` یا `free`، `green`، `standard` مقادیر `market` در `list_power_instruments(market='all', top=100)` سقف ۱ تا ۱۰۰۰ برای هر دسته است `top`.

`list_energy_securities(kind='all', active=True, top=100, enrich_futures=True)`:

- `kind`: یکی از `standard_salaf`، `capacity_certificate`، `energy_saving_certificate`، `deposit_certificate`، `all` یا `future`.
- دسته‌های جاری و منقضی‌شده `False` فقط دسته جاری منبع خوانده می‌شود؛ در حالت `True` در حالت `active`: منتشرشده نیز خوانده می‌شوند. این پارامتر به معنی فیلتر تاریخ حدسی از نام نماد نیست.
- سقف ردیف هر دسته، ۱ تا ۱۰۰۰: `top`.
- باشد برای هر آتی یک درخواست جزئیات زده می‌شود تا سررسید، اندازه قرارداد و قیمت `True` اگر `enrich_futures`: تعداد واقعی درخواست‌ها را نشان می‌دهد `DataFrame.attrs['request_count']`. رسمی پر شود `endpoint` تسویه از

ستون‌های مشترک این دو تابع شامل `Segment`، `InsCode`، `Symbol`، `Name`، `GregorianDate`، `JalaliDate`، `Close`، `Last`، `PreviousClose`، `Open`، `Low`، `High`، `TradeCount`، `Volume`، `Value`، `ValueUnit`، `ExpiryDate`، `ContractSize` و `SettlementPrice` است. در این `ValueUnit='rial'`، `feed` است.

مقادیر زیر را می‌پذیرد `get_energy_market_overview(market='all')`:

جریان رسمی	market
برق آزاد، flow 60	free_power
جمع بازار برق، flow 62	power
سلف استاندارد، flow 63	standard_salaf
برق استاندارد، flow 64	standard_power
سایر اوراق قابل معامله، flow 65	securities
آتی انرژی، flow 67	future
گواهی سپرده انرژی، flow 68	deposit_certificate

جریان رسمی	market
برق سبز، flow 69	green_power

خروجی `State`، `Value`، `Volume`، `TradeCount`، تاریخ شمسی/میلادی و `ValueUnit='rial'` دارد. تاریخ کوتاه یا `sentinel` داخلی منبع به عنوان تاریخ معتبر گزارش نمی شود و مقدار آن خالی می ماند.

مشخصات آتی انرژی

```
contract = att.get_energy_future_contract("13758785995633713")
```

دهدهی دقیق را می پذیرد و یک ردیف برمی گرداند. مشخصات `InsCode` فقط `get_energy_future_contract(ins_code)` شامل دارایی پایه اندازه و واحد قرارداد، شروع، آخرین روز معامله، تحویل و آماده سازی، قیمت های بازار، نسبت های وجه، `ISIN`، شامل دارایی پایه تضمین اولیه/نگهداری/اضافی، کارمزد خرید و فروش و تسویه نقدی/فیزیکی، جریمه، سقف موقعیت باز، ساعات معامله و قواعد تسویه است. مقدار صفر رسمی در فیلد تسویه با قیمت پایانی جایگزین نمی شود.

تابلوی مالی بورس کالا

```
certificates = att.get_commodity_market(kind="certificate")
salaf = att.get_commodity_market(kind="standard_salaf")
commodity_funds = att.get_commodity_market(kind="fund")
futures = att.get_commodity_market(kind="future")
all_rows = att.get_commodity_market(kind="all")
```

`get_commodity_market(kind='all')` یکی از `certificate` / `deposit_certificate`، `standard_salaf` / `salaf`، `fund`، `future` نیز معتبرند آتی و صندوق، سلف، گواهی را می پذیرد. معادل های فارسی `all` یا `future`.

برای گواهی، سلف و صندوق، خروجی این موارد را دارد:

- هویت: `InsCode`، `ISIN`، `Symbol`، `Name`، `EnglishName`، `Segment`
- بازار: `TradeCount`، `Volume`، `Value`، `Close`، `Last`، `PreviousClose`، `Open`، `Low`، `High`، `Change`، `ChangePct`
- حقیقی/حقوقی: تعداد و حجم خرید و فروش در ستون های `Individual*` و `Legal*`
- پنج سطح سفارش: `BidPrice1..5`، `BidCount1..5`، `BidVolume1..5` و ستون های متناظر `Ask*`
- مشتقه: `ExpiryJalali`، `SettlementPrice`، `OpenInterest`، `ContractSize`، `ContractUnit` در صورت انتشار منبع

بورس کالا مقدار `Value` این تابلو را با واحد هزار ریال منتشر می کند؛ بنابراین `ValueUnit='thousand_rial'` و `DataFrame.attrs['value_unit']` ثبت می شود. خالی بودن تابلوی آتی در یک روز، خروجی معتبر با `schema` کامل است و به خطای شبکه تبدیل نمی شود.

تابلوی جزئی قراردادهای اختیار کالایی در وبسایت رسمی IME از کانال realtime SignalR منتشر می‌شود و قرارداد HTTP پایدار و قابل بازپخش ندارد. نسخه ۱.۶.۰ برای جلوگیری از داده ناقص یا حدسی، option chain کالایی نمی‌سازد؛ اما آمار تجمیعی رسمی بازار اختیار را از `get_commodity_market_activity()` ارائه می‌دهد.

بازار فیزیکی و نمای کل بورس کالا

```
daily = att.get_commodity_physical_history(
    start="1405-01-01",
    end="1405-06-01",
)

halls = att.get_commodity_physical_summary(
    start="1405-01-01",
    end="1405-06-01",
    hall="all",
)

markets = att.get_commodity_market_activity(
    start="1405-01-01",
    end="1405-06-01",
    market="all",
)
```

- تاریخ شروع الزامی؛ شمسی یا میلادی با قرارداد تاریخ عمومی پکیج : `start`
 - باشد تاریخ تقویم تهران در زمان فراخوانی استفاده می‌شود `None` تاریخ پایان؛ اگر : `end`
 - شناسه تالار یا نام دقیق تالار رسمی، `all` مقدار `get_commodity_physical_summary()` در : `hall`
 - شناسه بازار یا نام دقیق رسمی؛ شناسه ۱ فیزیکی، ۲ آتی، `all` مقدار `get_commodity_market_activity()` در : `market`
۳. اختیار معامله، ۴ مالی و ۵ گواهی سپرده است

را می‌دهد `Value` و `Volume` سری روزانه `get_commodity_physical_history(start, end=None)`

`Volume` , `OfferVolume` برای هر تالار `get_commodity_physical_summary(start, end=None, hall='all')`

را می‌دهد. نسبت معامله به عرضه فقط وقتی مخرج رسمی غیرصفر باشد محاسبه می‌شود `Value` و `TradeToOfferRatio`

حجم و ارزش تجمیعی بازارهای رسمی را برمی‌گرداند؛ `get_commodity_market_activity(start, end=None, market='all')`

`attrs` است و بازه ورودی در `ValueUnit='million_rial'` این تابع جزئیات قرارداد اختیار یا آتی نیست. در هر سه تابع تاریخی نیز ثبت می‌شود

منحنی آتی، calendar spread و cash-and-carry

```
# contracts می‌تواند خروجی آتی انرژی/کالا یا DataFrame باشد.
curve = att.get_futures_curve(
    contracts,
    underlying="میعانات گازی",
    valuation_date="1405-07-01",
    price="settlement",
    spot_price=100_000,
)

spreads = att.get_calendar_spreads(curve)

carry = att.analyze_cash_and_carry(
    curve,
    annual_rate=0.30,
    storage_rate=0.02,
    convenience_yield=0.01,
)
```

```
get_futures_curve(contracts, underlying=None, valuation_date=None, price='settlement',
                  spot_price=None):
```

- و حداقل یکی از `MaturityDate` یا `Expiry` ، `ExpiryDate` با ستون سررسید records یا `DataFrame` : `contracts` .
`SettlementPrice` , `Close` , `Last` .
- `Underlying` ؛ اگر وجود نداشته باشد از نام ردیف استفاده می‌شود/ `BaseAsset` فیلتر اختیاری روی : `underlying` .
- ثبت می‌شود attrs یعنی تاریخ تقویم تهران و در `None` تاریخ ارزش‌گذاری؛ : `valuation_date` .
- ستونی را که انتخاب شده نشان می‌دهد `PriceSource` . `last` یا `close` , `settlement` اولویت قیمت : `price` .
- را از نام یا قیمت دیگری حدس spot محاسبه نمی‌شود؛ پکیج basis قیمت نقدی مثبت و صریح. بدون آن : `spot_price` نمی‌زند.
- حفظ می‌شود؛ ترکیب قراردادهایی با واحدهای calendar spread اگر در ورودی موجود باشد، در منحنی و : `ValueUnit` متوقف می‌شود `InvalidParameterError` متفاوت با

خروجی بر اساس سررسید مرتب است و `DaysToExpiry` , `AbsoluteBasis` , `BasisPct` و `AnnualizedBasis` دارد. قرارداد بدون سررسید معتبر حذف می‌شود.

`Spread = FarPrice - NearPrice` سررسیدهای مجاور را جفت می‌کند و deterministic به صورت `get_calendar_spreads(curve)` `flat` یا `backwardation` , `contango` با مقادیر `Structure` سالانه و `spread` ، فاصله روز ، `spread` درصد ، `NearPrice` می‌سازد.

نرخ‌ها را به صورت `analyze_cash_and_carry(curve, annual_rate, storage_rate=0.0, convenience_yield=0.0)` برابر است با attrs می‌گیرد. فرمول ثبت‌شده در effective اعشاری و سالانه

$$\text{FairValue} = \text{SpotPrice} \times (1 + \text{annual_rate} + \text{storage_rate} - \text{convenience_yield}) ** (\text{DaysToExpiry} / 365)$$

خروجی `FairValue` , `Mispricing` , `MispricingPct` و `signal` محاسباتی دارد. `signal` هزینه معامله، وجه تضمین، مالیات، محدودیت تحویل و امکان اجرای واقعی آربیتراژ را مدل نمی‌کند و توصیه معاملاتی نیست.

قرارداد منبع، تاریخ و کیفیت داده ۱.۶.۰

- منشأ را مشخص می‌کند `DataFrame.attrs` در `source` در هر ردیف و `Source`
- انتخاب شده کامل پردازش شده است، نه اینکه منبع همه تاریخ بازار را نگهداری `endpoint` یعنی پاسخ `is_partial=False` می‌کند.
- آتی هزینه درخواست جدا دارد `enrichment` تعداد فراخوانی‌های همان تابع را نشان می‌دهد؛ `request_count`
- و گزارش‌های تاریخی `thousand_rial` `IME` تابلوی مالی، `rial` را پیش از جمع‌زدن بررسی کنید: بورس انرژی `ValueUnit` هستند `IME million_rial`
- تاریخ‌های حراج `timestamp` میلادی منبع‌اند و تاریخ شمسی متناظر بدون تغییر روز تقویمی ساخته می‌شود. تاریخ‌های گزارش فیزیکی از تاریخ شمسی رسمی منبع به میلادی تبدیل می‌شوند.
- داده حراج، `snapshot` تابلو و گزارش تجمیعی سه سطح پوشش متفاوت‌اند و نباید بدون کلید هویت/تاریخ و کنترل واحد با هم `join` شوند.

سایر API‌های بازار

این بخش قابلیت‌های قدیمی را در همان مرجع واحد نگه می‌دارد. API‌های `legacy` برای `backward compatibility` در دسترس‌اند و در بسیاری از خطاهای قدیمی `None` /پیام کنسول می‌دهند؛ API‌های جدید بیشتر از `typed exception` استفاده می‌کنند.

intraday

```
ticks_or_candles = att.get_intraday(
    "فملى",
    interval="1min",          # tick, 1min, 5min, 15min, 30min, 1h, 4h, 12h
    start="1403-05-01",      # برای امروز start/end حذف
    end="1403-05-03",
    progress=False,
)
```

خروجی `candle` معمولاً `Open, High, Low, Close, Volume` با `index` زمانی است. `snapshot` `tick` خام معاملات/قیمت را می‌دهد. `interval` بزرگ‌تر از داده پایه `resample` می‌شود؛ تعطیلی و وقفه بازار را در محاسبه لحاظ کنید. نام‌های `canonical` بازه `1m` , `60min` , `4hour` , `240m` , `12h` هستند؛ `alias`‌های عددی/کوتاه مانند `1min` , `5min` , `15min` , `30min` , `1h` , `4h` و `12hour` , `720` و نیز `raw` / `ticks` پشتیبانی می‌شوند. این API رفتار `legacy` دارد: `interval` نامعتبر یا تاریخی که `validator` قدیمی نامعتبر تشخیص دهد را روی کنسول اعلام می‌کند و `None` برمی‌گرداند، نه اینکه عمداً `ValueError` قراردادشده‌ای بدهد.

اطلاعات نماد و سهامدار

```

detail = att.get_detail("فملی")
info = att.get_info("فملی")
stats = att.get_stats("فملی")
shareholders = att.get_shareholders("فملی", include_id=True)

# تاریخ معامله است؛ خروجی تاریخ مؤثر روز معاملاتی بعد را هم نشان می‌دهد
dated = att.get_shareholders("فولاد", date="1405-06-18", include_id=True)

# start/end در این تابع تاریخ‌های مؤثر منتشرشده
history = att.get_shareholder_history(
    "فولاد",
    start="1404-01-01",
    end="1404-12-29",
    frequency="daily",      # daily | weekly | monthly
    max_requests=150,
    progress=False,
)
capital = att.get_capital_increase("فملی")

```

- می‌دهد **id** و ردیف **value** ستون **key** برابر **index** کلید-مقدار با **DataFrame|None** یک **get_detail()**
- می‌دهند **key** برابر **index** کلید-مقدار با **DataFrame** **get_stats()** و **get_info()**
- **تاریخ معامله** است و **date**، آخرین فهرست را می‌دهد. در حالت تاریخی **get_shareholders(date=None)** دارایی پس از پایان آن معامله را با آن TSETMC در خروجی تاریخ روز معاملاتی بعدی است که **date/effective_date** هر دو مفهوم را صریح می‌کند **trade_date** منتشر می‌کند؛
- ترکیب سهامداران عمده یک نماد را روی **تاریخ مؤثر** می‌سازد **get_shareholder_history(start, end)** متوالی را استخراج می‌کند؛ برای نمونه ۲۳۹ جلسه سال ۱۴۰۴ به حدود ۱۲۰ snapshot از هر پاسخ دو **frequency='daily'** به‌ترتیب آخرین جلسه هفته معاملاتی ایران و آخرین جلسه ماه **monthly** و **weekly** درخواست نیاز دارد. فرکانس‌های شمسی را نگه می‌دارند.
- تاریخ گم‌شده از بودجه بیشتر شود، **retry** یک سقف سخت پیش از دریافت است؛ اگر پوشش جفتی یا **max_requests** شناسه **include_id=True** تاریخ شمسی یا میلادی می‌پذیرند و **start/end** می‌دهد **InvalidParameterError** تابع را اضافه می‌کند؛ این شناسه هویت پایدار و قابل اتصال بین روزها نیست **snapshot رکورد**
- تاریخچه تغییر تعداد سهام/سرمایه را می‌دهد **get_capital_increase()**
- این توابع **ins_code** و **asset_type** keyword-only را نیز می‌پذیرند.

نمونه ستون‌های تاریخ و تغییر در خروجی تاریخی:

share_holder_name	change_amount	trade_date	effective_date	change_quality
Holder_A	225000000	20260907	20260908	exact_normalized_name

و **shareHolderID** فقط با تطبیق دقیق نام نرمال‌شده محاسبه می‌شود، زیرا snapshot از اختلاف دو **change_amount** برمی‌گردد؛ کل دارایی سهامدار **NA** پاسخ تاریخی در داده واقعی صفر هستند. برای نام تازه‌وارد یا مبهم مقدار **changeAmount** نیز انجام نمی‌شود fuzzy matching تازه به‌عنوان خرید همان روز فرض نمی‌شود و

قرارداد رسمی منبع: تاریخ دارایی یک روز معاملاتی بعد از معاملات است؛ تغییر دارایی می‌تواند ناشی از معامله یا سایر نقل‌وانتقالات باشد؛ اطلاعات ساعات بازار مربوط به پایان جلسه قبل است؛ و فهرست عمومی ممکن است جزئیات یا دقت لازم برای استعلام رسمی را نداشته باشد. این موارد در `DataFrame.attrs` با کلیدهای `change_includes_non_trade_transfers` , `during_session_represents_previous_completed_session` , `is_authoritative_register` و `official_verification_required` ثبت می‌شوند.

فعالیت اخیر سهامداران عمده و timeline نماد

دو سطح داده سهامداران را از هم جدا نگه دارید. `get_shareholders()` برای یک نماد و یک تاریخ دلخواه در سابقه معاملاتی است؛ API‌های زیر تابلوی سراسری «تغییرات سهامداران عمده» را می‌خوانند که فقط یک پنجره گردان با حداکثر پنج تاریخ منتشرشده دارد:

```

# Holdings in the five published recent-change snapshots
snapshots = att.get_major_shareholder_snapshots(
    days=5,
    enrich_identity=True,    # exact InsCode identity join
    progress=False,
)

# Latest published change versus its previous snapshot
changes = att.get_major_shareholder_changes(
    days=1,
    direction="both",       # both, increase, decrease, unchanged
    progress=False,
)

# Exact symbol or holder filter
foolad = att.get_major_shareholder_changes(
    "1405-06-18", ins_code="46348559193224090"
)
holder = att.get_active_shareholders(holder="Exact Holder Name", days=5)

# Rank each holder/instrument pair
ranking = att.rank_shareholder_accumulation(
    days=5,
    direction="accumulation", # both | accumulation | distribution
    metric="percent",         # percent | shares
    top=20,
    progress=False,
)

# Latest-snapshot bipartite edge list
network = att.get_shareholder_network(
    min_holdings=1_000_000,
    progress=False,
)

# Per-instrument major-holder concentration
concentration = att.get_ownership_concentration(
    ins_code="46348559193224090", top_n=5, progress=False
)

# TSETMC-native symbol timeline
events = att.get_symbol_events(
    ins_code="778253364357513",
    start="1400-01-01",
    kinds=("messages", "capital_changes", "price_adjustments"),
    limit=100,
    progress=False,
)

```

خروجی نماینده `get_major_shareholder_changes()` :

HolderName	InsCode	Symbol	PreviousGregorianDate	GregorianDate	PreviousHoldings	Holdings
ChangeShares	Direction					
115000000	100000000		09-09-2026	08-09-2026	وبملت	سهامدار الف ...
15000000	increase					

خروجی `get_major_shareholder_snapshots()` تمام مقدارهای منتشرشده را با `HolderRecord`, `HolderName`, `InsCode`, `Symbol`, `InstrumentName`, `GregorianDate`, `JalaliDate`, `Holdings`, `Source`, `FetchAt` خروجی `changes` مقدارهای `ChangeShares`, `Holdings`, `PreviousHoldings` و `Direction` را اضافه می‌کند.

`get_active_shareholders()` برای هر جفت ردیف سهامدار/ابزار، مقدار اولیه و آخرین دارایی، تغییر خالص، افزایش و کاهش ناخالص و تعداد روزهای فعال را می‌دهد.

بر اساس قدرمطلق درصد `metric='percent'` همین جفت‌های سهامدار-نماد را با `rank_shareholder_accumulation()` بر اساس قدرمطلق تعداد سهم رتبه‌بندی می‌کند. تعداد سهم چند `metric='shares'` تغییر نسبت به دارایی ابتدای پنجره، و با شرکت با هم جمع نمی‌شود؛ جمع‌زدن سهم نمادهایی با سرمایه و قیمت متفاوت یک معیار مالی قابل‌اتکا نمی‌سازد.

باشد `distribution` یا `accumulation`، `both` می‌تواند `direction`.

دو سر یال‌اند و `HolderNode` و `InstrumentNode`: دوبخشی می‌دهد `edge list` یک `get_shareholder_network()` درجه هر گره را در همان `InstrumentHolderCount` و `HolderInstrumentCount` وزن یال است. ستون‌های `Holdings` `min_holdings` فقط آخرین تاریخ یا یک تاریخ موجود در پنجره پنج‌تایی را می‌پذیرد؛ API نشان می‌دهند. این snapshot یال‌های کوچک‌تر را حذف می‌کند.

برخلاف دو تحلیل قبلی از فهرست سهامداران عمده یک نماد استفاده می‌کند. خروجی `get_ownership_concentration()` بر پایه درصد کل شرکت و `MajorHolderHHI`، `Top-N` و `Top-1/3/5/10` شامل درصد افشاشده، سهم پس از نرمال‌کردن فقط بخش افشاشده است. باقیمانده گزارش‌نشده به صورت یک سهامدار فرض `NormalizedDisclosedHHI` کامل است و این شاخص‌ها تمرکز فهرست عمومی سهامداران عمده‌اند، نه `HHI` کران پایین `MajorHolderHHI` نمی‌شود؛ بنابراین دفتر رسمی کامل مالکیت.

محدودیت ضروری منبع: این feed فهرست کامل همه سهامداران عمده نیست؛ فقط مواردی را نشان می‌دهد که در تابلوی تغییرات اخیر TSETMC حاضرند. حداکثر پنج تاریخ منتشرشده در دسترس است و `backfill` سمت سرور وجود ندارد. مقدار `available_dates` را از `DataFrame.attrs` بخوانید و «امروز» را فرض نکنید. تغییر دارایی می‌تواند از معامله یا سایر نقل‌وانتقال‌ها ناشی شود. در ساعات معامله، دارایی منتشرشده مربوط به جلسه تکمیل‌شده قبلی است و تغییرات همان روز پس از پردازش پایان جلسه منتشر می‌شود. این اطلاعات برای اتکای حقوقی کافی نیست و باید از مرجع ذی‌صلاح استعلام شود.

چون endpoint شناسه پایدار سهامدار نمی‌دهد، `HolderRecord` فقط شماره ردیف همان پاسخ است و هویت پایدار بین دو فراخوانی محسوب نمی‌شود. نام‌های مشابه، به‌ویژه «شخص حقیقی»، با هم ادغام نمی‌شوند. `enrich_identity=True` فقط `Symbol` و `InstrumentName` را با اتصال دقیق `InsCode` از snapshot جاری تکمیل می‌کند؛ تعداد ابزارهای بدون تطبیق در `unmatched_identity_count` ثبت می‌شود.

را یکجا می‌آورد: پیام ناظر، تغییر تعداد سهام شرکت و تعدیل قیمت. TSETMC فقط داده‌های بومی `get_symbol_events()` `attrs['codal_included'] ==` نیستند؛ timeline هیئت‌مدیره و صورت‌های مالی داده افشای کدال‌اند و در این، DPS، مجمع تأییدشده تلقی DPS این مرز را ماشین‌خوان نگه می‌دارند. رخداد تعدیل قیمت نیز `excluded_disclosure_kinds` و `False`

نمی‌شود.

معرفی شرکت؛ API قدیمی unsupported

قدیمی مانده‌اند. معرفی ناشر دادهٔ import/signature فقط برای حفظ `stock_introduction()` و `get_introduction()` یا شبکه خطای زیر را می‌دهد **resolver** این پکیج قرار دارد؛ تابع همیشه پیش از source boundary کدال است و خارج از

```
try:
    att.get_introduction("فملی")
except att.UnsupportedDataSourceError as exc:
    print(exc)
```

UnsupportedDataSourceError: get_introduction/stock_introduction requires Codal data, which is outside algotik-tse's TSETMC market-data source boundary

برای اطلاعات TSETMC از `get_info()` و `get_detail()` استفاده کنید. helper افشای ناشر یا history آن در این پکیج وجود ندارد.

فهرست نمادها و ابزارها

```
symbols = att.get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False, options=False,
    mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=False,
)

etfs = att.list_etfs(progress=False)
bonds = att.list_bonds(progress=False)
debt = att.list_debt_instruments(["erad", "gam"], active_only=True, progress=False)
funds = att.list_funds(fund_type="fixed_income", progress=False)
listed_funds = att.list_listed_funds(progress=False)
options = att.list_options(underlying="خودرو", progress=False)
indices = att.list_indices(progress=False)
members = att.get_index_companies("شاخص صنعت بانکها", progress=False)
industry_indices = att.list_industry_indices(progress=False)
industry_snapshot = att.get_industry_snapshot("بانک", progress=False)
```

بازار/نوع ابزار را نگه می‌دارد metadata `dataframe` فقط نام نمادها را می‌دهد؛ `get_symbols(output="list")` ها را صریح تنظیم asset-type flag، اشتباه universe است. برای جلوگیری از `قرمز`، `نارنجی`، `زرد` یکی از `payeh_color` کنید.

`list_bonds()` را می‌دهد NAV/discount برای سازگاری قبلی فقط صندوق‌های بورسی عمومی نوع ۳۰۵ و `list_etfs()` و فیلتر `DebtType` ستون `list_debt_instruments()` اوراق و سررسید را با امضای قدیمی فهرست می‌کند؛ metadata مرجع سایر اوراق در فصل درآمد ثابت آمده است YTM محاسباتی اخزا و analytics. مستقیم اخزا/اراد/گام را اضافه می‌کند.

ابزارهای واقعاً قابل معامله نوع ۳۰۵ و ۳۸۰ را با `list_listed_funds()` صندوق‌هاست؛ `registry` `list_funds()` دقیق و طبقه‌بندی قابل فیلتر می‌دهد `InsCode/ISIN`.

شاخص‌ها

```
index_history = att.get_history("شاخص کل", limit=100, progress=False)
industry_history = att.get_industry_history("بانک", limit=100, progress=False)
members = att.get_industry_members("بانک", progress=False)
```

برای شاخص‌های عمومی همچنان `get_history()` را به‌کار ببرید. برای شاخص صنعت، API‌های اختصاصی فصل **شاخص‌ها و تحلیل صنایع** عضویت رسمی، تاریخچه اعضا، `snapshot` و `intraday` را یک‌دست ارائه می‌کنند. `schema` شاخص با سهام فرق دارد و `volume` جعلی ساخته نمی‌شود. `list_indices()` و `get_index_companies()` برای سازگاری با کد قدیمی حفظ شده‌اند.

طلا، ارز، نقره و سکه از TGJU

با همان رفتار قبلی حفظ `currency_coin()` و `get_currency()` است. `provider` نام عمومی این `get_tgju_history()` شده‌اند و اکنون تمام دارایی‌های کاتالوگ را هم می‌پذیرند:

```
fx = att.get_currency(
    "dollar", start="1403-01-01",
    output_type="standard", date_format="jalali", progress=False,
)

coins = att.get_currency(["seke", "nim-seke"], limit=10, progress=False)
gold = att.get_tgju_history(
    ["gold-18k", "طلای ۲۴ عیار", "gold-mesghal"],
    limit=30,
    date_format="both",
    progress=False,
)
```

پیش از انتخاب نام، کاتالوگ را ببینید:

```
catalog = att.list_tgju_assets()
gold_assets = att.list_tgju_assets(category=["gold", "silver"])
print(gold_assets[["Name", "PersianName", "Slug", "Unit"]])
```

گروه‌های کاتالوگ:

دسته	تعداد	نمونه‌ها	واحد
currency	۳۶	دلار، یورو، فرانک، ۱۰۰ ین، ارزهای منطقه	ریال برای هر واحد؛ ین ژاپن برای ۱۰۰ واحد
official_rate	۲۴	سری‌های منتشرشده سنا، نیما و بانکی	ریال برای هر واحد

دسته	تعداد	نمونه‌ها	واحد
gold	۱۱	۱۸ و ۲۴ عیار، مثقال، دست‌دوم، آب‌شده، عیار ۷۴۰	ریال بر گرم یا مثقال
silver	۲	نقره ۹۲۵ و ۹۹۹	ریال بر گرم
coin / coin_bubble	۵ + ۵	امامی، بهار آزادی، نیم، ربع، گرمی و حباب هرکدام	ریال بر سکه
global_metal	۴	انس طلا، نقره، پلاتین و پالادیوم	دلار بر انس تروا

نام‌های پرکاربرد:

gold-18k, gold-24k, gold-mesghal, used-gold,
melted-gold-cash, melted-gold-wholesale, silver-999, silver-925,
gold-ounce, silver-ounce, platinum-ounce, palladium-ounce,
dollar, euro, yuan, dirham, pound, swiss-franc,
seke, seke-bahar-azadi, nim-seke, rob-seke, seke-gerami,
seke-bubble, nim-seke-bubble, rob-seke-bubble

نام canonical، نام فارسی و slug رسمی هر سه پذیرفته می‌شوند؛ بنابراین `طلای ۱۸ عیار`، `gold-18k` و `geram18` یک سری هستند. رقم فارسی/عربی و ی/ک عربی نرمال می‌شود. برای نام نامعتبر `InvalidParameterError`، برای HTTP ناموفق `ConnectionError` و برای payload ناسازگار `DataParsingError` صادر می‌شود.

خروجی standard ستون‌های `Open, High, Low, Close` دارد و در حالت چندداریایی یک `DataFrame` با `MultilIndex` ستونی برمی‌گردد. در حالت تک‌داریایی، `attrs` شامل `source`، `asset`، `persian_name`، `slug`، `category`، `unit` است. `save_path` پوشه مقصد CSV است. «قیمت» و واحد را از `Unit / attrs['unit']` بخوانید؛ مخصوصاً انس جهانی ریالی نیست و بین ژاپن به ازای ۱۰۰ بین منتشر می‌شود.

تنظیمات و خطاها

تنظیمات singleton:

```
from algotik_tse import settings

settings.ssl_verify = True          # پیش‌فرض و توصیه شده
settings.timeout = 10              # ثانیه
settings.max_retries = 3
settings.retry_backoff_factor = 0.3
settings.rate_limit_delay = 0.3    # فاصله حداقل شروع درخواست‌ها

settings.market_snapshot_freshness_seconds = 120.0
settings.market_clock_skew_tolerance_seconds = 5.0
settings.client_volume_consistency_tolerance = 0.05
settings.industry_membership_cache_ttl = 3600.0
settings.order_book_max_requests = 250
settings.trade_max_requests = 250
```

خراب CA فقط برای محیط کنترل‌شده با opt-out است. `True` پیش‌فرض TLS verification

```
from algotik_tse.http_client import safe_get

response = safe_get("https://cdn.tsetmc.com/...", verify=False)
```

این opt-out را سراسری نکنید. URLهای TSETMC همگی HTTPS هستند.

قرارداد دقیق (`safe_get(url, **kwargs)`):

- نامجاز با URL/Codal path های مجاز باشد؛ provider و داخل HTTP(S) باید `url: str` رد می‌شود I/O و session و rate-limit، پیش از `UnsupportedDataSourceError`
- فقط `headers=settings.headers`، `timeout=settings.timeout` و `verify=settings.ssl_verify` های default نداده باشد اعمال می‌شوند caller override وقتی
- هستند. نوع نادرست اولی/دومی wrapper پارامترهای خود `allow_redirects: bool=True` و `max_redirects: int=5` است `ValueError`، `max_redirects<0` و `TypeError`
- درخواست زیرین همیشه `allow_redirects=False` دارد. redirect فقط `scheme,host,effective-` same-origin (port) و hop-by-hop است؛ `UnsupportedDataSourceError` cross-origin و loop/عبور از سقف `requests.exceptions.TooManyRedirects` می‌دهد.
- redirect در caller های header دوباره فرستاده نمی‌شود؛ redirect فقط روی درخواست اول اعمال می‌شود و روی `params` حفظ می‌شوند same-origin
- خطاهای transport خود `requests` بعد از retry propagate می‌شوند. `safe_get` به‌تنهایی روی status 4xx/5xx `raise_for_status()` نمی‌کند؛ API مصرف‌کننده باید پیش از parse آن را به خطای typed خود تبدیل کند.

سلسله‌مراتب خطا:

```
AlgotikTSEError
├─ AmbiguousSymbolError
├─ ConnectionError
├─ DataParsingError
├─ InvalidParameterError
├─ StockNotFoundError
└─ UnsupportedDataSourceError
```

legacy سطح بالای پکیج نیست. توابع export برای سازگاری داخلی وجود دارد ولی exceptions در مازول `RateLimitError` و پیام کنسول بدهند؛ قرارداد هر تابع را بررسی کنید `None`، exception ممکن است به‌جای

الگوی امن:

```
try:
    df = att.get_price_adjustments(ins_code="35425587644337450")
except att.InvalidParameterError as exc:
    print("bad input", exc)
except att.AmbiguousSymbolError as exc:
    print("pass ins_code", exc)
except att.ConnectionError as exc:
    print("provider unavailable", exc)
except att.DataParsingError as exc:
    print("provider schema changed", exc)
except att.UnsupportedDataSourceError as exc:
    print("outside supported sources", exc)
```

مرجع تفصیلی همه توابع

در این بخش همه export های عمومی پوشش داده شده‌اند. هر ردیف API در کنار همان تابع، ورودی‌ها و گزینه‌های اختصاصی، خروجی، خطاهای اصلی و مثال را توضیح می‌دهد. جدول‌های پارامتر مشترک فقط تعریف اصطلاحات را یکسان نگه می‌دارند؛ signature دقیق هر تابع نیز بلافاصله پیش از جدول همان گروه آمده است.

نمای کلی export ها و نام‌های قدیمی

جدول زیر inventory کامل export های `algotik_tse.__all__` است. جزئیات خروجی در بخش موضوعی مربوط آمده است.

هویت، تنظیمات و خطا

کاربرد	Export
تنظیمات singleton شبکه/بازار	<code>settings</code>
هویت immutable ابزار	<code>InstrumentRef</code>
نرمال‌سازی و حل دقیق هویت	<code>normalize_instrument_text</code> , <code>validate_ins_code</code> , <code>resolve_instrument</code>
خطاهای عمومی	<code>AlgotikTSEError</code> , <code>AmbiguousSymbolError</code> , <code>ConnectionError</code> , <code>DataParsingError</code> , <code>InvalidParameterError</code> , <code>StockNotFoundError</code> , <code>UnsupportedDataSourceError</code>

قیمت، trades، client type و اطلاعات نماد

Alias/legacy عمومی	Export canonical
<code>stock</code>	<code>get_history</code>
<code>stock_RI</code> , <code>stock_RL</code>	<code>get_client_type</code>
<code>stock_capital_increase</code>	<code>get_capital_increase</code>

Alias/legacy عمومی	Export canonical
<code>stock_intraday</code>	<code>get_intraday</code>
—	<code>get_trades</code> , <code>get_live_trades</code>
<code>stockdetail</code>	<code>get_detail</code>
<code>stock_information</code>	<code>get_info</code>
<code>stock_statistics</code>	<code>get_stats</code>
<code>stock_introduction</code> (همیشه unsupported)	<code>get_introduction</code> (همیشه unsupported)
<code>shareholders</code>	<code>get_shareholders</code>
—	<code>get_shareholder_history</code> , <code>get_major_shareholder_snapshots</code> , <code>get_major_shareholder_changes</code> , <code>get_active_shareholders</code>
—	<code>rank_shareholder_accumulation</code> , <code>get_shareholder_network</code> , <code>get_ownership_concentration</code>
<code>stocklist</code>	<code>get_symbols</code>
<code>currency_coin</code>	<code>get_currency</code>

پارامترهای قدیمی نیز پذیرفته می‌شوند: `stock` → `symbol` , `values` → `limit` , `tse_format` → `raw` , `output_type="complete"` → `"full"` و `multi_stock_drop` / `multi_currencies_drop` → `dropna` در مسیرهای مربوط. برای کد جدید نام canonical را به‌کار ببرید.

live، سفارش، watcher و history محلی

هاExport
<code>get_market_snapshot</code> , <code>get_market_client_type</code> , <code>get_order_book</code> , <code>get_live_market</code> , <code>get_live_symbol</code>
<code>get_order_book_history</code> , <code>get_orderbook_history</code> , <code>get_queue</code> , <code>get_queue_history</code>
<code>MarketEvent</code> , <code>MarketWatcher</code> , <code>watch_market</code>
<code>get_market_messages</code> , <code>get_instrument_state_changes</code> , <code>get_market_overview</code> , <code>get_market_breadth</code> , <code>get_sector_flow</code>
<code>MARKET_HISTORY_SCHEMA_VERSION</code> , <code>MARKET_HISTORY_APPLICATION_ID</code> , <code>check_market_history</code>
<code>save_market_snapshot</code> , <code>load_market_snapshots</code> , <code>get_live_market_history</code>
<code>get_market_overview_history</code> , <code>get_market_snapshot_summary_history</code> , <code>get_market_breadth_history</code> , <code>get_sector_flow_history</code>
<code>record_market_event</code> , <code>get_market_event_history</code> , <code>archive_market_records</code>
<code>get_market_messages_history</code> , <code>get_instrument_state_changes_history</code>

Exportها
<code>get_trading_calendar</code> , <code>get_market_activity</code> , <code>get_market_value_history</code> , <code>get_index_impact</code> , <code>get_market_trades</code>
<code>get_theoretical_opening_price</code> , <code>get_preopen_imbalance</code>
<code>compare_symbols</code> , <code>get_liquidity_metrics</code> , <code>get_market_regime</code> , <code>get_market_map</code> , <code>plot_market_map</code>

سه wrapper صفرآرگومان legacy نیز عمومی‌اند: `market_watch()` , `market_client_type()` , `market_data()` .
 wrapper deprecated `market_data()` است؛ برای کد جدید `get_market_snapshot()` یا `get_live_market()` را انتخاب کنید.

fundamentals، تعدیل قیمت و ابزارها

Exportها
<code>get_market_fundamentals</code> , <code>get_market_fundamentals_history</code>
<code>get_price_adjustments</code> , <code>get_latest_price_adjustment</code>
<code>get_instrument_master</code> , <code>get_instrument_changes</code>
<code>list_options</code> , <code>get_options_chain</code> , <code>list_etfs</code> , <code>list_bonds</code> , <code>list_debt_instruments</code> , <code>list_funds</code> , <code>list_listed_funds</code>
<code>list_indices</code> , <code>get_index_companies</code>
<code>list_industry_indices</code> , <code>get_industry_members</code> , <code>get_industry_snapshot</code> , <code>get_industry_membership_events</code>
<code>get_industry_history</code> , <code>get_industry_members_history</code> , <code>get_industry_membership_overlap</code> , <code>get_industry_membership_churn</code> , <code>get_industry_intraday</code> , <code>rank_industries</code>

TGJU: ارز و فلزات گران بها

Exportها
<code>list_tgju_assets</code> , <code>get_tgju_history</code> , <code>get_currency</code> , <code>currency_coin</code>

درآمد ثابت

Exportها
<code>IRAN_TREASURY_FACE_VALUE</code> , <code>YieldCurve</code>
<code>parse_treasury_maturity</code> , <code>day_count_fraction</code> , <code>treasury_yield</code>
<code>bond_price</code> , <code>yield_to_maturity</code> , <code>bond_analytics</code> , <code>build_yield_curve</code>
<code>get_ifb_yield_table</code> , <code>get_debt_yields</code> , <code>get_treasury_yields</code>
<code>get_treasury_yield_history</code> , <code>get_treasury_yields_history</code>

ها Export

`get_yield_curve` , `get_yield_curve_history`

اختيار معامله

ها Export

`OPTION_SNAPSHOT_SCHEMA_VERSION`

`black_scholes_price` , `black_scholes_greeks` , `option_price_bounds` , `implied_volatility`

`get_option_market` , `analyze_option_chain` , `option_put_call_ratios`

`get_option_history` , `save_option_snapshot` , `load_option_snapshots`

بورس انرژی و بورس کالا

ها Export

`get_energy_auctions` , `get_energy_auction` , `get_energy_market_overview`

`list_power_instruments` , `list_energy_securities` , `get_energy_future_contract`

`get_commodity_market` , `get_commodity_physical_history` , `get_commodity_physical_summary` ,
`get_commodity_market_activity`

`get_futures_curve` , `get_calendar_spreads` , `analyze_cash_and_carry`

signatureهای پرکاربرد

```

get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
get_trades(symbol=None, *, ins_code=None, start=None, end=None,
            include_canceled=False, max_requests=None, raw=False, progress=True)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None)
get_market_overview(flow=0, *, archive_to=None)
compare_symbols(symbols, start=None, end=None, benchmark=None, align='inner', annualization=240,
min_observations=20, include_client_type=True, progress=True, history_data=None, client_data=None,
strict=False)
get_liquidity_metrics(symbols=None, start=None, end=None, window=20, annualization=240,
include_live=True, progress=True, history_data=None, live_data=None, strict=False)
get_market_regime(benchmark='شاخص کل', start=None, end=None, lookback=60, liquidity_window=20,
annualization=240, weights=None, progress=True, max_requests=30, live_data=None,
benchmark_history=None, activity_history=None)
get_market_map(group_by='symbol', size='value', color='return', top=100, flow=None,
instrument_types=None, data=None)
plot_market_map(map_data=None, output_path=None, show=False, title=None)
get_energy_auctions(status='upcoming', board='physical', top=100)
get_energy_auction(auction_id, include_trades=True, include_instrument_history=False)
get_energy_market_overview(market='all')
list_power_instruments(market='all', top=100)
list_energy_securities(kind='all', active=True, top=100, enrich_futures=True)
get_energy_future_contract(ins_code)
get_commodity_market(kind='all')
get_commodity_physical_history(start, end=None)
get_commodity_physical_summary(start, end=None, hall='all')
get_commodity_market_activity(start, end=None, market='all')
get_futures_curve(contracts, underlying=None, valuation_date=None,
                  price='settlement', spot_price=None)
get_calendar_spreads(curve)
analyze_cash_and_carry(curve, annual_rate, storage_rate=0.0,
                      convenience_yield=0.0)

```

پارامترهایی که با `_` شروع می‌شوند seam داخلی تست‌اند و API کاربر محسوب نمی‌شوند، حتی اگر در `inspect.signature` دیده شوند.

روش خواندن مرجع تفصیلی

هر signature زیر عین خروجی پایدارشده `inspect.signature` است؛ فقط آدرس حافظه `safe_get` به خود نام `safe_get` نرمال شده است. نوع‌هایی که در signature قدیمی annotation ندارند در جدول پارامترها مشخص شده‌اند. مقدار `None` معمولاً یعنی «فیلتر/override اعمال نشود»، نه رشته `"None"`. پارامترهای مشترک فقط یک‌بار در واژه‌نامه زیر توضیح داده می‌شوند و هر مدخل API علاوه بر آن، `override` و `constraint` خاص خود را می‌گوید. «خطاها» خطاهای اصلی قرارداد است، نه فهرست همه خطاهای ممکن Python/pandas.

پارامترهای مشترک هویت، تاریخچه و خروجی

نام	معمول type و default	معنا و constraint
<code>symbol</code> / <code>symbols</code>	<code>str</code> <code>Iterable[str]</code> <code>API</code> 'None'؛ بسته به <code>None</code> یا	نماد فارسی، <code>InsCode</code> یا مجموعه آن‌ها؛ برای هویت مبهم <code>ins_code</code> بدهید. <code>symbols=None</code> یعنی کل <code>universe</code> .
<code>ins_code</code> / <code>inscode</code>	<code>str</code> <code>int</code> <code>None</code> = <code>None</code>	شناسه دقیق ۱ تا ۲۰ رقم ASCII؛ با <code>selector</code> متناقض مجاز نیست. <code>inscode</code> فقط <code>spelling</code> تاریخی <code>reader</code> وضعیت است.
<code>asset_type</code>	<code>str</code> = <code>'auto'</code>	تعیین می‌کند <code>provider</code> نوع را از <code>auto</code> حل هویت؛ <code>hint</code>
<code>start</code> , <code>end</code> , <code>date</code>	<code>str</code> <code>date</code> <code>datetime</code> <code>None</code>	مرز شامل ابتدا/انتها؛ ISO شمسی یا میلادی در API‌های بازار. <code>date</code> در <code>get_shareholders</code> یعنی تاریخ دلخواه نماد؛ در API فعالیت سهامداران باید یکی از حداکثر پنج تاریخ منتشرشده باشد.
<code>limit</code>	ها <code>reader</code> یا <code>int</code> = <code>0</code> یا <code>1000</code>	<code>0</code> در <code>history</code> های <code>provider</code> یعنی بدون محدودیت بعد از فیلتر؛ در <code>SQLite</code> حداکثر صفحه. منفی نامعتبر است.
<code>offset</code>	<code>int</code> = <code>0</code>	بعد از فیلترها؛ نامنفی <code>SQL offset</code>
<code>include_today</code>	<code>bool</code> = <code>False</code>	ردیف زنده امروز؛ ممکن است در ساعات بازار آخرین <code>opt-in</code> زنده دارد <code>provenance</code> مشاهده همین لحظه باشد و
<code>raw</code>	<code>bool</code> = <code>False</code>	ممکن است <code>order-book</code> ؛ در <code>provider</code> نزدیک <code>schema</code> باشد <code>delta/raw-mixed</code> .
<code>output_type</code>	<code>str</code> = <code>'standard'</code>	خروجی؛ مقادیر دقیق تابعی اند <code>schema</code> (<code>complete</code> یا <code>long / wide</code> یا <code>standard / full</code>) است. <code>full</code> قدیمی <code>alias</code>
<code>date_format</code>	<code>str</code> = <code>'jalali'</code>	شکل <code>index/stون تاریخ</code> ؛ <code>gregorian</code> , <code>jalali</code> , <code>both</code> در مسیرهای پشتیبانی‌شده.
<code>ascending</code>	<code>bool</code> = <code>True</code>	ترتیب زمانی خروجی بعد از فیلتر.
<code>progress</code>	<code>bool</code> = <code>True</code>	فقط پیام پیشرفت؛ در داده و <code>schema</code> اثر ندارد.
در فعالیت سهامداران <code>days</code>	<code>int</code> = <code>5</code>	تعداد تاریخ‌های انتهایی از پنجره گردان <code>provider</code> ؛ فقط ۱ تا ۵. تاریخ قدیمی‌تر <code>backfill</code> نمی‌شود.
<code>holder</code>	<code>str</code> <code>None</code> = <code>None</code>	فیلتر <code>exact</code> پس از نرمال‌سازی فارسی/عربی؛ شناسه پایدار سهامدار در <code>feed</code> سراسری موجود نیست.
<code>enrich_identity</code>	<code>bool</code> = <code>True</code>	تکمیل <code>Symbol/InstrumentName</code> فقط با اتصال دقیق <code>InsCode</code> به <code>MarketWatch</code> ؛ شکست <code>enrichment</code> داده مالکیت را حذف نمی‌کند و در <code>attrs</code> ثبت می‌شود.
<code>save_to_file</code>	<code>bool</code> = <code>False</code>	ذخیره <code>CSV</code> <code>opt-in</code> .
<code>save_path</code>	<code>str</code> <code>Path</code> <code>None</code>	پوشه مقصد، نه نام فایل؛ بدون <code>save_to_file=True</code> نوشته نمی‌شود.

نام	معمول type و default	معنا و constraint
dropna	bool = True	حذف ردیف/ستون کاملاً تهی طبق قرارداد همان API؛ ردیف partial معنادار order-book حفظ می‌شود.
return_type	str None	قیمت/ history سازگاری برای انتخاب نوع خروجی در alias را ترجیح دهید output_type ارز؛ برای کد جدید
max_requests	int None	سقف سخت fan-out قبل/حین I/O؛ معنای دقیق شمارش در مدخل تابع آمده است.
strict / selector_strict	bool	را فعال می‌کند؛ match خطای داده/فیلتر بدون strict مبهم/گم‌شده را scalar انتخاب selector_strict خطا می‌کند
allow_stale / include_stale	bool	اجازه نگه‌داشتن snapshot/اوراق stale؛ stale بودن همچنان در ستون attrs گزارش می‌شود.
archive_to , record_to , snapshot_path , path	str Path None	فایل SQLite/JSON صریح؛ نوشتن فقط با opt-in. path در reader/writer اجباری است.
kwargs	های سازگاری keyword	فقط aliasهای مستند مانند stock , values , tse_format و نام‌های انگلیسی stocklist keyword ناشناخته قرارداد عمومی نیست.
_request , _snapshot , _client_type , _clock , _wait , _random , _recorded_at , _live_fetch , _request_budget_state	داخلی seam	فقط تزریق deterministic در تست؛ برای مصرف عادی استفاده نشود و BC عمومی برای آن تضمین نمی‌شود.

پارامترهای مشترک live، watcher و SQLite

نام	type/default	معنا و constraint
flow	int None = 0	بازار/جریان provider؛ None در analytics یعنی بدون فیلتر.
sector	str Iterable None	فیلتر SectorCode .
instrument_types	Iterable[int] None	. 300,303,309 ابزار؛ پیش‌فرض تحلیلی سهام universe
traded_only	bool = False	فقط ابزار دارای معامله را در denominator نگه می‌دارد.
include_base_market	bool = True	نوع 309 بازار پایه را در universe پیش‌فرض نگه می‌دارد.
side	str = 'both'	برای صف both یا buy , sell
complete_only	bool = False	فقط snapshotهای پنج‌سطح کامل.
top	int = 20	تعداد پیام/وضعیت در درخواست؛ مثبت و bounded.
direction	str='both'	در تغییرات سهامدار increase , decrease , unchanged فقط تغییرهای غیرصفر؛ یا both
کinds در timeline	str Iterable[str]	یک یا چند مورد از , capital_changes , messages price_adjustments ؛ هیچ kind کدال پذیرفته نمی‌شود.

نام	type/default	معنا و constraint
<code>since_id</code>	<code>int str None</code>	فیلتر client-side رکوردهای پس از شناسه؛ cursor provider نیست.
<code>source</code>	<code>str = 'tsetmc'</code>	می‌تواند fixed-income ؛ در archive هویت/provenance باشد hybrid یا حالت مستند <code>tsetmc</code> .
<code>session_id</code>	<code>str None</code>	شناسه session watcher برای نوشتن/فیلتر replay.
<code>kind</code>	<code>str None</code>	kind یا <code>initial/delta/heartbeat/resync</code> های event پشتیبانی‌شده archive.
<code>recorded_at</code> , <code>as_of</code>	<code>None</code> یا timestamp-like	زمان مشاهده؛ <code>None</code> یعنی ساعت تهران/UTC داخلی معتبر تابع.
<code>max_records</code> , <code>record_max_records</code>	<code>int = 10000</code>	بر اساس تعداد؛ مثبت retention
<code>retention_seconds</code> , <code>record_retention_seconds</code>	<code>float None</code>	یعنی غیرفعال <code>None</code> زمانی؛ retention
<code>interval</code>	intraday: <code>str</code> در watcher <code>float=1.0</code> در	یا <code>tick/1min/5min/15min/30min/1h</code> (candle interval) بر حسب ثانیه polling فاصله
<code>max_updates</code>	<code>int None</code>	سقف eventهای iterator؛ <code>None</code> یعنی تا stop.
<code>notifications</code>	پیش‌فرض iterable، ('messages', 'state')	فقط این دو notification پشتیبانی می‌شوند؛ Codal وجود ندارد.
<code>error_policy</code> , <code>callback_error_policy</code> , <code>storage_error_policy</code>	<code>str</code>	enum های دقیق: <code>error_policy∈{retry,raise,stop}</code> ، <code>callback_error_policy∈{raise,ignore,stop}</code> و <code>storage_error_policy∈{raise,ignore}</code> ؛ مقدار نامعتبر خطا است I/O پیش از
<code>max_backoff</code> , <code>jitter</code> , <code>request_timeout</code>	<code>float None</code>	هر درخواست؛ نامنفی/مثبت طبق timeout و jitter ، سقف backoff کلاس.
<code>max_consecutive_retries</code> , <code>max_retries</code>	<code>int None</code>	سقف retry؛ <code>max_retries</code> نام قدیمی سازگار است.
<code>include_initial</code> , <code>emit_heartbeats</code> , <code>copy_snapshot</code> , <code>record_heartbeats</code>	<code>bool</code>	کنترل event emission/copy/persistence ها.
<code>notification_top</code> , <code>max_seen_notifications</code> , <code>checkpoint_interval</code>	<code>int</code>	بین ۱ و ۱۰۰۰؛ دو مقدار دیگر مثبت <code>notification_top</code> ، بین ۱ و ۱۰۰۰۰ است <code>record_max_records</code>

پارامترهای تحلیل مدیریتی

نام	type/default	معنا و constraint
<code>benchmark</code>	<code>str None</code>	شاخص یا نماد مبنا؛ در مقایسه اختیاری و در رژیم بازار پیش‌فرض شاخص کل است.

نام	type/default	معنا و constraint
align	str='inner'	برای اتحاد outer برای تقویم مشترک همه سری‌ها یا inner تاریخ‌ها.
annualization	int=240	تعداد جلسات سال برای نوسان، آلفا و tracking error؛ عدد صحیح ۱ تا ۳۶۶.
min_observations	int=20	حداقل بازده معتبر برای سنج‌های ریسک؛ کمبود با NaN و flag گزارش می‌شود.
include_client_type	bool=True	دریافت حقیقی/حقوقی در مقایسه؛ خاموش کردن آن fan-out را کم می‌کند.
history_data	Mapping[str, DataFrame] None	تاریخچه قیمت صریح به تفکیک نماد؛ جایگزین دریافت شبکه و بدون .mutation
client_data	Mapping[str, DataFrame] None	تاریخچه حقیقی/حقوقی صریح به تفکیک نماد.
window	int None=20	آخرین جلسات سنج نقدشوندگی؛ None یعنی تمام بازه موجود.
include_live	bool=True	افزودن گردش، spread، imbalance، depth و market cap از snapshot.
live_data	DataFrame None	برای تکرارپذیری و get_live_market() صریح snapshot جلوگیری از دریافت تازه.
lookback	int=60	تعداد مشاهدات انتهایی benchmark در مؤلفه trend.
liquidity_window	int=20	تعداد روزهای انتهایی فعالیت بازار برای میانه ارزش تاریخی.
weights	Mapping[str, number] None	وزن trend/breadth/flow/liquidity/queue ؛ نامنفی، finite و با مجموع مثبت.
benchmark_history	DataFrame None	تاریخچه صریح benchmark برای رژیم بازار.
activity_history	DataFrame None	تاریخچه صریح فعالیت کل بازار با ستون Value .
group_by	str='symbol'	گروه‌بندی نقشه: symbol ، sector ، flow ، instrument_type .
size	str='value'	اندازه: value ، volume ، market_cap ، queue_value ؛ treemap.
color	str='return'	رنگ: return ، net_individual_flow ، individual_power ، order_imbalance .
instrument_types	scalar/iterable/ None / 'all'	پیش‌فرض انواع سهامی ۳۰۹/۳۰۳/۳۰۰؛ 'all' فیلتر نوع را حذف می‌کند.
data	DataFrame None	ورودی صریح get_market_map() ؛ در نبود آن snapshot زنده گرفته می‌شود.
map_data	DataFrame None	جدول آماده نقشه برای renderer؛ None نقشه پیش‌فرض را می‌سازد.
output_path	str Path None	مسیر فایل HTML تعاملی؛ None یعنی فقط ساخت Figure .

نام	type/default	معنا و constraint
show	bool=False	نمایش figure با renderer تنظیم شده Plotly.
title	str None	عنوان دلخواه نمودار؛ None عنوان خودکار فارسی.

پارامترهای درآمد ثابت و اختیار

نام	type/default	معنا و constraint
settlement_date , maturity_date , issue_date , valuation_date	None یا date-like	تاریخ تسویه/سررسید/انتشار/ارزش گذاری؛ None در live یعنی امروز تهران.
face_value	float ؛ اخزا 1_000_000.0	ارزش اسمی مثبت؛ IRAN_TREASURY_FACE_VALUE همین default است.
day_count / convention	str='ACT/365F'	پشتیبانی شده؛ تاریخ پایان باید بعد از شروع convention باشد.
price , annual_yield , coupon_rate , accrued_interest	float	قیمت/بازده/کوپن/بهره تحقق یافته؛ bounds در تابع math اعتبارسنجی می شود.
cashflows	iterable (date, amount) یا None	جریان های نقدی؛ در حالت None از maturity/face/coupon ساخته می شود.
compounding , frequency , price_type	str , int , str	نوع مرکب، دفعات سالانه و dirty/clean .
nodes	DataFrame/iterable mapping	rate/discount و maturity شامل curve های node؛ تعیین کننده تکرار است duplicate policy
interpolation , extrapolate , duplicate_policy	log_discount , False , تابعی	سیاست in-extrapolation opt: discount interpolation ؛ مستند policy یا error duplicate
price_source	str='auto'	انتخاب Last/Close/Final/bid/ask با provenance.
maturity_map	None یا mapping	جلوگیری fuzzy صریح نماد→سررسید؛ از حدس override می کند
spot , strike , option_price , volatility	float	نارمفی volatility نامفی و premium ، مثبت spot/strike
time_to_expiry , rate , dividend_yield	float	سال تا سررسید، نرخ بدون ریسک و yield پیوسته.
option_type , exercise_style	str='call' , str='european'	را می پذیرد European فعلی فقط math call/put ؛
lower_volatility , upper_volatility , tolerance , max_iterations	0.0 , 5.0 , تابعی، تابعی	و شمار lower < upper : solver و همگرایی bracket مثبت iteration
risk_free_rate , yield_curve	None یا scalar/curve	نرخ ثابت یا YieldCurve ؛ اگر هر دو داده شوند قرارداد تحلیل آن ها را اعتبارسنجی می کند.

نام	type/default	معنا و constraint
parity_tolerance , liquidity_weights	float mapping None	default یعنی None scoring: و وزن‌های band parity داخلی مستند.

پارامترهای بورس انرژی، کالا و term structure

نام	type/default	معنا و constraint
status	str='upcoming'	وضعیت حراج: upcoming/ready , active , surplus/mazad , ended یا all .
board	str='physical'	تابلوی حراج: physical , power , special , green , free یا all .
auction_id	int	شناسهٔ مثبت حراج رسمی بورس انرژی؛ سقف قراردادی 10**12 .
include_trades	bool=True	افزودن معاملات خود همان حراج به dict خروجی.
include_instrument_history	bool=False	افزودن سابقهٔ عرضه‌های قبلی همان ابزار؛ با معاملات حراج جاری یکی نیست.
market در انرژی	str='all'	در برق: standard/green/free/all ؛ در overview یکی از label flowهای مستند؛ در activity کالا شناسه/نام بازار.
kind در انرژی	str='all'	standard_salaf , capacity_certificate , energy_saving_certificate , deposit_certificate , future یا all .
kind در کالا	str='all'	certificate , standard_salaf , fund , future یا all .
active	bool=True	دستهٔ جاری؛ False دسته‌های جاری و expired منتشرشده را نیز می‌خواند.
enrich_futures	bool=True	مشخصات رسمی آتی برای سررسید/اندازه/تسویه؛ هزینه در fan-out request_count .
hall	str int='all'	همه، ID تالار یا نام exact تالار رسمی بازار فیزیکی.
contracts	DataFrame iterable[Mapping]	ردیف‌های قرارداد با ستون سررسید و یکی از قیمت‌های settlement/close/last .
underlying	str None	فیلتر substring اختیاری روی دارایی پایه؛ fuzzy identity resolver نیست.
price در curve	str='settlement'	اولویت ستون قیمت: settlement , close یا last ؛ ستون انتخاب‌شده در PriceSource .
spot_price	float None	محاسبه نشود basis یعنی None ؛ basis مثبت و صریح برای spot .
curve	DataFrame iterable[Mapping]	خروجی get_futures_curve() یا جدول هم‌قرارداد با ستون‌های لازم.
annual_rate	float	نرخ مالی سالانهٔ effective به صورت اعشاری؛ مثلاً 0.30 .

نام	type/default	معنا و constraint
storage_rate	float=0.0	هزینه انبارداری سالانه effective به صورت اعشاری.
convenience_yield	float=0.0	منفعت نگهداری کالای فیزیکی به صورت نرخ سالانه اعشاری.

پارامترهای کم تکرار نیز بخشی از قراردادند:

دامنه	پارامترها و معنا
قیمت history	متناظر volume تعدیل (bool=False) adjust_volume : OHLC تعدیل (bool=True) auto_adjust
فهرست بازار	bourse , farabourse , payeh (bool=True) و haghe_taqadom , sandogh , bonds , options , mortgage , commodity , energy (bool=False) inclusion: output (str='dataframe') format: name (str list='') نام/InsCode نام (str) index_name : category: fund_type (str list None) نام ارز; شناسه سهامدار (bool=False) include_id : شاخص;
fundamentals	مثبت معتبر P/E فقط (bool=True) positive_pe : مرزهای شامل: (float None) pe_max , pe_min
live/option	category کد بازار اختیار; (int=0) exchange : point opt-in: مسیر (str='none') fallback (str='treasury') دسته IFB: (float=10.0) lock_timeout و (float=300.0) stale_lock_seconds : قفل فایل مثبت
fixed math	bump_size (float=0.0001) شوک DV01: rate_compounding (str='effective') , rate_frequency (int=1); enforce_monotonic_discount (bool=True) guard curve.
storage	event (MarketEvent) رخداد ورودی و frame (DataFrame) batch archive.
resolver	selector (Any) الزام فعالیت (bool=True) require_active : انتخاب ورودی;

فیلدهای باقی مانده `MarketEvent` همگی `constructor contract` هستند: (`sequence` (`int`) شماره افزایشی، `changed_order_levels` (`tuple[tuple[str,int],...]`) delta، و `changed_inscodes` (`tuple[str,...]`) `notification`، `market_state_changed` (`bool`)، `notification_tokens` (`tuple[str,str,str]`) `cursor` `state_changes` (`DataFrame|None`)، `cursor_before` / `cursor_after` (`int`)، `retry_count` (`int`)، `retry_error` (`persistence_status` / `persistence_error` (`str|None`) و (`str|None`)، `notification_errors` (`tuple[str,...]`) هستند. فیلد `is_active` (`bool|None`) در `InstrumentRef` سه حالت فعال/غیرفعال/نامعلوم دارد.

نوع ها، ثابت ها، تنظیمات و exception های عمومی

```
InstrumentRef(ins_code: 'str', symbol: 'str | None' = None, name: 'str | None' = None, asset_type: 'str' = 'unknown', is_active: 'bool | None' = None, provenance: 'str' = '', selector: 'str | None' = None) -> None
```

نماد/نام، نوع دارایی، وضعیت فعال، منبع اثبات و `canonical` هویت است؛ فیلدها به ترتیب شناسه `dataclass immutable` ورودی هستند. ساخت مستقیم فقط برای داده از قبل اعتبارسنجی شده مناسب است؛ مسیر عادی `selector` گم شده/اضافی است `field` برای `TypeError` استاندارد `constructor` است. خطای `resolve_instrument()`

```
MarketEvent(kind: 'str', sequence: 'int', fetched_at: 'pd.Timestamp', trade_date:
'Optional[dt.date]', snapshot: 'dict[str, Any]', changed_incodes: 'tuple[str, ...]' = (),
changed_order_levels: 'tuple[tuple[str, int], ...]' = (), market_state_changed: 'bool' = False,
notification_tokens: 'tuple[str, str, str]' = ('', '', ''), messages: 'Optional[pd.DataFrame]' = None,
state_changes: 'Optional[pd.DataFrame]' = None, cursor_before: 'int' = 0, cursor_after: 'int' = 0,
retry_count: 'int' = 0, retry_error: 'Optional[str]' = None, notification_errors: 'tuple[str, ...]' =
(), persistence_status: 'Optional[str]' = None, persistence_error: 'Optional[str]' = None) -> None
```

هستند؛ mutable کپی دفاعی ولی `copy_snapshot=True` با `snapshot/messages/state_changes` است. `event watcher` را نگه می‌دارند. مثال ساخت دستی لازم نیست؛ نمونه `persistence provenance` و رشته‌های خطا `delta/cursor` ها `tuple` ها. بالاتر است `watch_market()` موضوعی

```
YieldCurve(settlement_date: datetime.date, maturities: tuple, times: tuple, discount_factors: tuple,
continuous_zero_rates: tuple, day_count: str = 'ACT/365F', interpolation: str = 'log_discount',
extrapolate: bool = False, node_metadata: tuple = <factory>, diagnostics: dict = <factory>) -> None
```

بسازید. `build_yield_curve()` است؛ آن را با `metadata/diagnostics` محاسباتی با آرایه‌های هم‌طول و `curve immutable` می‌دهند `ValueError`، `extrapolate=False` خارج از محدوده با `interpolation` متدهای

ثابت export	type/value	معنا
MARKET_HISTORY_SCHEMA_VERSION	int = 2	نسخهٔ SQLite market history .schema
MARKET_HISTORY_APPLICATION_ID	int = 1096045381	برای رد فایل نامرتب SQLite application id
IRAN_TREASURY_FACE_VALUE	float = 1_000_000.0	ارزش اسمی پیش‌فرض اخزا، نه override اجباری همهٔ اوراق.
OPTION_SNAPSHOT_SCHEMA_VERSION	int = 1	نسخهٔ سند snapshot JSON اختیاری؛ reader mismatch را رد می‌کند.

`ssl_verify: bool=True`، `timeout: int|float=10`، `max_retries: int=3`، `retry_backoff_factor: float=0.3`، `rate_limit_delay: float=0.3`، `market_snapshot_freshness_seconds: float=120.0`، `market_clock_skew_tolerance_seconds: float=5.0`، `client_volume_consistency_tolerance: float=0.05`، `industry_membership_cache_ttl: float=3600.0`، `order_book_max_requests: int=250`، `trade_max_requests: int=250` و `order_book_discovery_lookback_days: int=10` هستند. `headers: dict`، `mapping` و پایه `url_*: str` های روز/ارز/صندوق/بازار `provider` قرارداد تنظیم `source-boundary` می‌تواند URL تغییر را نقض کند و برای مصرف عادی توصیه نمی‌شود.

دارند `AlgotikTSEError` مشترک `base` و `(*args)` ارث‌بردهٔ `constructor` های عمومی همگی `exception` برای `DataParsingError`، `HTTP/provider` برای `ConnectionError`، برای چند هویت هم‌رتبه `AmbiguousSymbolError`، برای نبود هویت و `StockNotFoundError`، برای ورودی نامعتبر `InvalidParameterError`، ناامن `schema/identity` در فصل تنظیمات است `catch` برای خروج از مرز منبع. نمونهٔ `UnsupportedDataSourceError`

قرارداد	constructor
base exception: <code>args: tuple[Any, ...]</code> را مانند context/پیام <code>Exception</code> نگه می‌دارد؛ <code>return</code> خودش ندارد.	<code>AlgotikTSEError</code>
چند هویت <code>exact</code> هم‌رتبه؛ راه‌حل ارائه <code>ins_code</code> است.	<code>AmbiguousSymbolError</code>
<code>HTTP/status/redirect/provider failure</code> با <code>cause</code> اصلی <code>(raise ... from exc)</code> .	<code>ConnectionError</code>
کود معمولاً مناسب نیست <code>retry</code> هویت/فایل ناسازگار؛ <code>payload/schema/</code> .	<code>DataParsingError</code>
<code>I/O</code> های جدید پیش از <code>API</code> ورودی؛ در <code>constraint</code> .	<code>InvalidParameterError</code>
انجام نمی‌شود <code>fuzzy-first-hit</code> پیدا نشده؛ <code>selector exact</code> .	<code>StockNotFoundError</code>
است <code>fail-before-I/O</code> نمونه <code>get_introduction</code> ؛ <code>boundary</code> نیازمند منبع خارج <code>API</code> .	<code>UnsupportedDataSourceError</code>
نامعتبر <code>field/shape</code> و خطای <code>object curve</code> بالا؛ خروجی <code>constructor dataclass</code> <code>build_yield_curve</code> ؛ <code>TypeError/ValueError</code> ؛ مثال	<code>YieldCurve</code>

قرارداد API: هویت، قیمت، معاملات و API‌های قدیمی

```

normalize_instrument_text(value: 'Any') -> 'str'
resolve_instrument(selector=None, *, ins_code=None, asset_type='auto', snapshot=None,
require_active=True) -> 'InstrumentRef'
validate_ins_code(value: 'Any') -> 'str'
get_history(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True,
output_type='standard', date_format='jalali', progress=True, save_to_file=False, dropna=True,
adjust_volume=False, return_type=None, ascending=True, save_path=None, include_today=False, *,
ins_code=None, asset_type='auto', **kwargs)
get_client_type(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
get_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
get_trades(symbol=None, *, ins_code=None, start=None, end=None, include_canceled=False,
max_requests=None, raw=False, progress=True)
get_live_trades(symbol=None, *, ins_code=None, include_canceled=False, max_requests=None, raw=False,
progress=True)
get_detail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_info(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_stats(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_symbols(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
get_shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
get_shareholder_history(symbol='', start=None, end=None, frequency='daily', max_requests=60,
include_id=False, progress=True, *, ins_code=None, asset_type='auto')
get_currency(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
get_tgju_history(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
list_tgju_assets(category=None, include_aliases=False)
get_market_snapshot(*args, **kwargs)
get_market_client_type(*args, **kwargs)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>normalize_instrument_text</code>	به رشته تهی و <code>value: Any None</code> ؛ متن با یکسان‌سازی ی/ک، فاصله و به کلید مقایسه تبدیل می‌شود ZWNJ	<code>mutate</code> ؛ ورودی <code>canonically</code> <code>str</code> نمی‌کند	خطای ویژه ندارد؛ <code>normalize_instrument_text("گل") == "گل"</code>
<code>validate_ins_code</code>	مثبت یا <code>integer</code> <code>value: str int</code> ؛ فاصله ابتدا/انتها ASCII رشته ۰۰۱۲۰۰۱ (می‌شود <code>float</code> ، <code>bool</code> ، رقم فارسی /، <code>sign</code> ، <code>space</code> و طول عربی، بیش از ۲۰ رد می‌شوند	معتبر دقیقاً به ASCII <code>str</code> ؛ رشته تبدیل می‌شود	<code>InvalidParameterError</code> ؛ resolver مثال بخش

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>resolve_instrument</code>	<code>snapshot:</code> منبع <code>dict DataFrame None</code> حتی اگر تهی؛ <code>authoritative</code> <code>require_active: bool=True</code> ابزار غیرفعال را حذف می‌کند. اولویت آمده است <code>resolver</code> در فصل <code>exact</code> .	<code>InstrumentRef</code> ; مسیر اثبات <code>provenance</code> (<code>explicit_ins_code</code> , <code>snapshot/search/index registry</code> و مشابه) را ثبت می‌کند.	<code>InvalidParameterError</code> , <code>StockNotFoundError</code> , <code>AmbiguousSymbolError</code> , <code>DataParsingError</code> , <code>ConnectionError</code> ; مثال: <code>exact/ambiguity</code> بالاتر.
<code>get_history</code>	<code>symbol/ins_code/asset_type</code> هویت؛ <code>e</code> <code>start/end/limit/ascending</code> <code>schema TSE:</code> بازه و ترتیب؛ <code>OHLC:</code> تعدیل <code>auto_adjust</code> تعدیل حجم؛ <code>adjust_volume</code> <code>output_type∈{standard,full</code> <code>};</code> <code>date_format∈{jalali,gregor</code> <code>;ian,both}</code> <code>return_type∈{simple,log,bo</code> <code>th,list}</code> <code>include_today</code> <code>progress/dropna</code> ردیف زنده؛ نمایش/ترکیب؛ <code>save_to_file/save_path</code> های <code>alias</code> فقط <code>kwargs</code> ؛ <code>CSV</code> ذخیره مستند.	برای سهم و <code>auto_adjust=True</code> , <code>output_type='standard'</code> دقیقاً <code>Open,High,Low,Close,Volume</code> است؛ <code>full</code> ستون‌های <code>Final,No.,Value</code> و تقویم/ <code>Ticker</code> را اضافه می‌کند. با <code>auto_adjust=False</code> , <code>standard</code> ستون <code>Adj Close</code> هم دارد. <code>index/industry schema</code> محدودتر خود را دارند؛ چند نماد <code>MultiIndex</code> . <code>attrs</code> پیش‌فرض تهی و با شامل <code>include_today=True</code> <code>include_today_appended/i</code> <code>nclude_today_warning</code> است.	ورودی‌های ناسازگار <code>legacy</code> معمولاً <code>ValueError / None</code> و <code>ConnectionError</code> ; <code>provider</code> مثال فصل قیمت.
<code>get_client_type</code>	<code>symbol/ins_code/asset_type</code> هویت؛ <code>e</code> <code>start/end/limit/ascending</code> <code>schema provider:</code> بازه؛ <code>output_type∈{standard,full</code> <code>};</code> <code>date_format∈{jalali,gregor</code> <code>;ian,both}</code> <code>include_today</code> <code>progress/dropna</code> ردیف امروز؛ نمایش/چندنمادی؛ <code>save_to_file/save_path</code> های قدیمی <code>alias</code> ؛ <code>kwargs</code> ؛ <code>CSV</code> .	روزانه <code>DataFrame</code> <code>Buy_I/N_Count</code> , <code>Buy_I/N_Volume</code> , <code>Sell_I/N_Count</code> , <code>Sell_I/N_Volume</code> , قدرت/ سرانه‌های مشتق؛ چند نماد <code>MultiIndex</code> . <code>opt-in</code> ردیف امروز.	خطای <code>selector/provider</code> یا <code>legacy</code> <code>None</code> ; مثال فصل حقیقی/حقوقی.
<code>get_capital_increase</code>	<code>stock = .</code> قدیمی <code>alias</code> و <code>selector</code> 	<code>index</code> با <code>DataFrame None</code> و <code>date</code> <code>old_shares_amount,new_shares_amount</code> .	در <code>provider/nomad</code> گم‌شده/ <code>index</code> <code>None</code> ؛ قرارداد قدیمی پیام و <code>att.get_capital_increase</code> . ("فملی") <code>e</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>get_intraday</code>	یکی از canonical <code>interval</code> <code>tick, 1min, 5min, 15min, 30min</code> های <code>alias</code> و <code>1h, 4h, 12h</code> مانند <code>_INTERVAL_MAP</code> <code>1m, 60min, 4hour, 240m, 12hour</code> بدون تاریخ snapshot <code>start</code> معاملات امروز، با تاریخ.	شامل <code>tick</code> ؛ <code>DataFrame None</code> شامل <code>candle</code> زمان/قیمت/حجم و <code>Open, High, Low, Close, Volume, TradeCount</code> با <code>DatetimeIndex</code> .	نامعتبر یا تاریخی که <code>interval</code> <code>validator</code> نامعتبر تشخیص <code>None</code> دهد پیام چاپ می‌کند و نماد نیز در مسیر/ <code>provider</code> می‌دهد؛ <code>legacy</code> ؛ <code>None</code> <code>ValueError</code> نیست؛ API خطای عمدی قرارداد این مثال <code>att.get_intraday("فملی", interval="5min")</code> .
<code>get_trades</code>	بدون تاریخ امروز تهران؛ <code>Thu/Fri</code> قبل از <code>budget</code> حذف؛ <code>include_canceled</code> ؛ <code>max_requests</code> فقط Trade endpoint؛ <code>raw</code> ستون‌های <code>provider</code> را اضافه می‌کند.	standard: <code>DataFrame[InsCode, Symbol, GregorianDate, JalaliDate, TradeNo, Time, Timestamp, Price, Volume, Value, Canceled, Source]</code> ؛ attrs شامل <code>request/provenance/failures</code> . <code>raw</code> نیز دارد <code>audit</code> ستون‌های	<code>InvalidParameterError</code> ، <code>resolver errors</code> ، <code>ConnectionError</code> ، مثال؛ <code>DataParsingError</code> ؛ فصل معاملات.
<code>get_live_trades</code>	هویت؛ <code>symbol/ins_code</code> رکورد ابطالی؛ <code>include_canceled</code> سقف درخواست؛ <code>max_requests</code> <code>raw</code> audit schema: <code>progress</code> نمایش. تاریخ به‌صورت خودکار امروز تهران است.	دقیق <code>attrs</code> و <code>schema</code> معاملات امروز با: <code>get_trades</code> زمان، قیمت، حجم، ارزش، وضعیت <code>provenance</code> ابطال و	<code>InvalidParameterError</code> ، <code>resolver errors</code> ، <code>ConnectionError</code> ، <code>DataParsingError</code> ؛ <code>att.get_live_trades(ins_code="...")</code> .
<code>get_detail</code>	قدیمی HTML API دقیق؛ <code>selector</code> .	با <code>index</code> <code>DataFrame None</code> به‌همراه، <code>value</code> و ستون <code>key</code> <code>row id</code> .	در رفتار <code>index/no-match/HTTP</code> مثال؛ <code>legacy</code> <code>None</code> <code>att.get_detail("فملی")</code> .
<code>get_info</code>	دقیق <code>selector</code> .	از <code>key/value</code> <code>DataFrame None</code> کامل <code>instrumentInfo</code> <code>flatten</code> .	یا <code>legacy</code> <code>None</code> ؛ مثال فصل <code>connection/parsing</code> اطلاعات نماد.
<code>get_stats</code>	دقیق <code>selector</code> .	<code>key/value</code> <code>DataFrame None</code> عددی <code>value</code> آمار با کلید فارسی و	یا <code>legacy</code> <code>None</code> <code>connection/parsing</code> ؛ <code>att.get_stats("فملی")</code> .
<code>get_introduction</code>	هیچ پارامتر BC فقط برای <code>signature</code> نمی‌شود I/O باعث	هرگز خروجی موفق ندارد.	همیشه <code>UnsupportedDataSourceError</code> پیش از I/O؛ جایگزین <code>market-data</code> ؛ <code>get_info/get_detail</code> .
<code>get_symbols</code>	<code>market/asset</code> ، های <code>boolean</code> <code>payeh_color</code> ؛ <code>output</code> ؛ <code>str list None</code> ؛ های <code>alias</code> ؛ <code>str='dataframe'</code> ؛ <code>kwargs</code> انگلیسی در	فهرست ابزارها یا <code>DataFrame</code> قدیمی انتخابی؛ ستون‌های <code>format</code> ؛ خروجی تهی <code>type</code> هویت/نام/بازار و <code>schema</code> پایدار دارد.	فیلتر <code>output</code> نامعتبر ؛ <code>legacy</code> <code>None</code> یا <code>ValueError</code> مثال فصل فهرست ابزارها.

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>get_shareholders</code>	<code>symbol/ins_code/asset_type</code> آخرین فهرست <code>date=None</code> هویت؛ و تاریخ مشخص، روز معامله است؛ شناسه رکورد <code>include_id=False</code> های <code>alias</code> را حذف می‌کند؛ snapshot حفظ شده‌اند <code>stock/shh_id</code>	<code>DataFrame None[share_holder_name,number_of_shares,percentage_of_shares,change_state,change_amount,date,trade_date,effective_date,trade_date_jalali,effective_date_jalali,identity_quality,change_quality,source]</code> و با <code>opt-in share_holder_id . date</code> و <code>effective_date</code> همان معمولاً روز معاملاتی بعد از است؛ تغییر با نام <code>trade_date</code> محاسبه و برای ورودی مبهم <code>exact</code> هشدار انتقال <code>attrs</code> می‌شود <code>NA</code> غیرمعاملاتی، تأخیر جلسه، نبود شناسه پایدار و غیررسمی بودن ثبت را دارد	تاریخ در قرارداد/ <code>provider/selector</code> <code>None</code> ؛ مثال فصل پیام و <code>legacy</code> اطلاعات نماد
<code>get_shareholder_history</code>	اجباری و بر مبنای تاریخ <code>start/end</code> مؤثر؛ <code>frequency={daily,weekly,monthly}</code> <code>max_requests=60</code> <code>include_id</code> ؛ سقف سخت <code>!۱۰۰۰</code> ؛ با <code>exact</code> هویت؛ <code>progress</code> <code>symbol/ins_code/asset_type</code> پاسخ را snapshot هر دو <code>daily</code> آخرین جلسه <code>weekly</code> مصرف می‌کند؛ آخرین جلسه <code>monthly</code> هفته شنبه محور و ماه شمسی است	همان schema توسعه یافته <code>get_shareholders</code> در <code>long-form</code> برای چند تاریخ. <code>attrs</code> شامل <code>date_basis='effective_date'</code> , <code>coverage_start/end</code> , <code>requested_snapshot_count</code> , <code>returned_snapshot_count</code> , <code>missing_effective_dates</code> , <code>shareholder_requests_used</code> , <code>pair_request_optimization</code> , <code>change_period</code> و <code>n</code> تمام هشدارهای رسمی منبع است.	بازه <code>enum/bool/b</code> و <code>num</code> ؛ <code>InvalidParameterError</code> ؛ هویت، شبکه یا schema خطاهای <code>typed</code> ؛ مثال <code>att.get_shareholder_history('فولاد', start='1404-01-01', end='1404-12-29', max_requests=150)</code>
<code>get_currency</code>	<code>canonical</code> نام <code>str list</code> ؛ برای هر دارایی کاتالوگ؛ slug فارسی یا <code>start/end/limit/ascending</code> <code>schema: output_type</code> ؛ بازه؛ <code>date_format={jalali,gregorian,both}</code> بازه؛ <code>return_type</code> ؛ نمایش <code>progress/dropna</code> ؛ <code>save_to_file/save_path</code> های قدیمی <code>alias</code> ؛ <code>kwargs</code> ؛ CSV	تک دارایی <code>DataFrame[Open,High,Low,Close]</code> با <code>attrs</code> منبع/ نام/ <code>slug/category/unit</code> ؛ چند دارایی ستون <code>MultilIndex</code> .	نام <code>InvalidParameterError</code> ، <code>HTTP ConnectionError</code> ، <code>schema DataParsingError</code> ؛ <code>att.get_currency("gold-18k", limit=30)</code>
<code>get_tgju_history</code>	قرارداد و <code>default</code> های دقیق <code>get_currency</code> ؛ نام پیشنهادی چون دامنه فقط ارز نیست.	همان <code>frame/MultilIndex</code> و <code>attrs</code> ؛ <code>wrapper</code> معنایی است.	همان؛ <code>att.get_tgju_history("طلا ۲۴ عیار", limit=30)</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>list_tgju_assets</code>	<code>category=None</code> یا <code>str/sequence</code> از هفت دسته؛ <code>include_aliases=False</code> tuple ها را اضافه می‌کند. <code>alias</code>	<code>DataFrame[Name, PersianName, Slug, Category, Unit]</code> و <code>Aliases</code> attrs ؛ <code>opt-in</code> در حالت و عرض ردیف تأیید شده endpoint.	نامعتبر <code>category/bool</code> : <code>InvalidParameterError</code> بدون شبکه؛ <code>att.list_tgju_assets("gold")</code> .
<code>get_market_snapshot</code>	به تابع BC برای <code>*args/**kwargs</code> <code>market_watch</code> صفر آرگومان می‌شود؛ در عمل آرگومان غیرتهی <code>forward</code> می‌دهد <code>TypeError</code> .	با کلیدهای دقیق <code>dict</code> <code>stocks, order_book, market_time, index_value, migration, trade_date, market_state, exchange_time, fetched_at, snapshot_age_seconds, is_today_trade_date, is_history_eligible, is_realtime_fresh, is_previous_trade_date, is_stale, is_partial</code> .	<code>ConnectionError</code> , <code>DataParsingError</code> ; مثال live.
<code>get_market_client_type</code>	<code>*args/**kwargs</code> wrapper <code>market_client_type</code> .	<code>DataFrame[InsCode, Buy_I_Count, ..., Net_I_Volume, Net_N_Volume]</code> .	<code>ConnectionError</code> , <code>DataParsingError</code> ; مثال live.

قرارداد live: API، سفارش، watcher و تحلیل بازار

```

get_order_book(symbol=None, *, selector_strict=False)
get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback='none')
get_order_book_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_orderbook_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_queue(symbol=None, side='both', strict=True, *, selector_strict=False)
get_queue_history(symbol='', start=None, end=None, limit=0, date_format='jalali', progress=True,
save_to_file=False, dropna=True, ascending=True, save_path=None, include_today=False,
complete_only=False, side='both', strict=True, *, max_requests=None, **kwargs)
MarketWatcher(symbol=None, interval=1.0, max_updates=None, include_initial=True, emit_heartbeats=True,
notifications=('messages', 'state'), notification_top=50, stop_event=None, error_policy='retry',
max_backoff=30.0, jitter=0.1, callback_error_policy='raise', max_consecutive_retries=5,
max_retries=None, max_seen_notifications=10000, copy_snapshot=True, request_timeout=None, *,
record_to=None, record_heartbeats=False, checkpoint_interval=100, record_max_records=10000,
record_retention_seconds=None, storage_error_policy='raise', _request=safe_get, _clock=None,
_wait=None, _random=None)
watch_market(*args, **kwargs)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_market_overview(flow=0, *, archive_to=None, _recorded_at=None, _request=safe_get)
get_market_breadth(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None)
get_sector_flow(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None, _client_type=None)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_order_book	scalar/list/ None ; فقط selector_strict انتخاب را سخت می‌کند resolution.	با long DataFrame InsCode, Symbol, Name, Level, BidOrderCount, BidVolume, BidPrice, AskPrice, AskVolume, AskOrderCount و metadata trade_date, market_state, exchange_time, fetched_at, is_* response پنج سطح از همان . is_*	InvalidParameterError , StockNotFoundError در strict. connection/parsing مثال: فصل سفارش.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_live_market</code>	های <code>selector</code> <code>strict=False</code> گم‌شده را در <code>attrs['missing_selectors']</code> آن‌ها را خطا می‌کند <code>strict</code> ثبت می‌کند؛ <code>]</code>	با <code>DataFrame</code> <code>STOCK_COLUMNS</code> ، <code>client</code> <code>columns</code> ، <code>wide</code> سطح‌های <code>BidPrice1..5/AskPrice1..5</code> از جمله <code>metrics</code> و <code>5</code> <code>EstimatedNetIndividualFlow</code> ؛ <code>freshness</code> ستون <code>row-wise</code> ؛ <code>attrs</code> دقیق است: <code>field_validity, source_schema_presence, migration, missing_selectors</code> . <code>Last</code> پایانی است <code>Close</code> آخرین معامله و	<code>InvalidParameterError</code> ، <code>StockNotFoundError</code> در <code>strict</code> ، <code>ConnectionError/DataParsingError</code> ؛ <code>quickstart</code> و مثال <code>attrs</code> بالا.
<code>get_live_symbol</code>	<code>fallback: 'none' 'point'</code> ؛ <code>point</code> فقط در غیاب <code>MarketWatch</code> . <code>fallback='none'</code> برای <code>no-match</code> <code>frame</code> نمی‌دهد.	<code>exception</code> ؛ دقیقاً یک‌ردیفی یا <code>frame</code> <code>provenance</code> <code>SnapshotSource='market_watch' 'closing_price_info'</code> و ستون‌های <code>_fallback'</code> <code>PresentInMarketWatch, MarketStateTitle, has_trade_today, PriceActionable, fallbackReason, IdentityVerified</code> .	<code>no-match</code> با <code>fallback none</code> : <code>StockNotFoundError</code> ؛ همچنین <code>InvalidParameterError</code> ، <code>AmbiguousSymbolError</code> ، <code>DataParsingError</code> ؛ فصل <code>live</code> .
<code>get_order_book_history</code>	<code>raw</code> ؛ <code>output_type='standard'</code> <code>long</code> و <code>'wide'</code> ؛ <code>complete_only</code> ؛ <code>budget</code> همهٔ <code>calls</code> این <code>workflow</code> .	<code>DataFrame</code> ؛ <code>long/wide/raw</code> ؛ <code>identity/time/5</code> سطح و <code>is_partial, is_complete, market_partial_status, is_reconstructed, is_stale, record_type, source</code> ؛ <code>attrs</code> <code>schema, failed_requests, request_count, max_requests</code> .	<code>InvalidParameterError</code> ، <code>DataParsingError</code> ، <code>resolver/connection: failure</code> ؛ مثال فصل <code>attrs</code> + <code>warning</code> جزئی تاریخچه سفارش.
<code>get_orderbook_history</code>	کاملاً یکسان با <code>signature</code> هویتی و <code>alias</code> <code>get_order_book_history</code> .	دقیقاً همان <code>.object/return/schema/attrs</code>	همان خطاها و همان مثال؛ <code>att.get_orderbook_history</code> <code>is</code> <code>att.get_order_book_history</code> <code>.ry</code>
<code>get_queue</code>	فقط <code>side</code> ؛ <code>strict=True</code> های قطعی را نگه می‌دارد <code>queue</code> .	<code>DataFrame[InsCode, Symbol, Name, ..., Side, QueuePrice, QueueVolume, QueueOrders, QueueValue, PriceLimit, is_queue, book_state, ...]</code> .	نامعتبر <code>side</code> <code>InvalidParameterError</code> ؛ مثال <code>selector/provider errors</code> صف.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_queue_history</code>	<code>symbol/start/end/limit/asc</code> تاریخ؛ <code>date_format</code> بازه؛ <code>include_today</code> live؛ <code>complete_only</code> snapshot؛ کامل؛ <code>side∈{buy,sell,both}</code> <code>strict</code> فقط صف قطعی؛ <code>max_requests</code> بودجه؛ <code>progress/dropna</code> نمایش/تهی؛ <code>save_to_file/save_path</code> از CSV؛ <code>kwargs</code> alias. threshold و است as-of snapshot همان تاریخ و	<code>QUEUE_COLUMNS</code> <code>is_queue</code> nullable. crossed book false؛ attrs budget/failures.	<code>InvalidParameterError</code> , resolver/connection/parsing؛ مثال فصل صف
<code>MarketWatcher</code>	فقط <code>notifications</code> ها؛ policy enum؛ glossary دقیقاً در <code>interval,max_backoff>=0</code> , نامنفی <code>retry limit</code> , <code>0<=jitter<=1</code> , <code>request_timeout/record_reten</code> <code>tion_seconds>0</code> ; دارای <code>stop_event</code> <code>is_set/wait</code> ; <code>record_to</code> opt-in.	<code>MarketEvent</code> ; سنکرون iterator های kind <code>initial/delta/heartbeat/</code> <code>resync</code> ; خود constructor I/O نمی‌کند.	نامعتبر policy/range <code>InvalidParameterError</code> iteration: runtime <code>ConnectionError/DataPar</code> <code>singError</code> : مثال policy طبق watcher.
<code>watch_market</code>	تمام <code>*args/**kwargs</code> بدون تغییر به <code>MarketWatcher</code> می‌روند.	نه ، <code>MarketWatcher</code> DataFrame.	همان constructor/runtime؛ مثال <code>for event in</code> <code>att.watch_market(max_upd</code> <code>ates=3): ...</code>
<code>get_market_messages</code>	local؛ فیلتر <code>since_id</code> opt-in. نوشتن اتمیک <code>archive_to</code>	<code>DataFrame[message_id,date,time,timestamp,title,description,flow]</code> ; attrs provenance/archive.	<code>InvalidParameterError</code> , <code>ConnectionError</code> , <code>DataParsingError</code> , storage error؛ مثال فصل پیام
<code>get_instrument_state_changes</code>	<code>since_id</code> bounded؛ تعداد <code>top</code> فقط رکوردهای بعد از شناسه؛ opt-in؛ SQLite مسیر <code>archive_to</code> فقط <code>_recorded_at/_request</code> seam تست.	<code>DataFrame[event_id,date,time,timestamp,InsCode,Symbol,Name,state_code,state,real_time,under_supervision,state_title]</code> .	<code>InvalidParameterError</code> , <code>ConnectionError</code> , <code>DataParsingError</code> یا خطای storage؛ مثال فصل وضعیت
<code>get_market_overview</code>	تهی. صفر ردیف است <code>payload</code> ; <code>flow</code>	با <code>DataFrame</code> overview <code>payload/fast-view</code> و فیلدهای <code>flow</code> attrs source/time/archive.	parameter/connection/parsing/storage؛ مثال overview.
<code>get_market_breadth</code>	فیلترهای universe؛ denominator ابرار انتخاب شده.	یک ردیف <code>DataFrame</code> با counts/percentages. A/D، و volume/value. limit counts .freshness. attrs analytics/source	<code>InvalidParameterError</code> , <code>DataParsingError</code> ; مثال breadth.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_sector_flow</code>	فیلتر: <code>symbol/flow/sector</code> universe <code>traded_only</code> معامله‌شده؛ بازار پایه: <code>include_base_market</code> کد نوع ابزار: <code>instrument_types</code> فقط <code>_snapshot/_client_type</code> فقط برای ردیف client feed. تست reconcile به کار می‌رود	یک ردیف در هر <code>SectorCode</code> با <code>breadth +</code> <code>client_coverage, net_individual_volume, estimated_net_individual_value, value_available, value_method</code> و <code>freshness</code>	parameter/connection/parsing: مثال <code>sector</code>.

قرارداد API: فعالیت سهامداران عمده و timeline نمود

```

get_major_shareholder_snapshots(date=None, days=5, symbol=None, *, ins_code=None, holder=None,
enrich_identity=True, progress=True)
get_major_shareholder_changes(date=None, days=5, symbol=None, *, ins_code=None, holder=None,
direction='both', enrich_identity=True, progress=True)
get_active_shareholders(symbol=None, days=5, *, ins_code=None, holder=None, enrich_identity=True,
progress=True)
rank_shareholder_accumulation(days=5, symbol=None, *, ins_code=None, holder=None, direction='both',
metric='percent', top=20, enrich_identity=True, progress=True)
get_shareholder_network(date=None, symbol=None, *, ins_code=None, holder=None, min_holdings=0,
enrich_identity=True, progress=True)
get_ownership_concentration(symbol='', date=None, *, ins_code=None, asset_type='auto', top_n=5,
progress=True)
get_symbol_events(symbol=None, start=None, end=None, kinds=('messages', 'capital_changes',
'price_adjustments'), limit=100, ascending=False, progress=True, *, ins_code=None, max_requests=3)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_major_shareholder_snapshots</code>	یا یک تاریخ منتشرشده؛ <code>date=None</code> نادیده <code>date</code> و در حضور <code>days=1..5</code> <code>symbol/ins_code</code> گرفته می‌شود؛ exact: فیلتر نام <code>holder</code> فیلتر ابزار؛ اتصال <code>enrich_identity=True</code> <code>progress</code> دقیق هویت؛	<code>DataFrame[HolderRecord, HolderName, InsCode, Symbol, InstrumentName, GregorianDate, JalaliDate, Holdings, Source, FetchedAt]</code> . attrs شامل <code>available_dates</code> , <code>coverage_start/end</code> , <code>rolling_window=True</code> , <code>no_backfill=True</code> , <code>is_complete_major_shareholder_list=False</code> , <code>stable_shareholder_id_available=False</code> , <code>unmatched_identity_count</code> .	خارج پنجره <code>days/bool/date</code> : <code>InvalidParameterError</code> قرارداد/عدد/شناسه خراب : <code>DataParsingError</code> شبکه مثال فصل . <code>ConnectionError</code> فعالیت سهامداران

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_symbol_events</code>	هویت <code>start/end</code> inclusive؛ exact؛ <code>kinds</code> یک یا چند مورد <code>messages/capital_changes/price_adjustments</code>؛ <code>limit=100</code> و صفر یعنی همه؛ <code>ascending=False</code>؛ <code>max_requests=3</code> باید پیش از I/O تعداد source‌های انتخابی را پوشش دهد.	<code>DataFrame[EventID, EventType, InsCode, Symbol, Name, GregorianDate, JalaliDate, Timestamp, Title, Description, OldValue, NewValue, ChangeValue, Source, FetchedAt]</code> . شامل <code>sources</code> , <code>request_count</code> , <code>codal_included=False</code> , <code>excluded_disclosure_kinds</code> , <code>price_adjustment_is_not_confirmed_dps=True</code> , <code>no_fuzzy_join=True</code> .	هویت/enum/تاریخ/budget یا <code>InvalidParameterError</code> resolver error؛ شبکه/schema typed. مثال فصل timeline نماد.

قرارداد API: تقویم، نمای کلان، مستر ابزار و پیش‌گشایش

```

get_trading_calendar(start=None, end=None, market='all', include_closed=False, limit=90)
get_market_activity(start=None, end=None, market='all', max_requests=30, progress=True)
get_market_value_history(start=None, end=None, market='all', limit=90)
get_index_impact(date=None, market='all', top=10, direction='both')
get_market_trades(date=None, market='all', progress=True)
get_instrument_master(active=None, market=None, asset_type=None, progress=True)
get_instrument_changes(previous, current=None, progress=True)
get_theoretical_opening_price(*, username=None, password=None, market='all', timeout=None, progress=True)
get_preopen_imbalance(top_data=None, *, username=None, password=None, market='all', timeout=None, progress=True)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_trading_calendar</code>	تاریخ شمسی/میلادی؛ <code>start/end</code> یا بورس/فرا بورس؛ <code>market='all'</code> فقط <code>include_closed=False</code> در نبود بازه، <code>limit=90</code>؛ مثبت و حداکثر ۵۰۰۰ session.	<code>DataFrame[Market, Flow, GregorianDate, JalaliDate, IsTradingDay, ReferenceIndex, ReferenceInsCode, Source]</code> شامل <code>attrs</code>؛ <code>calendar_semantics='published_reference_index_sessions'</code> .	ترتیب/نوع تاریخ، بازار، <code>limit</code> یا <code>bool</code> نامعتبر یا <code>InvalidParameterError</code>؛ payload/شبکه typed؛ مثال فصل تقویم.
<code>get_market_activity</code>	session؛ بازه <code>start/end</code>؛ <code>market</code>؛ <code>max_requests=30</code> . <code>progress</code> بودجه سخت ۳۶۶۰۰۱؛ بدون بازه آخرین جلسه را می‌گیرد.	<code>DataFrame[Market, GregorianDate, JalaliDate, InstrumentCount, TradedInstrumentCount, TradeCount, Volume, Value, AverageTradeValue, Source]</code> شامل <code>attrs</code> و <code>bulk daily</code>؛ مشتق از <code>request count</code>.	بازه/بودجه market نامعتبر یا <code>InvalidParameterError</code>؛ provider typed؛ مثال فصل نمای کلان.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_market_value_history</code>	<code>start/end ; market ; limit=90</code> ؛ با start مثبت تا ۵۰۰۰؛ با fetch است ۵۰۰۰ داخلی سقف	<code>DataFrame[Market, Flow, GregorianDate, JalaliDate, MarketValue, Change, ChangePct, Source]</code> ؛ واحد <code>attrs['unit']='rial'</code> .	پارامتر/تاریخ <code>InvalidParameterError</code> و <code>connection/schema typed</code> ؛ مثال فصل نمای کلان.
<code>get_index_impact</code>	آخرین جلسه؛ <code>date=None</code> ؛ مثبت تا ۱۰۰۰؛ <code>market ; top=10</code> ؛ <code>direction∈{both, positive, negative}</code> .	<code>DataFrame[Market, Flow, GregorianDate, JalaliDate, Rank, InsCode, Symbol, Name, Close, Impact, Direction, AbsSharePct, Source]</code> ؛ <code>attrs</code> درصد در denominator.	<code>direction/top/date/market</code> نامعتبر <code>InvalidParameterError</code> ؛ مثال <code>provider typed</code> ؛ <code>att.get_index_impact(top=10)</code> .
<code>get_market_trades</code>	آخرین جلسه؛ <code>date=None</code> ؛ فیلتر بازار با <code>progress</code> ؛ <code>market ;</code> است ISIN پیشوند	یک خلاصه روزانه برای هر ابزار با هویت، OHLC، معامله/حجم/ارزش و تغییر؛ <code>attrs['transaction_level']=False</code> و شمار طبقه‌بندی نامعلوم.	<code>parameter/connection/schema typed</code> ؛ برای معاملات ریز از <code>get_trades</code> استفاده شود؛ مثال <code>bulk</code> فصل.
<code>get_instrument_master</code>	آخرین جلسه؛ <code>date=None</code> ؛ فیلتر بازار با <code>progress</code> ؛ <code>market ;</code> است ISIN پیشوند	<code>DataFrame[InsCode, ISIN, Symbol, Name, EnglishName, CompanyCode, CompanyISIN, Market, MarketGroup, Industry, AssetType, InstrumentStatus, Active, Source]</code> ؛ <code>identity</code> key در <code>attrs</code> .	<code>filter/type</code> نامعتبر <code>InvalidParameterError</code> ؛ <code>HTML/schema/network typed</code> ؛ مثال ذخیره Parquet.
<code>get_instrument_changes</code>	اجباری و <code>previous</code> ؛ <code>current=None</code> ؛ هرکدام <code>DataFrame</code> یا <code>CSV/Parquet/JSON</code> ؛ <code>InsCode</code> کلید اجباری. <code>progress</code> .	long-form <code>DataFrame[ChangeType, InsCode, Field, OldValue, NewValue, Symbol, Source]</code> ؛ <code>added/removed/changed</code> و <code>count</code> در <code>attrs</code> snapshot دو	مسیر/پسوند/ستون نامعتبر <code>InvalidParameterError</code> ؛ <code>current None</code> ممکن است <code>provider error</code> بدهد؛ بدون <code>backfill</code> تاریخی.
<code>get_theoretical_opening_price</code>	keyword-only اشتراک <code>username/password</code> ؛ <code>market ; timeout ;</code> رسمی؛ <code>credentials</code> در package ذخیره نمی‌شوند.	رسمی با قیمت/حجم تئوریک TOP، <code>remainder</code> ، <code>buy/sell</code> ؛ <code>attrs</code> تغییر؛ <code>official=True, subscriber_service=True, credentials_stored=False</code> .	ناموجود <code>credentials</code> <code>InvalidParameterError</code> ؛ <code>SOAP/network</code> یا <code>ConnectionError</code> ؛ مثال <code>DataParsingError</code> ؛ فصل TOP.
<code>get_preopen_imbalance</code>	<code>top_data: DataFrame None</code> ؛ از <code>None</code> در <code>username/password/market/top_data/timeout/progress</code> برای TOP استفاده می‌کند.	همه ستون‌های TOP + <code>ImbalanceVolume, ImbalanceRatio, ImbalanceSide</code> ؛ <code>attrs</code> فرمول دقیق در.	نامعتبر <code>bool/stون/type</code> <code>InvalidParameterError</code> ؛ مثال <code>fetch</code> در حالت TOP خطاهای فصل TOP.

قرارداد API: تاریخچه محلی و fundamentals

```
get_market_fundamentals(symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=False, archive_to=None, max_requests=1, progress=True)
get_market_fundamentals_history(path, start=None, end=None, symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=True, limit=1000, offset=0)
get_price_adjustments(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
get_latest_price_adjustment(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
check_market_history(path)
save_market_snapshot(path, snapshot=None, *, as_of=None, _live_fetch=None)
load_market_snapshots(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_live_market_history(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_market_overview_history(path, start=None, end=None, flow=0, *, source='tsetmc', limit=1000, offset=0)
get_market_snapshot_summary_history(path, start=None, end=None, flow=None, *, limit=1000, offset=0)
get_market_breadth_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
get_sector_flow_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
record_market_event(path, event, *, session_id=None, include_snapshot=True, max_records=10000, retention_seconds=None)
get_market_event_history(path, start=None, end=None, kind=None, *, session_id=None, limit=1000, offset=0)
archive_market_records(path, kind, frame, *, source='tsetmc', recorded_at=None)
get_market_messages_history(path, start=None, end=None, flow=0, since_id=None, *, source='tsetmc', limit=1000, offset=0)
get_instrument_state_changes_history(path, start=None, end=None, symbol=None, since_id=None, *, inscode=None, source='tsetmc', limit=1000, offset=0)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_market_fundamentals	pe_min/pe_max: float None positive_pe=True شامل مرز؛ archive_to: مثبت؛ P/E فقط max_requests=1 bulk.	دقیق attrs ثابت بالا؛ schema request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_lookahead,no_backfill,archive_path .	در bounds/filter/no-match strict: InvalidParameterError /resolver error: stale با frame allow_stale=False exception: نه attr تهی و مثال: provider typed errors fundamentals.
get_market_fundamentals_history	path SQLite؛ start/end/limit/offset فیلتر؛ symbols صفحه و بازه؛ pe_min/pe_max/positive_pe P/E: غربال instrument_types strict no-match: allow_stale snapshot نگه‌داشتن قدیمی.	attrs زنده با schema missing_selectors,strict,source,price_source,eps_source,current_eps_used,no_lookahead,no_backfill,coverage_start,coverage_end ; current_eps_used=False .	file/schema/filter InvalidParameterError یا DataParsingError ; مثال: backtest.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_price_adjustments</code>	شامل date bounds دقیق و selector	<code>DataFrame[InsCode, Symbol, GregorianCalendar, JalaliDate, AdjustedClosingPrice, UnadjustedClosingPrice, AdjustmentAmount, CorporateTypeCode, CorporateActionType, IsConfirmedDPS, IdentityVerified, Source, FetchedAt] ;</code> <code>IsConfirmedDPS=False ,</code> <code>attrs dps_available=False .</code>	resolver/parameter/connection/parsing مثال فصل تعدیل
<code>get_latest_price_adjustment</code>	هویت: <code>symbol/ins_code</code> فیلتر تاریخ: <code>start/end</code> نمایش. پس از اعمال فیلتر <code>progress</code> فقط جدیدترین رخداد انتخاب می‌شود	schema <code>get_price_adjustments ,</code> attrs و typed صفر یا یک ردیف provenance.	<code>InvalidParameterError ,</code> resolver errors, <code>ConnectionError ,</code> مثال: <code>DataParsingError</code> فصل تعدیل
<code>check_market_history</code>	موجود باشد SQLite باید <code>path</code>	دقیقاً <code>dict[str, bool int]</code> با <code>ok=True, SchemaVersion, ApplicationID</code> DataFrame نیست.	فایل گم‌شده مثال: <code>InvalidParameterError</code> غیر SQLite/نسخه سازگار مثال: <code>DataParsingError</code> <code>att.check_market_history(db)</code>
<code>save_market_snapshot</code>	<code>snapshot:</code> <code>DataFrame dict None</code> ; اگر fetch: فقط یک <code>as_of</code> مشاهده override	<code>str , SHA-256 snapshot_id ;</code> هم‌زمان و live/client/order با DB داخل attrs اتمیک؛ transaction بازیابی می‌شود reader	path/type/schema/storage <code>InvalidParameterError/DataParsingError</code> ; network مثال: <code>snapshot=None</code> فقط هنگام SQLite.
<code>load_market_snapshots</code>	فیلتر inclusive زمان و symbol ; pagination SQL	ردیف‌های <code>DataFrame</code> با <code>STOCK_COLUMNS</code> observation و meta <code>SnapshotID, AsOf, Source, SchemaVersion, NoBackfill, CoverageStart, SnapshotAtomic, PersistenceAtomic, SourceAtomic, PriceSourceAsOf, ClientSourceAsOf, NoLookahead .</code>	file/schema/filter typed مثال: SQLite.
<code>get_live_market_history</code>	SQLite: <code>path</code> <code>start/end/symbol/limit/offset/ascending/include_stale</code> معنایی alias. برای فیلتر و صفحه‌بندی reader rich live نه alias identity.	سازگار با frame/meta <code>load_market_snapshots .</code>	<code>InvalidParameterError</code> و file/schema: خطاهای <code>att.get_live_market_history(db, symbol="فملی") .</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_market_overview_history</code>	<code>flow/source</code> ، مستقل archive	payload exact provider قبلی + meta archive <code>ArchiveIdentity, Version, FirstObservedAt, ObservedAt, ProviderTimestamp, AsOf, Source, SchemaVersion, NoBackfill, ...</code>	نامعتبر <code>kind/source/schema</code> ؛ مثال <code>archive_to</code> ، <code>typed</code> .
<code>get_market_snapshot_summary_history</code>	atomic snapshot مشتق از <code>summary</code> ؛ همه <code>flow=None</code>	<code>DataFrame[flow, instrument_count, trade_count, total_volume, total_value, market_cap, trade_date, exchange_time, fetched_at, is_realtime_fresh]</code> + meta.	؛ مثال <code>file/filter/schema errors</code> ، <code>SQLite</code> .
<code>get_market_breadth_history</code>	همان فیلترهای <code>live</code> روی هر snapshot. <code>.persisted</code>	<code>BREADTH_COLUMNS</code> + meta؛ بدون snapshot-by-snapshot، look-ahead.	؛ مثال <code>file/filter/schema errors</code> ، <code>breadth history</code> .
<code>get_sector_flow_history</code>	همان فیلترهای <code>live</code> ، <code>client</code> و <code>prices</code> ؛ همان <code>observation</code> .	<code>SECTOR_FLOW_COLUMNS</code> + meta/no-lookahead.	؛ مثال <code>file/filter/schema errors</code> ، <code>sector history</code> .
<code>record_market_event</code>	<code>event: MarketEvent</code> ؛ <code>include_snapshot</code> ؛ <code>retention/session</code> .	<code>event_id</code> ، SHA-256 <code>str</code> ؛ در notification/snapshot و transaction.	<code>type/kind/storage</code> ؛ مثال <code>InvalidParameterError/DataParsingError</code> ؛ مثال <code>record_to</code> watcher.
<code>get_market_event_history</code>	<code>kind=None initial delta heartbeat resync</code> ؛ <code>session/pagination</code> .	<code>DataFrame[event_id, SessionID, kind, SnapshotMode, sequence, fetched_at, trade_date, changed_incodes, changed_order_levels, market_state_changed, notification_tokens, cursor_before, cursor_after, retry_count, retry_error, notification_errors, snapshot, messages, state_changes]</code> + meta.	ناشناخته <code>kind</code> ؛ مثال <code>InvalidParameterError</code> ؛ <code>schema</code> حتی برای فایل گم‌شده؛ <code>replay</code> ؛ مثال <code>errors</code> .
<code>archive_market_records</code>	یکی از <code>kind</code> ؛ <code>messages, state_changes, overview</code> ؛ <code>frame: DataFrame</code> ؛ <code>recorded_at/source</code> هویت archive.	های تاریخچه <code>version</code> تعداد <code>int</code> ؛ صفر <code>true duplicate</code> نوشته‌شده؛ <code>transaction atomic</code> .	؛ مثال <code>kind/type/storage errors</code> ، <code>archive_to</code> ؛ <code>helper</code> های.
<code>get_market_messages_history</code>	و <code>flow/since_id/source</code> ؛ <code>time/page filters</code> .	schema پیام + archive meta؛ <code>dedup provider ID</code> .	؛ مثال <code>file/schema/filter errors</code> ، <code>archive</code> .
<code>get_instrument_state_changes_history</code>	قدیمی <code>spelling</code> و/یا <code>symbol</code> ؛ <code>inscode</code> ؛ <code>since_id/source</code> .	schema state + archive meta.	؛ مثال <code>selector/schema/متناقض/فایل errors</code> ، <code>archive</code> .

قرارداد API: درآمد ثابت

```

parse_treasury_maturity(symbol)
day_count_fraction(start, end, convention='ACT/365F')
treasury_yield(price, maturity_date, settlement_date=None, face_value=1000000.0, day_count='ACT/365F')
bond_price(annual_yield, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None)
yield_to_maturity(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, tolerance=1e-12, max_iterations=300)
bond_analytics(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, annual_yield=None, bump_size=0.0001)
build_yield_curve(nodes, settlement_date, interpolation='log_discount', extrapolate=False,
duplicate_policy='error', day_count='ACT/365F', rate_compounding='effective', rate_frequency=1,
enforce_monotonic_discount=True)
get_ifb_yield_table(category='treasury')
get_debt_yields(debt_type=None)
get_treasury_yields(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, price_source='auto', day_count='ACT/365F', strict=False, face_value_source=None,
source='tsetmc', maturity_date=None, maturity_map=None, allow_no_trade=False)
get_treasury_yield_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_treasury_yields_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_yield_curve(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, min_nodes=3, price_source='auto', day_count='ACT/365F', interpolation='log_discount',
extrapolate=False, duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
source='tsetmc')
get_yield_curve_history(symbol=None, start=None, end=None, limit=0, face_value=1000000.0,
include_today=False, date_format='jalali', price_source='auto', day_count='ACT/365F', min_nodes=3,
interpolation='log_discount', duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
ascending=True, progress=True, max_requests=250, maturity_map=None)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>parse_treasury_maturity</code>	<code>symbol: Any</code> ; فقط full-match پس از تبدیل رقم YYMMDD اخذ ZWNJ فارسی/عربی و حذف و 00..79→1400..1479 80..99→1380..1399	موفق: dict None maturity_jalali: str, maturity_gregorian: datetime.date, maturity_source='user_confirmed_symbol_jalali_ymmd' mdd'	، None تاریخ نامعتبر/non-match، deterministic ؛ مثال exception نه بالاتر.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>day_count_fraction</code>	<code>start/end: date-like ; convention .</code>	<code>float</code> year fraction.	نامعتبر <code>date order/convention</code> <code>ValueError ;</code> <code>day_count_fraction(date(2025,1,1), date(2026,1,1)) == 1.0 .</code>
<code>treasury_yield</code>	zero-coupon price/face/maturity/settlement.	شامل <code>dict</code> <code>DiscountFactor, EffectiveAnnualYield, ContinuousYield, SimpleAnnualYield, BankDiscountYield, MacaulayDuration, ModifiedDuration, Convexity, DV01, DaysToMaturity/Tenor .</code>	face/date/day-count/قیمت نامعتبر <code>ValueError ;</code> مثال <code>.deterministic</code>
<code>bond_price</code>	یا <code>cashflows</code> صریح، یا پارامترهای ساخت <code>annual_yield ; schedule . price_type</code>	<code>float</code> clean یا dirty price.	cashflow/rate/frequency/date مثال: <code>ValueError ;</code> <code>bond_price(0.2, [(date(...), amount)], ...)</code>
<code>yield_to_maturity</code>	قیمت مشاهده‌شده؛ <code>price</code> صریح یا <code>cashflows</code> <code>settlement_date/maturity_date/face_value/coupon_rate/frequency/issue_date</code> برای schedule: <code>accrued_interest/price_type clean/dirty ; compounding tolerance/max_iterations solver.</code>	با <code>float</code> annual yield یا انتخابی compounding.	price bounds/عدم bracket <code>ValueError ;</code> یا عدم همگرایی فصل درآمد ثابت round-trip مثال
<code>bond_analytics</code>	ورودی schedule مانند <code>yield_to_maturity ; annual_yield=None</code> یعنی حل از YTM <code>price ;</code> <code>day_count/compounding/price_type convention</code> <code>bump_size=0.0001</code> شوک DV01.	<code>dict</code> با price/yield. <code>MacaulayDuration, ModifiedDuration, Convexity, DV01</code> و metadata schedule.	نامعتبر cashflow/rate/date/price <code>ValueError ;</code> ناموفق solver یا مثال deterministic.
<code>build_yield_curve</code>	node های rate/discount, compounding, duplicate و monotonic guard.	<code>YieldCurve ;</code> و <code>node_metadata</code> با کلیدهای دقیق <code>diagnostics</code> <code>input_node_count, node_count, duplicate_count, duplicate_policy, monotonic_discount_enforced .</code>	node کم/duplicate/discount غیرمثمر/non-monotonic <code>ValueError ;</code> مثال curve.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_ifb_yield_table</code>	<code>category: str='treasury'</code> IFB. دسته جدول صفحه	<code>DataFrame[Symbol, Price, LastTradeJalali, LastTradedate, PublishJalali, PublishDate, MaturityJalali, Maturity, Volume, ReferenceYTM, ReferenceSimpleYield, ReferenceSource]</code> ; attrs <code>provenance URL/time</code> .	<code>category/HTML/schema/connection</code> IFB. comparison : مثال typed
<code>get_debt_yields</code>	<code>debt_type=None</code> یا <code>str/sequence</code> از <code>treasury, erad, gam, other</code> و طبقه‌بندی IFB رسمی <code>all</code> نمای بر پیشوند نماد <code>exact</code> .	schema IFB + : <code>DebtType</code> attrs <code>cashflows_inferred=False</code> , <code>classification='exact_symbol_prefix'</code> .	فیلتر <code>InvalidParameterError</code> و HTTP/HTML <code>ConnectionError/DataParameterError</code> : <code>singError</code> <code>att.get_debt_yields(["erad", "gam"])</code>
<code>get_treasury_yields</code>	live universe: <code>min_volume</code> ; <code>face_value_source</code> ; maturity override/map: <code>allow_no_trade</code> ; source.	هویت / : <code>TREASURY_COLUMNS</code> <code>price provenance</code> , yield, چهار <code>duration/convexity/DV01</code> , stale/status: attrs source/universe/fetch time.	<code>InvalidParameterError</code> , <code>StockNotFoundError/AmbiguousSymbolError</code> , <code>ConnectionError/DataParameterError</code> ; singError ; اخرا live مثال .
<code>get_treasury_yield_history</code>	history OHLC: <code>max_requests</code> فقط <code>use_ifb_reference</code> reference: settlement per row مگر override.	<code>TREASURY_HISTORY_COLUMNS</code> با <code>S</code> <code>TradeDate, JalaliDate, Open, High, Low, Close, Final, Volume...</code> و analytics: attrs failures/request/provenance.	parameter/resolver/provider/parsing: partial در attrs/warning history.
<code>get_treasury_yield_history</code>	کامل یکسان signature با alias identity	object/schema/attrs همان <code>get_treasury_yield_history</code> .ry	همان؛ <code>att.get_treasury_yields_history is att.get_treasury_yield_history</code>
<code>get_yield_curve</code>	حداقل <code>min_nodes</code> ; duplicate ;default volume-weighted extrapolation opt-in	از <code>YieldCurve</code> calibrated snapshot live: diagnostics و metadata nodeها را ثبت <code>provenance/no-lookahead</code> می‌کند.	node ناکافی/invalid curve provider typed: <code>ValueError</code> و curve live. مثال
<code>get_yield_curve_history</code>	trade date: جدا برای هر curve ; بدون budget hard , <code>maturity_map</code> عمومی extrapolate.	panel <code>DataFrame</code> با <code>CURVE_HISTORY_COLUMNS</code> , <code>CurveID, CurveStatus, CurveError, CurveNodeCount</code> و flags no-lookahead: attrs failures/request.	parameter/budget/provider/curve errors در status/attrs: curve history. مثال

قرارداد API: شاخص‌های صنعت

```
list_industry_indices(progress=True, include_member_count=False, refresh=False, max_workers=6)
get_industry_members(industry, include_live=True, include_client_type=False, include_orderbook=False,
progress=True, refresh=False)
get_industry_snapshot(industries=None, include_client_type=True, include_orderbook=False,
include_empty=False, progress=True, refresh=False, max_workers=6)
get_industry_history(industry, start=None, end=None, limit=0, ascending=True, progress=True)
get_industry_members_history(industry, days=30, ascending=True, progress=True, refresh=False)
compare_industries(industries, start=None, end=None, limit=0, metric="close", ascending=True,
progress=True, max_workers=6)
get_industry_relative_strength(industries, benchmark, start=None, end=None, limit=0, metric="close",
ascending=True, progress=True, max_workers=6)
get_industry_correlation(industries, start=None, end=None, limit=0, ascending=True, progress=True,
max_workers=6)
get_industry_membership_overlap(industries, progress=True, refresh=False, max_workers=6)
get_industry_membership_churn(industries, days=0, progress=True, refresh=False, max_workers=6)
get_industry_membership_events(industries, days=30, progress=True, refresh=False, max_workers=6)
get_industry_concentration(industries, top_n=10, include_weights_by_market_value=True, progress=True,
refresh=False, max_workers=6)
get_industry_momentum_profile(industries, windows=(5, 20, 60), start=None, end=None, progress=True,
max_workers=6)
get_industry_correlation_neighborhood(industry, industries=None, top=5, min_correlation=0.0,
by_absolute=False, start=None, end=None, limit=0, ascending=False, progress=True, max_workers=6)
get_industry_health_score(industries=None, start=None, end=None, top_concentration=10,
momentum_windows=(20, 60), progress=True, refresh=False, max_workers=6)
get_industry_intraday(industry, interval='1min', progress=True)
rank_industries(metric='IndexChangePct', top=None, ascending=False, include_client_type=True,
include_orderbook=False, progress=True, refresh=False, max_workers=6)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_industry_i ndices</code>	پیام: <code>progress: bool=True</code> <code>include_member_count:</code> عضویت ۴۵ صنعت را <code>bool=False</code> واکشی می‌کند؛ <code>refresh:</code> را دور می‌زند؛ <code>cache</code> در بازهٔ <code>max_workers: int=6</code> 1..16.	<code>DataFrame[IndustryName,I ndustryNameEn,IndustryGro upCode,IndexInsCode,Index Value,IndexPreviousValue, DayHigh,DayLow,IndexChang e,IndexChangePct,MemberCo unt,HasMembers,ExchangeTi me]</code> attrs source/cache/count.	نامعتبر <code>bool/worker</code> : <code>InvalidParameterError</code> provider <code>ConnectionError/DataPar</code> singError؛ مثال فصل صنعت
<code>get_industry_me mbers</code>	نام/alias/ <code>industry: str int</code> ؛ <code>include_live=True</code> MarketWatch؛ <code>include_client_type=False</code> client metrics؛ پنج <code>include_orderbook=False</code> دو. <code>refresh=False</code> سطح/صف؛ نیاز دارند به live به enrichment گزینه	<code>INDUSTRY_MEMBER_COLUMNS</code> ثابت با هویت، قیمت، معامله market cap تخمینی، client، order book و freshness: attrs exact membership/current/cache/coverage request.	صنعت گم‌شده : <code>StockNotFoundError</code> ترکیب/نوع نامعتبر : <code>InvalidParameterError</code> مثال provider typed <code>get_industry_members("ب ل‌انک")</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_industry_snapshot	<code>industries: ;None str int iterable=None</code> <code>include_client_type=True</code> <code>include_orderbook=False</code> <code>include_empty=False</code> <code>refresh=False</code> <code>max_workers=6</code>	یک ردیف در هر شاخص با <code>INDUSTRY_SNAPSHOT_COLUMN</code> S: وضعیت شاخص، breadth، equal-weight/median/dispersion معامله، flow/coverage، queue/coverage و freshness attrs overlap/current membership/cache	selector/bool/worker یا <code>InvalidParameterError</code> ؛ <code>StockNotFoundError</code> snapshot/schema/provider typed؛ مثال فصل صنعت
get_industry_history	<code>industry</code> اجباری؛ <code>start/end: str None</code> شمسی/میلادی؛ <code>limit: int=0</code> پس از فیلتر؛ <code>ascending: bool=True</code> ؛ <code>progress</code>	<code>DataFrame[IndustryName, IndustryIndexCode, TradeDate, JalaliDate, High, Low, Close, Change, ChangePct]</code> ؛ attrs <code>volume_available=False</code> ، <code>completed_sessions_only=True</code>	تاریخ/limit/order نامعتبر ؛ <code>InvalidParameterError</code> صنعت/typed/provider؛ مثال تاریخچه
get_industry_members_history	<code>industry</code> ؛ <code>days: int=30</code> تاریخ آخر؛ <code>1..30</code> دقیقاً <code>ascending=True</code> ؛ <code>refresh=False</code>	long-form <code>INDUSTRY_MEMBER_HISTORY_COLUMNS</code> ؛ attrs برای اعضای فعلی؛ <code>point_in_time_membership=False</code> ، <code>survivorship_bias_possible=True</code>	days/bool/selector typed؛ provider/schema typed؛ مثال فصل صنعت
compare_industries	<code>industries (str</code> <code>start/end: str None</code> ، <code>limit: int=0</code> ، <code>metric: close price change_pct log_return</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>	int	iterable)؛ <code>start/end: str None</code> ، <code>limit: int=0</code> ، <code>metric: close price change_pct log_return</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>
get_industry_relative_strength	<code>industries</code> ، <code>benchmark</code> ، <code>start/end: str None</code> ، <code>limit: int=0</code> ، <code>metric: close price change_pct log_return</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>	long-form <code>TradeDate, JalaliDate, BenchmarkIndexCode, BenchmarkName, IndustryIndexCode, IndustryName, IndustryReturn, BenchmarkReturn, RelativeStrength</code> ؛ attrs شامل <code>analysis='industry_relative_strength'</code> ، <code>benchmark_index_code</code> ، <code>industry_count</code>	benchmark/generic نامعتبر یا داده worker metrics/worker ناکافی ؛ <code>InvalidParameterError</code> ؛ مثال <code>relative_strength = ...</code>
get_industry_correlation	<code>industries (str</code> <code>start/end: str None</code> ، <code>limit: int=0</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>	int	iterable)؛ <code>start/end: str None</code> ، <code>limit: int=0</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_industry_membership_overlap	industries (str)	int	iterable), progress=True , refresh=False , max_workers=6 .
get_industry_membership_events	industries (str)	int	iterable), days: int=1..30 , progress=True , refresh=False , max_workers=6 .
get_industry_membership_churn	industries (str)	int	iterable), days: int=1..30 , progress=True , refresh=False , max_workers=6 .
get_industry_concentration	industries (str)	int	iterable), top_n: int=10 , include_weights_by_market_value =True, progress=True , refresh=False , max_workers=6 .
get_industry_momentum_profile	industries (str)	int	iterable), windows=(5,20,60) , start/end: str None , progress=True , max_workers=6 .
get_industry_correlation_neighborhood	هدف industry (str)	int), industries , top=5 , min_correlation=0.0 , by_absolute=False , start/end: str None , limit=0 , ascending=False , progress=True , max_workers=6 .	DataFrame[IndustryName, IndustryIndexCode, NeighborName, NeighborIndexCode ,Correlation,CorrelationAbs,CorrelationRank] ; شامل attrs analysis='industry_correlation_neighborhood' , target_industry_code , target_industry_name , top , min_correlation , by_absolute , start_date , end_date .
get_industry_health_score	industries=None (str)	int	iterable), start/end: str None , top_concentration=10 , momentum_windows=(20,60) , progress=True , refresh=False , max_workers=6 .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_industry_intraday</code>	<code>industry</code> : <code>interval</code> : یکی از <code>'1min'</code> , <code>raw</code> , <code>1min</code> , <code>5min</code> , <code>15min</code> , <code>30min</code> , <code>60min</code> , <code>1h</code> ; <code>progress</code> .	<code>DataFrame[IndustryName, IndustryIndexCode, Timestamp, JalaliDate, Interval, Open, High, Low, Close, Change, ChangePct]</code> ; <code>timezone</code> : تهران؛ <code>attrs</code> : مصنوعی و بدون volume latest-day.	<code>interval/selector</code> <code>InvalidParameterError/StockNotFoundError</code> : <code>provider/schema typed</code> مثال: <code>get_industry_intraday("minخودرو", "5")</code> .
<code>rank_industries</code>	<code>metric</code> : مستند؛ canonical/alias <code>top</code> : <code>int None</code> ; <code>ascending=False</code> ; <code>client/order</code> switches: <code>refresh</code> ; <code>max_workers</code> .	<code>Rank</code> + <code>snapshot</code> : تمام ستون‌های <code>attrs</code> : حذف metric تهی <code>ranking_metric, ranking_ascending</code> و <code>provenance</code> . <code>snapshot</code> .	<code>metric/top/bool/worker</code> نامعتبر <code>InvalidParameterError</code> : <code>provider typed</code> مثال: <code>rank_industries(metric="breadth", top=5)</code> .

قرارداد API: اختیار معامله و فهرست ابزارها

```

list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
black_scholes_price(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
black_scholes_greeks(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
option_price_bounds(spot, strike, time_to_expiry, rate, option_type='call', dividend_yield=0.0,
exercise_style='european')
implied_volatility(option_price, spot, strike, time_to_expiry, rate, option_type='call',
dividend_yield=0.0, exercise_style='european', lower_volatility=0.0, upper_volatility=5.0,
tolerance=1e-08, max_iterations=200)
get_option_market(exchange=0, underlying=None, progress=True, max_requests=1)
analyze_option_chain(options=None, underlying=None, spot=None, risk_free_rate=None, yield_curve=None,
dividend_yield=0.0, valuation_date=None, exercise_style='european', parity_tolerance=None,
liquidity_weights=None, progress=True, *, allow_unverified_freshness=True)
option_put_call_ratios(options, group_by='market')
get_option_history(symbol, start=None, end=None, limit=0, include_today=False, snapshot_path=None,
progress=True, max_requests=3)
save_option_snapshot(path, options=None, exchange=0, progress=True, *, lock_timeout=10.0,
stale_lock_seconds=300.0)
load_option_snapshots(path)
list_etfs(progress=True)
list_bonds(progress=True)
list_debt_instruments(debt_type=None, active_only=False, progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False, strategy=None,
commodity_underlying=None, classification_status=None)
list_listed_funds(progress=True, *, fund_category=None, strategy=None, commodity_underlying=None,
classification_status=None, include_unknown=True)
list_indices(progress=True)
get_index_companies(index_name, progress=True)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_options</code>	<code>underlying: str None</code> filter نام underlying.	قراردادها با هویت <code>DataFrame</code> ، <code>OptionType, Underlying*, Strike, BeginDate, EndDate, DaysToExpiry, ContractSize</code> و قیمت/حجم.	یا <code>provider parse/connection</code> frame تهی؛ مثال فصل اختیار
<code>get_options_chain</code>	اجباری؛ <code>underlying</code> fan-out OI از <code>fetch_oi=False</code> جلوگیری می‌کند.	<code>calls:</code> دقیقاً شامل <code>dict DataFrame, puts: DataFrame, underlying_name, underlying_price, expiry_dates, market_time</code> ؛ <code>fetch_oi</code> ستون‌های <code>OpenInterest, ContractSize, BeginDate, EndDate</code> .	؛ مثال <code>underlying/provider errors</code> ؛ فصل اختیار <code>chain</code> .
<code>black_scholes_price</code>	فقط European: scalar math params.	<code>float</code> premium.	نامعتبر <code>bounds/type/style</code> ؛ مثال <code>ValueError</code> ؛ <code>deterministic</code> .
<code>black_scholes_greeks</code>	ورودی‌های <code>spot/strike/time_to_expiry/rate/volatility</code> عددی؛ <code>option_type∈{call,put}</code> ؛ <code>dividend_yield</code> نرخ پیوسته؛ <code>exercise_style='european'</code> تنها سبک پشتیبانی‌شده.	<code>dict[Delta, Gamma, Vega, Vega1Pct, ThetaPerYear, ThetaPerDay, Rho, Rho100bp, Status]</code> ؛ یا <code>Status='ok'</code> یا <code>undefined_at_expiry_or_zero_volatility</code> .	نامعتبر <code>constraint</code> ؛ مثال <code>ValueError</code> ؛ <code>Greeks</code> .
<code>option_price_bounds</code>	بدون <code>volatility</code> ؛ <code>no-arbitrage bound</code> .	<code>tuple (lower: float, upper: float)</code> .	مثال <code>ValueError</code> ؛ <code>deterministic</code> با خروجی <code>(4.87705755, 100.0)</code> .
<code>implied_volatility</code>	<code>premium + bracket/tolerance/iterations: tolerance>0</code> ، عدد صحیح مثبت <code>max_iterations</code> و <code>0<=lower<upper</code> .	با <code>dict ImpliedVolatility, Status, Iterations</code> ؛ <code>status</code> های <code>ok/missing/expiry/out_of_bounds/no_bracket/non_converged</code> و کلیدهای تشخیصی اختیاری.	و <code>bounds</code> ناموجود/خارج <code>premium</code> هستند، نه <code>status</code> عدم همگرایی؛ فقط <code>exception</code> ؛ نامعتبر <code>constraint/style/bracket</code> ؛ مثال <code>ValueError</code> ؛ <code>IV</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_option_mark</code> <code>et</code>	<code>exchange: int=0 ; underlying</code> <code>filter: max_requests=1 bulk.</code>	<code>DataFrame[InsCode, PairID</code> <code>, PairSequence, ISIN, Symbol</code> <code>, Name, OptionType, Underlyi</code> <code>ngInsCode, UnderlyingSymbo</code> <code>l, UnderlyingName, Contract</code> <code>Size, Strike, BeginDate, End</code> <code>Date, DaysToExpiry, Last, Cl</code> <code>ose, Yesterday, Volume, Valu</code> <code>e, TradeCount, NotionalValu</code> <code>e, OpenInterest, YesterdayO</code> <code>penInterest, BidPrice, AskP</code> <code>rice, BidVolume, AskVolume,</code> <code>UnderlyingLast, Underlying</code> <code>Close, Price, PriceSource, A</code> <code>sOf, AsOfSource, SnapshotFr</code> <code>eshnessKnown, PriceFreshne</code> <code>ssKnown, Stale, NoTrade, Ana</code> <code>lyticsEligible, AnalyticsE</code> <code>ligibilityReason, Metadata</code> <code>Conflict, Source] ; attrs</code> <code>snapshot provenance.</code>	<code>exchange/budget/filter</code> <code>InvalidParameterError ;</code> <code>provider typed</code> مثال: <code>snapshot.</code>
<code>analyze_option_</code> <code>chain</code>	می‌کند: <code>fetch</code> <code>options=None</code> <code>spot/rate/curve overrides:</code> <code>allow_unverified_freshnes</code> <code>s</code> explicit risk switch.	<code>input columns +</code> <code>TimeToExpiry, Spot, RiskFr</code> <code>eeRate, DividendYield, Impl</code> <code>iedVolatility* , Greeks per</code> <code>unit/contract.</code> <code>spread/depth/liquidity.</code> <code>ParityResidual, ParitySta</code> <code>tus, ImpliedForward, Analyt</code> <code>icsReliability ; attrs</code> <code>assumptions/warnings.</code>	<code>input type TypeError ;</code> <code>math/freshness/rate/column</code> <code>problems ValueError ;</code> <code>provider errors if fetch</code> مثال: <code>حرفه‌ای.</code>
<code>option_put_call</code> <code>_ratios</code>	از یک <code>options: DataFrame</code> دقیقاً یکی <code>group_by</code> ؛ <code>AsOf</code> از <code>market, underlying, expiry, u</code> <code>nderlying_expiry .</code>	<code>DataFrame</code> با <code>call/put</code> <code>volume/value/OI.</code> <code>PCRVVolume, PCRValue, PCROp</code> <code>enInterest</code> و <code>status</code> دقیق <code>PCRVVolumeStatus, PCRValue</code> <code>Status, PCROpenInterestSta</code> <code>tus ; attrs coverage/source</code> ورودی	ستون/ <code>AsOf</code> یا <code>group</code> یا <code>قرارداد تکراری</code> نامعتبر <code>ValueError</code> و نوع غیر <code>TypeError</code> <code>DataFrame</code> مثال <code>PCR.</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_option_history</code>	قرارداد دقیق؛ + <code>server history</code> <code>include_today</code> ; فقط <code>snapshot_path</code> join <code>snapshot</code> های <code>opt-in</code>: <code>budget</code>	<code>DataFrame[Timestamp, InsCode, Symbol, Open, High, Low, Close, Last, Volume, Value, TradeCount, OpenInterest, BidPrice, AskPrice, BidVolume, AskVolume, UnderlyingLast, UnderlyingClose, ContractSize, Strike, EndDate, Price, PriceSource, Source, AsOf, Stale, NoTrade, AnalyticsEligible]</code> ; <code>attrs failures/no-lookahead</code>.	<code>selector/date/budget/provider/snapshot</code> مثال تاریخچه <code>schema errors</code>؛ اختیار.
<code>save_option_snapshot</code>	lock: <code>fetch</code> یک <code>options=None</code> مثبت stale lock نامنفی و <code>timeout</code>	شده با <code>dedupely</code> ترکیب <code>DataFrame</code> <code>OPTION_COLUMNS</code> ; <code>attrs schema_version, source, path</code> و <code>lock</code> با <code>JSON versioned</code> فایل . اتمیک نوشته می‌شود <code>replace</code>.	گم‌شده <code>parent</code> <code>FileNotFoundError</code> ، <code>DataFrame</code> غیر <code>options</code> مقدار نامعتبر ، <code>TypeError</code> قفل ، <code>ValueError</code> اگر <code>provider</code> <code>TimeoutError</code> ؛ <code>snapshot</code> ؛ مثال <code>fetch</code>.
<code>load_option_snapshots</code>	<code>JSON versioned</code> <code>path</code>	با <code>DataFrame</code> ؛ فایل <code>OPTION_COLUMNS</code> و <code>attrs</code> تهی با <code>frame</code> گم‌شده خطا نیست و <code>attrs['status']='missing'</code> می‌دهد .	خراب <code>JSON</code> و <code>version</code> <code>JSONDecodeError</code> مثال ؛ <code>ValueError</code> ناسازگار <code>history</code>.
<code>list_etfs</code>	فقط <code>progress</code> .	<code>DataFrame[InsCode, ISIN, Symbol, Name, Last, Close, Yesterday, Volume, Value, TradeCount, Low, High, NAV, NAV_Discout, Change, ChangePct, MarketCode]</code> .	؛ مثال <code>provider typed/empty</code> <code>list_etfs().query("NAV_Discout < -1")</code> .
<code>list_bonds</code>	فقط <code>progress</code> ؛ <code>signature</code> قدیمی ثابت است.	<code>DataFrame[InsCode, ISIN, Symbol, Name, BondType, Ticker, MaturityJalali, MaturityGregorian, DaysToMaturity, Last, Close, Yesterday, Volume, Value, TradeCount, Change, ChangePct]</code> نیز پشتیبانی <code>'gam'</code> <code>BondType</code> می‌شود.	نامعلوم <code>maturity</code> <code>parse/provider</code>؛ <code>nullable</code>؛ <code>att.list_bonds(progress=False)</code> .
<code>list_debt_instruments</code>	<code>str/sequence</code> یا <code>debt_type=None</code> از <code>treasury, erad, gam, murabaha, ijara, salaf, other</code> ؛ <code>active_only=False</code> ؛ <code>progress</code> .	+ <code>list_bonds</code> <code>schema</code> منبع <code>DebtType</code> <code>attrs</code> ، <code>coverage</code> و فیلترها	فیلتر <code>bool</code>؛ <code>InvalidParameterError</code> و خطاهای <code>provider</code>؛ <code>att.list_debt_instruments("gam", .active_only=True)</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_funds</code>	از <code>fund_type: str list None</code> ۱۵ دسته ثابتی؛ <code>listed_only=False</code> ؛ فیلترهای <code>keyword-only: strategy</code> ، <code>commodity_underlying</code> ، <code>classification_status</code> ؛ نام است <code>alias</code> <code>venture</code> قدیمی	با <code>registry rich funds</code> <code>NAV/returns/composition/manager</code> و محورهای <code>registry_category, asset_</code> <code>exposure, strategy_tags</code> و شواهد طبقه‌بندی؛ بدون تضمین <code>InsCode</code> .	<code>type/category/combo/filter</code> نامعتبر <code>InvalidParameterError</code> یا <code>legacy: provider</code> رفتار سازگار خطا دار کنار گذاشته می‌شود؛ مثال فصل صندوق
<code>list_listed_funds</code>	یک <code>MarketWatch bulk</code> ؛ <code>fund_category</code> ، <code>strategy</code> ، <code>commodity_underlying</code> ، <code>classification_status</code> ؛ <code>.include_unknown=True</code>	شامل <code>LISTED_FUND_COLUMNS</code> و تمام <code>InstrumentType</code> ؛ هر دو نوع <code>taxonomy</code> ۳۰۵ و <code>attrs</code> ۳۸۰؛ <code>no_fuzzy_join=True, regis</code> <code>try_joined=False, taxonomy</code> <code>_version</code> .	فیلتر نامعتبر <code>InvalidParameterError</code> ؛ <code>connection/parsing</code> ؛ مثال <code>list_listed_funds(commo</code> <code>dity_underlying="gold")</code>
<code>list_indices</code>	فقط <code>progress</code> .	<code>DataFrame[Name, InsCode, V</code> <code>alue, High, Low, Change, Chan</code> <code>gePct]</code> ؛ از <code>Change</code> <code>ChangePct</code> واحد شاخص و حرکت درصد است.	مسیر <code>legacy</code> در خطای <code>provider</code> <code>frame</code> تهی/پیام؛ برای صنعت <code>list_industry_indices</code> پیشنهاد می‌شود.
<code>get_index_companies</code>	فارسی یا <code>index_name: str</code> <code>InsCode</code> .	<code>DataFrame[Symbol, Name, In</code> <code>sCode, Close, Yesterday, Las</code> <code>t]</code> .	در مسیر <code>provider/gm</code> شده <code>index</code> تهی/پیام؛ مثال فصل <code>legacy frame</code> شاخص.

قرارداد API: تحلیل مدیریتی بازار

```
compare_symbols(symbols, start=None, end=None, benchmark=None, align='inner', annualization=240,
min_observations=20, include_client_type=True, progress=True, history_data=None, client_data=None,
strict=False)
get_liquidity_metrics(symbols=None, start=None, end=None, window=20, annualization=240,
include_live=True, progress=True, history_data=None, live_data=None, strict=False)
get_market_regime(benchmark='شاخص کل', start=None, end=None, lookback=60, liquidity_window=20,
annualization=240, weights=None, progress=True, max_requests=30, live_data=None,
benchmark_history=None, activity_history=None)
get_market_map(group_by='symbol', size='value', color='return', top=100, flow=None,
instrument_types=None, data=None)
plot_market_map(map_data=None, output_path=None, show=False, title=None)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>compare_symbols</code>	<code>benchmark</code> ، الزامی؛ تاریخ <code>symbols</code> ، و حداقل <code>align</code> ، <code>annualization</code> تقویم و <code>client type</code> مشاهده؛ دریافت ورودی‌های صریح قابل کنترل اند	۳۱ ستون شامل بازده/ریسک/افت، <code>beta/correlation/alpha</code> ، نقدشوندگی و حقیقی؛ <code>attrs</code> مبنا، <code>alignment</code> ، پوشش، نماد مفقود و واحد ریال.	نامعتبر <code>enum/count/date</code> <code>InvalidParameterError</code> ؛ یا <code>malformed</code> صریح <code>history</code> ناموفق <code>strict</code> <code>DataParsingError</code> ؛ نمونه فصل تحلیل مدیریتی

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_liquidity_metrics</code>	کل snapshot فقط <code>symbols=None</code> بازار؛ نماد مشخص تاریخچه را هم می‌گیرد؛ و ورودی <code>live</code> کل بازه؛ <code>window=None</code> صریح قابل کنترل.	۳۲ ستون؛ <code>zero-trade/return</code> ، <code>Amihud</code> به میلیارد ریال، ارزش/حجم/معامله، <code>volatility</code> ، <code>turnover</code> ، <code>spread</code> ، <code>L1/L5</code> و <code>depth</code> ؛ <code>attrs</code> پوشش و واحد.	پارامتر <code>date</code> /نامعتبر <code>InvalidParameterError</code> ؛ <code>history malformed/strict</code> <code>DataParsingError</code> ؛ نمونه فصل نقدشوندگی.
<code>get_market_regime</code>	و پنجره‌ها؛ پنج وزن اختیاری؛ <code>benchmark</code> صریح <code>DataFrame</code> بودجه درخواست؛ سه <code>replay</code> برای.	یک ردیف ۳۰ ستونی با <code>label/score/confidence</code> ، هر پنج <code>component</code> و <code>weight</code> ، اجزای خام و <code>quality</code> ؛ <code>attrs</code> آستانه، روش و <code>missing</code> .	وزن/نوع <code>date/budget</code> نامعتبر <code>InvalidParameterError</code> ؛ تاریخچه <code>benchmark malformed</code> <code>DataParsingError</code> ؛ نمونه فصل رژیم.
<code>get_market_map</code>	چهار گروه‌بندی، چهار معیار اندازه، چهار معیار رنگ؛ <code>flow</code> ، <code>top</code> ، نوع ابزار و <code>snapshot</code> صریح.	۲۳ ستون <code>renderer-independent</code> شامل اندازه/رنگ، دو وزن <code>universe/display</code> ، پوشش رنگ، <code>freshness</code> و ریال؛ <code>attrs</code> انتخاب‌ها و <code>partial</code> بودن.	نامعتبر <code>enum/filter</code> <code>InvalidParameterError</code> ؛ ستون لازم ناقص <code>DataParsingError</code> ؛ نمونه نقشه.
<code>plot_market_map</code>	HTML، اختیاری، مسیر <code>map_data</code> نمایش و عنوان.	<code>plotly.graph_objects.Fig</code> فقط با HTML <code>ure</code> ؛ نوشتن <code>output_path</code> .	<code>Plotly</code> مفقود <code>ImportError</code> ؛ تهی/ناقص <code>frame</code> <code>DataParsingError</code> ؛ مثال فصل نقشه.

قرارداد API: بورس انرژی و بورس کالا

```

get_energy_auctions(status='upcoming', board='physical', top=100)
get_energy_auction(auction_id, include_trades=True, include_instrument_history=False)
get_energy_market_overview(market='all')
list_power_instruments(market='all', top=100)
list_energy_securities(kind='all', active=True, top=100, enrich_futures=True)
get_energy_future_contract(ins_code)
get_commodity_market(kind='all')
get_commodity_physical_history(start, end=None)
get_commodity_physical_summary(start, end=None, hall='all')
get_commodity_market_activity(start, end=None, market='all')
get_futures_curve(contracts, underlying=None, valuation_date=None, price='settlement', spot_price=None)
get_calendar_spreads(curve)
analyze_cash_and_carry(curve, annual_rate, storage_rate=0.0, convenience_yield=0.0)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_energy_auctions</code>	<code>status</code> ، <code>board</code> ، <code>all</code> fan-out ؛ <code>top=1..1000</code> صریح.	کامل عرضه با قیمت، <code>DataFrame</code> ، <code>lot</code> ، شرایط پرداخت/تحويل منبع/ <code>attrs</code> وضعیت و معامله؛ درخواست.	نامعتبر <code>enum/top</code> <code>InvalidParameterError</code> ؛ مثال فصل <code>typed/schema</code> /شبکه انرژی.
<code>get_energy_auction</code>	مثبت؛ <code>auction_id</code> ؛ <code>include_trades=True</code> تاریخچه <code>opt-in</code> ابزار.	و کلیدهای <code>auction</code> با <code>dict</code> <code>TradeScope</code> معامله درخواستی؛ تفاوت دامنه را ثبت می‌کند.	نامعتبر یا پاسخ ناقص <code>ID/bool</code> <code>typed</code> ؛ مثال <code>single auction</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_energy_market_overview</code>	market flow label یکی از هشت <code>all</code> مستند یا	<code>DataFrame[Market, Flow, GregorianDate, JalaliDate, Time, TradeCount, Volume, Value, ValueUnit, StateCode, State, Source]</code>	market/provider/schema typed: overview مثال
<code>list_power_instruments</code>	<code>market ∈ {standard, green, free, all}</code> <code>top=1..1000</code>	مشترک ابزار انرژی schema، <code>ValueUnit='rial'</code>	enum/top/provider typed: مثال برق سبز
<code>list_energy_securities</code>	<code>kind ∈ {all, active, top; enrich_futures}</code> ؛ پنج دسته	enrichment مشترک بازار؛ attrs سررسید/اندازه/تسویه؛ <code>request_count</code>	enum/bool/top/provider/detail typed: مثال اوراق انرژی
<code>get_energy_future_contract</code>	دقیق <code>ins_code</code>	یک ردیف مشخصات، تاریخ‌ها، قیمت‌ها، position، fee، margin، delivery rules و limits	InsCode/provider/schema typed: مثال فصل آتی
<code>get_commodity_market</code>	<code>kind ∈ {certificate, standard, _salaf, fund, future, all}</code> و های مستند alias	هویت/قیمت / <code>DataFrame</code> پنج سطح/client type/معامله سفارش و مشتقه؛ <code>ValueUnit='thousand_rial'</code>	kind/provider/schema typed: معتبر است frame بازار خالی
<code>get_commodity_physical_history</code>	یعنی <code>end=None</code> ، الزامی <code>start</code> امروز تهران	روزانه <code>GregorianDate, JalaliDate, Volume, Value, ValueUnit, Source</code> ؛ واحد میلیون ریال	date order/provider/embedded JSON typed: مثال تاریخچه
<code>get_commodity_physical_summary</code>	بازه مانند بالا؛ <code>hall='all'</code> یا ID/نام exact	<code>HallID, Hall, Volume, Offer Volume, TradeToOfferRatio, Value</code> و بازه؛ واحد میلیون ریال	no-match: provider typed/تاریخ frame خالی معتبر
<code>get_commodity_market_activity</code>	بازه؛ <code>market='all'</code> یا ID/نام exact	فعالیت تجمیعی فیزیکی/آتی/اختیار/ مالی/گواهی؛ جزئیات قرارداد نیست	تاریخ/provider typed: مثال market ID 3 برای اختیار
<code>get_futures_curve</code>	underlying صریح؛ فیلتر contracts؛ اختیاری price: spot؛ اولویت valuation؛ مثبت	tenor، basis مرتب با term structure، provenance مطلق/درصدی/سالانه و قیمت	ستون/date/price/spot/نامعتبر <code>InvalidParameterError</code> ؛ مثال curve
<code>get_calendar_spreads</code>	دارای <code>curve</code> <code>ExpiryDate, Price</code>	جفت سررسید مجاور، spread، annualized spread و structure	نامعتبر/date/schema <code>InvalidParameterError</code> ؛ مثال spread
<code>analyze_cash_and_carry</code>	نرخ‌های spot/price/tenor با curve؛ مثبت base و finite	ورودی + <code>FairValue, Mispricing, MispricingPct, Signal</code> و فرض‌ها در attrs	نامعتبر/rate/schema <code>InvalidParameterError</code> ؛ توصیه معامله نیست signal

قرارداد alias و wrapperهای backward-compatible

قدیمی و تفاوت رفتاری مخفی نماند. تمام پارامترها signature های این جدول مدخل مستقل دارند تا alias است override را دارند؛ فقط تفاوت صریح جدول target تابع type/default/constraint

```

stock(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, adjust_volume=False,
return_type=None, ascending=True, save_path=None, include_today=False, *, ins_code=None,
asset_type='auto', **kwargs)
stock_RI(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_RL(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
stockdetail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_information(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_statistics(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stocklist(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
currency_coin(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
market_watch()
market_client_type()
market_data()

```

alias/wrapper	و تفاوت target	خروجی/schema/attrs	خطا و مثال
stock	همان target <code>get_history</code> ; signature و implementation history.	همان DataFrame/attrs.	همان خطاها؛ <code>att.stock("فملی", .limit=10)</code>
stock_RI	target <code>get_client_type</code> .	همان DataFrame/attrs.	همان؛ <code>att.stock_RI("فملی", .limit=10)</code>
stock_RL	که به wrapper compatibility identity می‌کند؛ <code>stock_RI</code> delegate alias signature نیست ولی برابر است.	همان DataFrame/attrs.	همان؛ برای کد جدید <code>. get_client_type</code>
stock_capital_increase	target canonical <code>get_capital_increase</code> .	همان frame/None قدیمی.	همان.
stock_intraday	target <code>get_intraday</code> ; signature برابر.	همان tick/candle frame.	همان.
stockdetail	target <code>get_detail</code> .	همان key/value frame یا <code>None</code> .	همان.
stock_information	target <code>get_info</code> .	همان key/value frame یا <code>None</code> .	همان.
stock_statistics	target <code>get_stats</code> .	همان key/value frame یا <code>None</code> .	همان.

alias/wrapper	و تفاوت target	خروجی/schema/attrs	خطا و مثال
<code>stock_introduction</code>	target <code>get_introduction</code> ; wrapper legacy unsupported.	هیچ خروجی موفق.	همیشه <code>UnsupportedDataSourceError</code> پیش از I/O . <code>ror</code>
<code>stocklist</code>	همان target <code>get_symbols</code> ; kwarg های alias و signature.	همان فهرست.	همان.
<code>shareholders</code>	target <code>get_shareholders</code> .	همان frame/None.	همان.
<code>currency_coin</code>	target <code>get_currency</code> ; اکنون سازگار را می پذیرد TGLJ همه ۸۷ سری کاتالوگ	همان frame/MultiIndex و attrs دقیق source/category/unit .	نام/HTTP/schema typed؛ برای کد جدید <code>get_tgju_history</code> .
<code>market_watch</code>	تابع صفرآرگومان canonical قدیمی target :snapshot مفهومی target <code>get_market_snapshot</code> .	<code>dict[stocks,order_book,market_time,...]</code> ; ستون های legacy <code>Yesterday/BaseVolume/Change</code> حفظ شده و ستون های corrected additive هستند.	<code>ConnectionError/DataParsingError</code> ; <code>snapshot=att.market_watch()</code> .
<code>market_client_type</code>	تابع صفرآرگومان bulk target مفهومی target <code>get_market_client_type</code> .	<code>CLIENT_COLUMNS</code> DataFrame.	typed provider errors.
<code>market_data</code>	صفرآرگومان به wrapper deprecated target <code>market_watch</code> ; identity alias نیست.	همان dict.	همان؛ برای کد جدید <code>get_market_snapshot/get_live_market</code>

دو alias identity غیر legacy نیز قبلاً مدخل دارند: `get_orderbook_history is get_order_book_history` و `get_treasury_yields_history is get_treasury_yield_history`. تفاوت signature یا خروجی ندارند.

ها نیز دقیقاً چنین است signature constructor exception

```

AlgotikTSEError(*args)
AmbiguousSymbolError(*args)
ConnectionError(*args)
DataParsingError(*args)
InvalidParameterError(*args)
StockNotFoundError(*args)
UnsupportedDataSourceError(*args)

```

الگوهای کاربردی

فیلتر قدرت خریدار حقیقی با نقدشوندگی

```
live = att.get_live_market()
screen = live.loc[
    (live["IndividualPower"] > 1.5)
    & (live["Value"] > 50_000_000_000)
    & live["is_realtime_fresh"].fillna(False)
].sort_values("EstimatedNetIndividualFlow", ascending=False)

print(screen[[
    "Symbol", "Last", "ChangePct", "IndividualPower",
    "EstimatedNetIndividualFlow", "SpreadBps", "L5Imbalance",
]])
```

look-ahead بدون backtest

```
db = "research.sqlite"
att.save_market_snapshot(db)

fund = att.get_market_fundamentals_history(db, symbols="فملی")
assert fund.attrs["no_lookahead"] is True
assert fund.attrs["current_eps_used"] is False

curves = att.get_yield_curve_history(
    start="1403-01-01", end="1403-03-31", max_requests=100, progress=False
)
safe_curves = curves.loc[curves["CurveNoLookahead"].fillna(False)]
```

مانیتور بازار و archive مستقل

```
db = "monitor.sqlite"

for event in att.watch_market(
    interval=3,
    max_updates=20,
    record_to=db,
    notifications=("messages", "state"),
):
    if event.kind in {"initial", "delta"}:
        print(event.sequence, len(event.changed_inscodes))

# archive_to مستقل فعال شود helper باید روی
att.get_market_messages(flow=0, top=50, archive_to=db)
messages = att.get_market_messages_history(db, flow=0)
```

منابع داده

منبع	استفاده
های subdomain و tsetmc.com (TSETMC رسمی)	قیمت، client type، market watch، سفارش، trades، پیام، وضعیت، ابزار، صندوق و اطلاعات بازار
TSETMC / بورس انرژی	حراجها، معاملات همان عرضه، تابلوهای برق و انرژی و مشخصات آتی انرژی
بورس کالای ایران (ime.co.ir) و (cdn.ime.co.ir)	تابلوی گواهی/سلف/صندوق/آتی، بازار فیزیکی و آمار تجمیعی بازارها
فرا بورس ایران (ifb.ir/ytm.aspx)	جدول مرجع YTM برای مقایسه/دسته بندی اوراق؛ منبع مجاز و با provenance جدا
TGJU (api.tgju.org)	تاریخچه ۸۷ سری تأیید شده ارز آزاد/رسمی، طلا، نقره، سکه/حباب و فلزات گران بهای جهانی؛ نام و واحد در کاتالوگ عمومی

کدال منبع این پکیج نیست؛ حتی endpointهای proxy شده آن زیر host دیگر در boundary شبکه رد می شوند.

تست و مشارکت

تست پیش فرض کاملاً آفلاین است:

```
python -m pytest -m "not online"
```

یا:

```
make test
```

است: opt-in آفلاین محدود و smoke

```
python -m pytest -m online --timeout=30
```

تست آفلاین به وضعیت بازار/provider وابسته است و جایگزین تست deterministic آفلاین نیست. پیش از PR:

```
python -m pytest tests/test_release_offline.py -q
python -m pytest -m "not online" -q
```

برای مشارکت، issue یا pull request در [GitHub](https://github.com) باز کنید. انتشار PyPI فقط با مسیر دستی و تأیید صریح انجام می شود؛ target عادی [release](#) صرفاً artifact محلی می سازد و upload نمی کند.

مجوز و ارتباط

این پروژه تحت [GNU General Public License v3](#) منتشر می‌شود.

- وبسایت: algotik.com
- تلگرام: t.me/algotik
- نویسنده: Mohsen Alipour — alipour@algotik.ir

تاریخچه تغییرات

این بخش در هر انتشار به‌روزرسانی می‌شود؛ جزئیات کامل‌تر در [HISTORY.rst](#) ثبت شده است.

- **1.8.0 – 2026-09-13:** افزودن شناسایی و فیلتر مستقیم اراد/گام و سررسیدهای شش/هشت‌رقمی، YTM مرجع رسمی همه اوراق بدون حدس جریان نقدی، و توسعه TGLU به کاتالوگ ۸۷ سری شامل ۳۶ ارز آزاد، ۲۴ نرخ رسمی، طلای ۱۸/۲۴ عیار و سایر طلاها، نقره، سکه/حباب و فلزات جهانی با alias فارسی، slug رسمی، واحد و خطاهای typed.
- **1.7.0 – 2026-09-13:** افزودن مقایسه چندنمادی بازده/ریسک/افت/بنا/نقدشوندگی و جریان حقیقی؛ سنج‌های نقدشوندگی تاریخی و زنده با Amihud، گردش، spread و عمق پنج‌سطحی؛ رژیم توضیح‌پذیر بازار از پنج مؤلفه با confidence و رفتار صریح داده مفقود؛ داده و treemap نقشه بازار با گروه‌بندی و معیارهای اندازه/رنگ قابل انتخاب؛ پشتیبانی ورودی DataFrame صریح برای schema/atrrs، replay پایدار، واحدهای روشن و Plotly اختیاری.
- **1.6.0 – 2026-09-13:** افزودن حراج‌های بورس انرژی با شرایط کامل عرضه و تفکیک معاملات حراج از سابقه ابزار؛ تابلوهای برق استاندارد/سبز/آزاد، سلف و گواهی‌ها و مشخصات آتی انرژی؛ تابلوی رسمی بورس کالا برای گواهی، سلف، صندوق و آتی با پنج سطح سفارش و حقیقی/حقوقی؛ تاریخچه و خلاصه بازار فیزیکی و آمار تجمیعی بازارهای فیزیکی/آتی/اختیار/مالی؛ منحنی سررسید، calendar spread و cash-and-carry با فرض‌های صریح و ثبت دقیق واحد ریال/هزار ریال/میلیون ریال.
- **1.5.1 – 2026-09-13:** اصلاح هویت و `change_amount` در `get_shareholders(date=...)` بر پایه اختلاف دو snapshot و تطبیق exact؛ تفکیک صریح تاریخ معامله از تاریخ مؤثر روز معاملاتی بعد؛ افزودن `get_shareholder_history()` با فرکانس روزانه/هفتگی/ماه شمسی و بودجه سخت درخواست؛ افزودن رتبه‌بندی انباشت/توزیع، edge list شبکه سهامدار-نماد و سنج‌های Top-N/HHI تمرکز مالکیت با metadata کامل محدودیت‌های رسمی منبع.
- **1.5.0 – 2026-09-12:** تفکیک صریح تاریخچه بلندمدت هر نماد از پنجره گردان حداکثر پنج‌تاریخ بازار؛ افزودن snapshot، تغییرات متوالی و جمع‌بندی سهامداران فعال با اتصال دقیق `InsCode` و metadata محدودیت منبع؛ افزودن timeline بومی TSETMC برای پیام ناظر، تغییر سرمایه و تعدیل قیمت بدون برچسب‌گذاری ساختگی DPS یا ادغام داده کدال.
- **1.4.0 – 2026-09-12:** تجمیع تقویم و نمای کلان بازار با ابزارهای پیش‌گشایش؛ تقویم رسمی بورس/فرا بورس، ارزش بازار تاریخی، فعالیت روزانه، اثر نمادها بر شاخص، خلاصه bulk روز، مستر کامل ابزارها و diff مبتنی بر TOP، `InsCode` رسمی مشترکان و عدم تعادل پیش‌گشایش.
- **1.3.0 – 2026-09-12:** طبقه‌بندی چندمحوره صندوق‌ها، اصلاح شناسه‌های registry، افزودن پنج دسته جدید، پوشش صندوق‌های بورسی نوع ۳۸۰، فیلتر طلا/نقره/زعفران و شواهد قابل ممیزی.

- **1.2.4 – 2026-09-02:** رویدادهای افزوده/حذف‌شدهٔ اعضای شاخص صنعت.
- **1.2.3 – 2026-09-01:** churn، همسایگی همبستگی و امتیاز سلامت صنایع.
- **1.2.2 – 2026-09-01:** سنجش همپوشانی اعضای رسمی صنایع.
- **1.2.1 – 2026-09-01:** مقایسه، قدرت نسبی و همبستگی صنایع.
- **1.2.0 – 2026-09-01:** API، کامل شاخص‌های صنعت، اعضا، تاریخچه.
- **1.1.3 – 2026-08-28:** بازطراحی راهنمای PDF و هماهنگ‌سازی badge نسخه.
- **1.1.2 – 2026-08-28:** بازطراحی README و تکمیل قرارداد ورودی/خروجی توابع.
- **1.1.1 – 2026-08-25:** بهبود نمایش فارسی و تکمیل badgeهای پروژه.
- **1.1.0 – 2026-08-24:** دادهٔ زنده/تاریخی، SQLite، watcher، اخزا، منحنی بازده و اختیار معامله.
- **1.0.3 – 2026-07-12:** جست‌وجوی املای canonical نماد با نرمال‌سازی فارسی/عربی.
- **1.0.2 – 2026-03-24:** گسترش شاخص‌های صنعت و افزودن فهرست شاخص‌ها و اعضا.
- **1.0.1 – 2026-02-19:** تکمیل وابستگی‌ها و اصلاح نوع دادهٔ `NAV_Discount`.
- **1.0.0 – 2025-07-12:** انتشار پایدار با HTTP client مرکزی، تنظیمات و exception‌های اختصاصی.
- **0.3.12 تا 0.2.8 – 2023 تا 2024:** نسخه‌های آغازین قیمت، سهامداران، افزایش سرمایه، اطلاعات نماد، ارز و فهرست ابزارها.